Kotlin 语言指南

Kotlin 官方文档中文版 · 对应 Kotlin 2.1.21

原文：kotlinlang.org/docs · 中文：book.kotlincn.net（github.com/hltj/kotlin-web-site-cn，Apache 2.0）

2026-04 版本 · 2026-10-13 排版

目录

语言指南

- 基本语法
- 习惯用法
- 编码规范
- 基本类型
- 数字
- 无符号整型
- 布尔
- 字符
- 字符串
- 数组
- 类型检测与类型转换
- 条件与循环
- 返回与跳转
- 包与导入
- 类
- 继承
- 属性
- 接口
- 函数式（SAM）接口
- 可见性修饰符
- 扩展
- 数据类
- 泛型：in、out、where
- 嵌套类与内部类
- 枚举类
- Inline value classes
- 对象声明与对象表达式
- 委托
- 属性委托
- 类型别名
- 函数
- 高阶函数与 lambda 表达式
- 内联函数
- 操作符重载
- 类型安全的构建器
- 空安全
- 相等性

this 表达式
异步程序设计
注解
解构声明
反射

Android

Kotlin 用于 Android 开发

语言指南

基本语法

这是一组基本语法元素及示例。在每段的末尾都有一个指向相关主题详述的链接。

也可以通过 JetBrains 学院的免费 Kotlin 核心课程学习所有 Kotlin 要领。

包的定义与导入

包的声明应处于源文件顶部：

```
package my.demo

import kotlin.text.*

// ……
```

目录与包的结构无需匹配：源代码可以在文件系统的任意位置。

参见包。

程序入口点

Kotlin 应用程序的入口点是 `main` 函数：

```
fun main() {
    println("Hello world!")
}
```

`main` 的另一种形式接受可变数量的 `String` 参数：

```
fun main(args: Array<String>) {
    println(args.contentToString())
}
```

输出打到标准输出

`print` 将其参数打到标准输出：

```
fun main() {
    print("Hello ")
    print("world!")
}
```

`println` 输出其参数并添加换行符，以便接下来输出的内容出现在下一行：

```
fun main() {
    println("Hello world!")
    println(42)
}
```

Read from the standard input

The `readln()` function reads from the standard input. This function reads the entire line the user enters as a string.

You can use the `println()`, `readln()`, and `print()` functions together to print messages requesting and showing user input:

```
// Prints a message to request input
println("Enter any word: ")

// Reads and stores the user input. For example: Happiness
val yourWord = readln()

// Prints a message with the input
print("You entered the word: ")
print(yourWord)
// You entered the word: Happiness
```

For more information, see Read standard input.

函数

带有两个 `Int` 参数、返回 `Int` 的函数：

```
fun sum(a: Int, b: Int): Int {
    return a + b
}
fun main() {
    print("sum of 3 and 5 is ")
    println(sum(3, 5))
}
```

函数体可以是表达式。其返回类型可以推断出来：

```
fun sum(a: Int, b: Int) = a + b
fun main() {
    println("sum of 19 and 23 is ${sum(19, 23)}")
}
```

返回无意义的值的函数：

```
fun printSum(a: Int, b: Int): Unit {
    println("sum of $a and $b is ${a + b}")
}
fun main() {
    printSum(-1, 8)
}
```

`Unit` 返回类型可以省略：

```
fun printSum(a: Int, b: Int) {
    println("sum of $a and $b is ${a + b}")
}
fun main() {
    printSum(-1, 8)
}
```

参见函数。

变量

In Kotlin, you declare a variable starting with a keyword, `val` or `var`, followed by the name of the variable.

Use the `val` keyword to declare variables that are assigned a value only once. These are immutable, read-only local variables that can’t be reassigned a different value after initialization:

```
fun main() {
    // Declares the variable x and initializes it with the value of 5
    val x: Int = 5
    // 5
    println(x)
}
```

Use the `var` keyword to declare variables that can be reassigned. These are mutable variables, and you can change their values after initialization:

```
fun main() {
    // Declares the variable x and initializes it with the value of 5
    var x: Int = 5
    // Reassigns a new value of 6 to the variable x
    x += 1
    // 6
    println(x)
}
```

Kotlin supports type inference and automatically identifies the data type of a declared variable. When declaring a variable, you can omit the type after the variable name:

```
fun main() {
    // Declares the variable x with the value of 5;`Int` type is inferred
    val x = 5
    // 5
    println(x)
}
```

You can use variables only after initializing them. You can either initialize a variable at the moment of declaration or declare a variable first and initialize it later. In the second case, you must specify the data type:

```
fun main() {
    // Initializes the variable x at the moment of declaration; type is not required
    val x = 5
    // Declares the variable c without initialization; type is required
    val c: Int
    // Initializes the variable c after declaration
    c = 3
    // 5
    // 3
    println(x)
    println(c)
}
```

可以在顶层声明变量：

```
val PI = 3.14
var x = 0

fun incrementX() {
    x += 1
}
// x = 0; PI = 3.14
// incrementX()
// x = 1; PI = 3.14
fun main() {
    println("x = $x; PI = $PI")
    incrementX()
    println("incrementX()")
    println("x = $x; PI = $PI")
}
```

For information about declaring properties, see Properties.

创建类与实例

使用 `class` 关键字定义类：

```
class Shape
```

类的属性可以在其声明或主体中列出：

```
class Rectangle(val height: Double, val length: Double) {
    val perimeter = (height + length) * 2
}
```

具有类声明中所列参数的默认构造函数会自动可用：

```
class Rectangle(val height: Double, val length: Double) {
    val perimeter = (height + length) * 2
}
fun main() {
    val rectangle = Rectangle(5.0, 2.0)
    println("The perimeter is ${rectangle.perimeter}")
}
```

类之间继承由冒号（`:`）声明。默认情况下类都是 `final` 的；如需让一个类可继承， 请将其标记为 `open`：

```
open class Shape

class Rectangle(val height: Double, val length: Double): Shape() {
    val perimeter = (height + length) * 2
}
```

For more information about constructors and inheritance, see Classes and Objects and instances.

注释

与大多数现代语言一样，Kotlin 支持单行（或*行末*）与多行（*块*）注释：

```
// 这是一个行注释

/* 这是一个多行的
   块注释。 */
```

Kotlin 中的块注释可以嵌套：

```
/* 注释从这里开始
/* 包含嵌套的注释 */
并且在这里结束。 */
```

参见编写 Kotlin 代码文档 查看关于文档注释语法的信息。

字符串模板

```
fun main() {
    var a = 1
```

```
    // 模板中的简单名称:
    val s1 = "a is $a"

    a = 2
    // 模板中的任意表达式:
    val s2 = "${s1.replace("is", "was")}, but now is $a"
    println(s2)
}
```

参见字符串模板。

条件表达式

```
fun maxOf(a: Int, b: Int): Int {
    if (a > b) {
        return a
    } else {
        return b
    }
}
fun main() {
    println("max of 0 and 42 is ${maxOf(0, 42)}")
}
```

在 Kotlin 中，`if` 也可以用作表达式：

```
fun maxOf(a: Int, b: Int) = if (a > b) a else b
fun main() {
    println("max of 0 and 42 is ${maxOf(0, 42)}")
}
```

参见`if` 表达式。

for 循环

```
fun main() {
    val items = listOf("apple", "banana", "kiwifruit")
    for (item in items) {
        println(item)
    }
}
```

或者：

```
fun main() {
    val items = listOf("apple", "banana", "kiwifruit")
    for (index in items.indices) {
        println("item at $index is ${items[index]}")
```

```
    }
}
```

参见 for 循环。

while 循环

```
fun main() {
    val items = listOf("apple", "banana", "kiwifruit")
    var index = 0
    while (index < items.size) {
        println("item at $index is ${items[index]}")
        index++
    }
}
```

参见 while 循环。

when 表达式

```
fun describe(obj: Any): String =
    when (obj) {
        1          -> "One"
        "Hello"    -> "Greeting"
        is Long    -> "Long"
        !is String -> "Not a string"
        else       -> "Unknown"
    }
fun main() {
    println(describe(1))
    println(describe("Hello"))
    println(describe(1000L))
    println(describe(2))
    println(describe("other"))
}
```

See when expressions and statements.

使用区间（range）

使用 `in` 操作符来检测某个数字是否在指定区间内：

```
fun main() {
    val x = 10
    val y = 9
    if (x in 1..y+1) {
        println("fits in range")
```

```
    }
}
```

检测某个数字是否在指定区间外：

```
fun main() {
    val list = listOf("a", "b", "c")

    if (-1 !in 0..list.lastIndex) {
        println("-1 is out of range")
    }
    if (list.size !in list.indices) {
        println("list size is out of valid list indices range, too")
    }
}
```

区间迭代：

```
fun main() {
    for (x in 1..5) {
        print(x)
    }
}
```

或数列迭代：

```
fun main() {
    for (x in 1..10 step 2) {
        print(x)
    }
    println()
    for (x in 9 downTo 0 step 3) {
        print(x)
    }
}
```

参见区间与数列。

集合

对集合进行迭代：

```
fun main() {
    val items = listOf("apple", "banana", "kiwifruit")
    for (item in items) {
        println(item)
    }
}
```

使用 `in` 操作符来判断集合内是否包含某实例：

```
fun main() {
    val items = setOf("apple", "banana", "kiwifruit")
    when {
        "orange" in items -> println("juicy")
        "apple" in items -> println("apple is fine too")
    }
}
```

使用 lambda 表达式来过滤（filter）与映射（map）集合：

```
fun main() {
    val fruits = listOf("banana", "avocado", "apple", "kiwifruit")
    fruits
      .filter { it.startsWith("a") }
      .sortedBy { it }
      .map { it.uppercase() }
      .forEach { println(it) }
}
```

参见集合概述。

空值与空检测

当可能用 `null` 值时，必须将引用显式标记为可空。可空类型名称以问号（`?`）结尾。

如果 `str` 的内容不是数字返回 `null`：

```
fun parseInt(str: String): Int? {
    // ……
}
```

使用返回可空值的函数：

```
fun parseInt(str: String): Int? {
    return str.toIntOrNull()
}
fun printProduct(arg1: String, arg2: String) {
    val x = parseInt(arg1)
    val y = parseInt(arg2)

    // 直接使用 `x * y` 会导致编译错误，因为它们可能为 null
    if (x != null && y != null) {
        // 在空检测后，x 与 y 会自动转换为非空值（non-nullable）
        println(x * y)
    }
    else {
        println("'$arg1' or '$arg2' is not a number")
    }
}
fun main() {
    printProduct("6", "7")
```

```
    printProduct("a", "7")
    printProduct("a", "b")
}
```

或者：

```
fun parseInt(str: String): Int? {
    return str.toIntOrNull()
}

fun printProduct(arg1: String, arg2: String) {
    val x = parseInt(arg1)
    val y = parseInt(arg2)
    // ……
    if (x == null) {
        println("Wrong number format in arg1: '$arg1'")
        return
    }
    if (y == null) {
        println("Wrong number format in arg2: '$arg2'")
        return
    }

    // 在空检测后，x 与 y 会自动转换为非空值
    println(x * y)
}

fun main() {
    printProduct("6", "7")
    printProduct("a", "7")
    printProduct("99", "b")
}
```

参见空安全。

类型检测与自动类型转换

`is` 操作符检测一个表达式是否某类型的一个实例。 如果一个不可变的局部变量或属性已经判断出为某类型，那么检测后的分支中可以直接当作该类型使用，无需显式转换：

```
fun getStringLength(obj: Any): Int? {
    if (obj is String) {
        // `obj` 在该条件分支内自动转换成 `String`
        return obj.length
    }

    // 在离开类型检测分支后，`obj` 仍然是 `Any` 类型
    return null
}
fun main() {
    fun printLength(obj: Any) {
        println("Getting the length of '$obj'. Result: ${getStringLength(obj) ?: "Error: The
```

```
object is not a string"} ")
    }
    printLength("Incomprehensibilities")
    printLength(1000)
    printLength(listOf(Any()))
}
```

或者：

```
fun getStringLength(obj: Any): Int? {
    if (obj !is String) return null

    // `obj` 在这一分支自动转换为 `String`
    return obj.length
}
fun main() {
    fun printLength(obj: Any) {
        println("Getting the length of '$obj'. Result: ${getStringLength(obj) ?: "Error: The
object is not a string"} ")
    }
    printLength("Incomprehensibilities")
    printLength(1000)
    printLength(listOf(Any()))
}
```

甚至：

```
fun getStringLength(obj: Any): Int? {
    // `obj` 在 `&&` 右边自动转换成 `String` 类型
    if (obj is String && obj.length > 0) {
        return obj.length
    }

    return null
}
fun main() {
    fun printLength(obj: Any) {
        println("Getting the length of '$obj'. Result: ${getStringLength(obj) ?: "Error: The
object is not a string"} ")
    }
    printLength("Incomprehensibilities")
    printLength("")
    printLength(1000)
}
```

参见类以及类型转换。

习惯用法

一些在 Kotlin 中广泛使用的语法习惯，如果你有更喜欢的语法习惯或者风格，建一个 pull request 贡献给我们吧！

创建 DTO（POJO/POCO）

```
data class Customer(val name: String, val email: String)
```

会为 `Customer` 类提供以下功能：

- 所有属性的 getter （对于 `var` 定义的还有 setter）
- `equals()`
- `hashCode()`
- `toString()`
- `copy()`
- 所有属性的 `component1()`、 `component2()`……等等（参见数据类）

函数的默认实参

```
fun foo(a: Int = 0, b: String = "") { …… }
```

过滤 list

```
val positives = list.filter { x -> x > 0 }
```

或者可以更短：

```
val positives = list.filter { it > 0 }
```

了解 Java 与 Kotlin 过滤的区别。

检测元素是否存在于集合中

```
if ("john@example.com" in emailsList) { …… }

if ("jane@example.com" !in emailsList) { …… }
```

字符串内插

```
println("Name $name")
```

了解 Java 与 Kotlin 字符串连接的区别。

Read standard input safely

```
// Reads a string and returns null if the input can't be converted into an integer. For exam
ple: Hi there!
val wrongInt = readln().toIntOrNull()
println(wrongInt)
// null

// Reads a string that can be converted into an integer and returns an integer. For example:
13
val correctInt = readln().toIntOrNull()
println(correctInt)
// 13
```

For more information, see Read standard input.

类型判断

```
when (x) {
    is Foo -> ……
    is Bar -> ……
    else   -> ……
}
```

只读 list

```
val list = listOf("a", "b", "c")
```

只读 map

```
val map = mapOf("a" to 1, "b" to 2, "c" to 3)
```

访问 map 条目

```
println(map["key"])
map["key"] = value
```

遍历 map 或者 pair 型 list

```
for ((k, v) in map) {
    println("$k -> $v")
}
```

`k` 与 `v` 可以是任何适宜的名称，例如 `name` 与 `age`。

区间迭代

```
for (i in 1..100) { …… }  // 闭区间：包含 100
for (i in 1..< 100) { …… } // 左闭右开区间：不包含 100
for (x in 2..10 step 2) { …… }
for (x in 10 downTo 1) { …… }
(1..10).forEach { …… }
```

延迟属性

```
val p: String by lazy { // 该值仅在首次访问时计算
    // 计算该字符串
}
```

扩展函数

```
fun String.spaceToCamelCase() { …… }

"Convert this to camelcase".spaceToCamelCase()
```

创建单例

```
object Resource {
    val name = "Name"
}
```

Use inline value classes for type-safe values

```
@JvmInline
value class EmployeeId(private val id: String)

@JvmInline
value class CustomerId(private val id: String)
```

If you accidentally mix up `EmployeeId` and `CustomerId`, a compilation error is triggered.

注意: The `@JvmInline` annotation is only needed for JVM backends.

实例化一个抽象类

```
abstract class MyAbstractClass {
    abstract fun doSomething()
    abstract fun sleep()
}

fun main() {
    val myObject = object : MyAbstractClass() {
        override fun doSomething() {
            // ……
        }

        override fun sleep() { // ……
        }
    }
    myObject.doSomething()
}
```

if-not-null 缩写

```
val files = File("Test").listFiles()

println(files?.size) // 如果 files 不是 null，那么输出其大小（size）
```

if-not-null-else 缩写

```
val files = File("Test").listFiles()

// For simple fallback values:
println(files?.size ?: "empty") // 如果 files 为 null，那么输出“empty”

// 如需在代码块中计算更复杂的备用值，请使用 `run`
val filesSize = files?.size ?: run {
    val someSize = getSomeSize()
    someSize * 2
}
println(filesSize)
```

if null 执行一个语句

```
val values = ……
val email = values["email"] ?: throw IllegalStateException("Email is missing!")
```

在可能会空的集合中取第一元素

```
val emails = …… // 可能会是空集合
val mainEmail = emails.firstOrNull() ?: ""
```

了解 Java 与 Kotlin 获取第一元素的差别。

if not null 执行代码

```
val value = ……

value?.let {
    …… // 如果非空会执行这个代码块
}
```

映射可空值（如果非空的话）

```
val value = ……

val mapped = value?.let { transformValue(it) } ?: defaultValue
// 如果该值或其转换结果为空，那么返回 defaultValue。
```

返回 when 表达式

```
fun transform(color: String): Int {
    return when (color) {
        "Red" -> 0
        "Green" -> 1
        "Blue" -> 2
        else -> throw IllegalArgumentException("Invalid color param value")
    }
}
```

try-catch 表达式

```
fun test() {
    val result = try {
        count()
    } catch (e: ArithmeticException) {
        throw IllegalStateException(e)
    }

    // 使用 result
}
```

if 表达式

```
val y = if (x == 1) {
    "one"
} else if (x == 2) {
    "two"
} else {
    "other"
}
```

返回类型为 Unit 的方法的构建器风格用法

```
fun arrayOfMinusOnes(size: Int): IntArray {
    return IntArray(size).apply { fill(-1) }
}
```

单表达式函数

```
fun theAnswer() = 42
```

等价于

```
fun theAnswer(): Int {
    return 42
}
```

单表达式函数与其它惯用法一起使用能简化代码，例如和 `when` 表达式一起使用：

```
fun transform(color: String): Int = when (color) {
    "Red" -> 0
    "Green" -> 1
    "Blue" -> 2
    else -> throw IllegalArgumentException("Invalid color param value")
}
```

对一个对象实例调用多个方法 （with）

```
class Turtle {
    fun penDown()
    fun penUp()
    fun turn(degrees: Double)
    fun forward(pixels: Double)
}

val myTurtle = Turtle()
with(myTurtle) { // 画一个 100 像素的正方形
    penDown()
```

```
    for (i in 1..4) {
        forward(100.0)
        turn(90.0)
    }
    penUp()
}
```

配置对象的属性（apply）

```
val myRectangle = Rectangle().apply {
    length = 4
    breadth = 5
    color = 0xFAFAFA
}
```

这对于配置未出现在对象构造函数中的属性非常有用。

Java 7 的 try-with-resources

```
val stream = Files.newInputStream(Paths.get("/some/file.txt"))
stream.buffered().reader().use { reader ->
    println(reader.readText())
}
```

需要泛型信息的泛型函数

```
//  public final class Gson {
//     ……
//     public <T> T fromJson(JsonElement json, Class<T> classOfT) throws JsonSyntaxException
{
//     ……

inline fun <reified T: Any> Gson.fromJson(json: JsonElement): T = this.fromJson(json, T::cla
ss.java)
```

交换两个变量

```
var a = 1
var b = 2
a = b.also { b = a }
```

将代码标记为不完整（TODO）

Kotlin 的标准库有一个 `TODO()` 函数，该函数总是抛出一个 `NotImplementedError`。 其返回类型为 `Nothing`，因此无论预期类型是什么都可以使用它。 还有一个接受原因参数的重载：

```
fun calcTaxes(): BigDecimal = TODO("Waiting for feedback from accounting")
```

IntelliJ IDEA 的 kotlin 插件理解 `TODO()` 的语言，并且会自动在 TODO 工具窗口中添加代码指示。

下一步做什么?

- 使用地道 Kotlin 风格求解 Advent of Code 谜题。
- 了解如何在 Java 与 Kotlin 中处理字符串的典型任务。
- 了解如何在 Java 与 Kotlin 中处理集合的典型任务。
- 了解如何在 Java 与 Kotlin 中处理可空性。

编码规范

众所周知且易于遵循的编码规范对于任何编程语言都至关重要。 在这里，我们为使用 Kotlin 的项目提供关于代码风格与代码组织的准则。

在 IDE 中配置风格

两个最流行的 Kotlin IDE——IntelliJ IDEA 与 Android Studio 都为代码风格提供了强大的支持。 可以将它们配置为根据指定的代码风格自动格式化代码。

应用风格指南

1. 转到 **Settings/Preferences | Editor | Code Style | Kotlin**。
2. 点击 **Set from...**。
3. 选择 **Kotlin style guide** 。

验证代码是否遵循风格指南

1. 转到 **Settings/Preferences | Editor | Inspections | General**。
2. 打开 **Incorrect formatting** 探查项。 默认启用验证样式指南中描述的其他问题（例如命名约定）的附加探查项。

源代码组织

目录结构

在纯 Kotlin 项目中，推荐的目录结构遵循省略了公共根包的包结构。例如，如果项目中的所有代码都位于 `org.example.kotlin` 包及其子包中，那么 `org.example.kotlin` 包的文件应该直接放在源代码根目录下，而 `org.example.kotlin.network.socket` 中的文件应该放在源代码根目录下的 `network/socket` 子目录中。

> **注意:** 对于 JVM 平台：Kotlin 源文件应当与 Java 源文件位于同一源文件根目录下， 并遵循相同的目录结构（每个文件应存储在与其 package 语句对应的目录中。

源文件名称

如果 Kotlin 文件包含单个类或接口（以及可能相关的顶层声明），那么文件名应该与该类的名称相同，并追加 `.kt` 扩展名。 这适用于所有类型的类和接口。 如果文件包含多个类或只包含顶层声明， 那么选择一个描述该文件所包含内容的名称，并以此命名该文件。 使用首字母大写的驼峰风格，其中每个单词的首字母都大写。 例如 `ProcessDeclarations.kt`。

文件的名称应该描述文件中代码的作用。因此，应避免在文件名中使用诸如 `Util` 之类的无意义词语。

多平台项目

在多平台项目中，平台特有的源代码集中包含顶层声明的文件应具有与该源代码集名称相关联的后缀。 例如：

- **jvm**Main/kotlin/Platform.**jvm**.kt
- **android**Main/kotlin/Platform.**android**.kt
- **ios**Main/kotlin/Platform.**ios**.kt

对于公共源代码集，包含顶层声明的文件不应该有后缀。 例如，`commonMain/kotlin/Platform.kt`.

技术细节

由于 JVM 的限制〔不允许顶层成员（函数、属性）〕，我们建议在多平台项目中遵循这种文件命名方案。

为了解决 JVM 的这个问题，Kotlin JVM 编译器会创建包含顶层成员声明的包装类〔所谓的 “文件门面”（file facades）〕。 文件门面有一个源自文件名的内部名称。

另外，JVM 不允许多个类具有相同的完整限定名称 (FQN，fully qualified name)。 这可能会导致 Kotlin 项目无法编译到 JVM 的情况：

```
root
|- commonMain/kotlin/myPackage/Platform.kt // 包含 'fun count() { }'
|- jvmMain/kotlin/myPackage/Platform.kt // 包含 'fun multiply() { }'
```

这里两个 `Platform.kt` 文件都在同一个包中，因此 Kotlin JVM 编译器生成两个文件门面，它们都有 FQN `myPackage.PlatformKt`。 这就会产生 "Duplicate JVM classes"（“重复的 JVM 类”）错误。

避免这种情况的最简单的方式就是按照上述指南重命名其中一个文件。 这种命名方案有助于避免冲突，同时保持代码的可读性。

> **提示:** 对于以下两种场景，上述建议可能看起来多余，但我们仍然建议遵循之：
>
> - 非 JVM 平台不存在重复文件门面的问题。 但是，这种命名方案有助于保持文件命名的一致性。
> - 在 JVM 中，如果源文件没有顶层声明就不会生成文件门面，也就不会遇到命名冲突。
>
>   但是，这种命名方案有助于避免简单的重构或添加可能包含顶层函数并同样导致 "Duplicate JVM classes"（“重复 JVM 类”错误）的情况。

源文件组织

鼓励多个声明（类、顶级函数或者属性）放在同一个 Kotlin 源文件中， 只要这些声明在语义上彼此紧密关联，并且文件保持合理大小 （不超过几百行）。

特别是在为类定义与类的所有客户都相关的扩展函数时， 请将它们放在与类自身相同的地方。而在定义仅对指定客户有意义的扩展函数时，请将它们放在紧挨该客户代码之后。避免只是为了保存 某个类的所有扩展函

数而创建文件。

类布局

一个类的内容应按以下顺序排列：

1. 属性声明与初始化块
2. 次构造函数
3. 方法声明
4. 伴生对象

不要按字母顺序或者可见性对方法声明排序，也不要将常规方法与扩展方法分开。而是要把相关的东西放在一起，这样从上到下阅读类的人就能够跟进所发生事情的逻辑。选择一个顺序（高级别优先，或者相反）并坚持下去。

将嵌套类放在紧挨使用这些类的代码之后。如果打算在外部使用嵌套类，而且类中并没有引用这些类，那么把它们放到末尾，在伴生对象之后。

接口实现布局

在实现一个接口时，实现成员的顺序应该与该接口的成员顺序相同（如果需要， 还要插入用于实现的额外的私有方法）。

重载布局

在类中总是将重载放在一起。

命名规则

在 Kotlin 中，包名与类名的命名规则非常简单：

- 包的名称总是小写且不使用下划线（`org.example.project`）。 通常不鼓励使用多个词的名称，但是如果确实需要使用多个词，可以将它们连接在一起或使用驼峰风格（`org.example.myProject`）。
- 类与对象的名称使用首字母大写的驼峰风格：

```
open class DeclarationProcessor { /*……*/ }

object EmptyDeclarationProcessor : DeclarationProcessor() { /*……*/ }
```

函数名

函数、属性与局部变量的名称以小写字母开头、使用驼峰风格而不使用下划线：

```
fun processDeclarations() { /*……*/ }
var declarationCount = 1
```

例外：用于创建类实例的工厂函数可以与抽象返回类型具有相同的名称：

```
interface Foo { /*……*/ }

class FooImpl : Foo { /*……*/ }

fun Foo(): Foo { return FooImpl() }
```

测试方法的名称

当且仅当在测试中，可以使用反引号括起来的带空格的方法名。 请注意，API 级别在 30 及以上的 Android 运行时才支持这样的方法名。 测试代码中也允许方法名使用下划线。

```
class MyTestCase {
    @Test fun `ensure everything works`() { /*……*/ }

    @Test fun ensureEverythingWorks_onAndroid() { /*……*/ }
}
```

属性名

常量名称（标有 `const` 的属性，或者保存不可变数据的没有自定义 `get` 函数的顶层/对象 `val` 属性）应该使用全大写、下划线分隔的名称，遵循 screaming snake case 约定：

```
const val MAX_COUNT = 8
val USER_NAME_FIELD = "UserName"
```

保存带有行为的对象或者可变数据的顶层/对象属性的名称应该使用驼峰风格名称：

```
val mutableCollection: MutableSet<String> = HashSet()
```

保存单例对象引用的属性的名称可以使用与 `object` 声明相同的命名风格：

```
val PersonComparator: Comparator<Person> = /*……*/
```

对于枚举常量，可以使用全大写、下划线分隔的 (screaming snake case) 名称 （`enum class Color { RED, GREEN }`）也可使用首字母大写的常规驼峰名称，具体取决于用途。

幕后属性的名称

如果一个类有两个概念上相同的属性，一个是公共 API 的一部分，另一个是实现细节，那么使用下划线作为私有属性名称的前缀：

```
class C {
    private val _elementList = mutableListOf<Element>()

    val elementList: List<Element>
```

```
        get() = _elementList
}
```

选择好名称

类的名称通常是用来解释类*是*什么的名词或者名词短语：`List`、 `PersonReader`。

方法的名称通常是动词或动词短语，说明该方法*做*什么：`close`、 `readPersons`。 修改对象或者返回一个新对象的名称也应遵循建议。例如 `sort` 是对一个集合就地排序，而 `sorted` 是返回一个排序后的集合副本。

名称应该表明实体的目的是什么，所以最好避免在名称中使用无意义的单词 （`Manager`、 `Wrapper`）。

当使用首字母缩写作为声明名称的一部分时，请遵循以下规则：

- 对于两个字母的缩写，两字母都大写。例如， `IOStream`。
- 对于超过两个字母的缩写，只大写第一个字母。例如， `XmlFormatter` 或 `HttpInputStream`。

格式化

缩进

使用 4 个空格缩进。不要使用 tab。

对于花括号，将左花括号放在结构起始处的行尾，而将右花括号放在与左括结构横向对齐的单独一行。

```
if (elements != null) {
    for (element in elements) {
        // ……
    }
}
```

> **注意:** 在 Kotlin 中，分号是可选的，因此换行很重要。语言设计采用 Java 风格的花括号格式，如果尝试使用不同的格式化风格，那么可能会遇到意外的行为。

横向空白

- 在二元操作符左右留空格（`a + b`）。例外：不要在“range to”操作符（`0..i`）左右留空格。
- 不要在一元运算符左右留空格（`a++`）。
- 在控制流关键字（`if`、 `when`、 `for` 以及 `while`）与相应的左括号之间留空格。
- 不要在主构造函数声明、方法声明或者方法调用的左括号之前留空格。

```
class A(val x: Int)

fun foo(x: Int) { …… }

fun bar() {
```

```
    foo(1)
}
```

- 绝不在 `(`、 `[` 之后或者 `]`、 `)` 之前留空格。
- 绝不在 `.` 或者 `?.` 左右留空格：`foo.bar().filter { it > 2 }.joinToString()`，`foo?.bar()`。
- 在 `//` 之后留一个空格：`// 这是一条注释`。
- 不要在用于指定类型参数的尖括号前后留空格：`class Map<K, V> { …… }`。
- 不要在 `::` 前后留空格：`Foo::class`、 `String::length`。
- 不要在用于标记可空类型的 `?` 前留空格：`String?`。

作为一般规则，避免任何类型的水平对齐。将标识符重命名为不同长度的名称不应该影响声明或者任何用法的格式。

冒号

在以下场景中的 `:` 之前留一个空格：

- 当它用于分隔类型与超类型时。
- 当委托给一个超类的构造函数或者同一类的另一个构造函数时。
- 在 `object` 关键字之后。

而当分隔声明与其类型时，不要在 `:` 之前留空格。

在 `:` 之后总要留一个空格。

```
abstract class Foo<out T : Any> : IFoo {
    abstract fun foo(a: Int): T
}

class FooImpl : Foo() {
    constructor(x: String) : this(x) { /*……*/ }

    val x = object : IFoo { /*……*/ }
}
```

类头

具有少数主构造函数参数的类可以写成一行：

```
class Person(id: Int, name: String)
```

具有较长类头的类应该格式化，以使每个主构造函数参数都在带有缩进的独立的行中。 另外，右括号应该位于一个新行上。如果使用了继承，那么超类的构造函数调用或者所实现接口的列表应该与右括号位于同一行：

```
class Person(
    id: Int,
    name: String,
```

```
    surname: String
) : Human(id, name) { /*……*/ }
```

对于多个接口，应该将超类构造函数调用放在首位，然后将每个接口应放在不同的行中：

```
class Person(
    id: Int,
    name: String,
    surname: String
) : Human(id, name),
    KotlinMaker { /*……*/ }
```

对于具有很长超类型列表的类，在冒号后面换行，并横向对齐所有超类型名：

```
class MyFavouriteVeryLongClassHolder :
    MyLongHolder<MyFavouriteVeryLongClass>(),
    SomeOtherInterface,
    AndAnotherOne {

    fun foo() { /*……*/ }
}
```

为了将类头与类体分隔清楚，当类头很长时，可以在类头后放一空行 （如上例所示）或者将左花括号放在独立行上：

```
class MyFavouriteVeryLongClassHolder :
    MyLongHolder<MyFavouriteVeryLongClass>(),
    SomeOtherInterface,
    AndAnotherOne
{
    fun foo() { /*……*/ }
}
```

构造函数参数使用常规缩进（4 个空格）。这确保了在主构造函数中声明的属性与 在类体中声明的属性具有相同的缩进。

修饰符

如果一个声明有多个修饰符，请始终按照以下顺序安放：

```
public / protected / private / internal
expect / actual
final / open / abstract / sealed / const
external
override
lateinit
tailrec
vararg
suspend
inner
enum / annotation / fun // 在 `fun interface` 中是修饰符
```

```
companion
inline/ value
infix
operator
data
```

将所有注解放在修饰符前：

```
@Named("Foo")
private val foo: Foo
```

除非你在编写库，否则请省略多余的修饰符（例如 `public`）。

注解

通常将注解放在单独的行上，在它们所依附的声明之前，并使用相同的缩进：

```
@Target(AnnotationTarget.PROPERTY)
annotation class JsonExclude
```

无参数的注解可以放在同一行：

```
@JsonExclude @JvmField
var x: String
```

无参数的单个注解可以与相应的声明放在同一行：

```
@Test fun foo() { /*……*/ }
```

文件注解

文件注解位于文件注释（如果有的话）之后、`package` 语句之前， 并且用一个空白行与 `package` 分开（为了强调其针对文件而不是包）。

```
/** 授权许可、版权以及任何其他内容 */
@file:JvmName("FooBar")

package foo.bar
```

函数

如果函数签名不适合单行，请使用以下语法：

```
fun longMethodName(
    argument: ArgumentType = defaultValue,
    argument2: AnotherArgumentType,
): ReturnType {
```

```
    // 函数体
}
```

函数参数使用常规缩进（4 个空格）。有助于确保与构造函数参数一致

对于由单个表达式构成的函数体，优先使用表达式形式。

```
fun foo(): Int {     // 不良
    return 1
}

fun foo() = 1        // 良好
```

表达式函数体格式化

如果函数的表达式函数体与函数声明不适合放在同一行，那么将 `=` 留在第一， 并将表达式函数体缩进 4 个空格。

```
fun f(x: String, y: String, z: String) =
    veryLongFunctionCallWithManyWords(andLongParametersToo(), x, y, z)
```

属性格式化

对于非常简单的只读属性，请考虑单行格式：

```
val isEmpty: Boolean get() = size == 0
```

对于更复杂的属性，总是将 `get` 与 `set` 关键字放在不同的行上：

```
val foo: String
    get() { /*……*/ }
```

对于具有初始化器的属性，如果初始化器很长，那么在 `=` 号后增加一个换行并将初始化器缩进四个空格：

```
private val defaultCharset: Charset? =
    EncodingRegistry.getInstance().getDefaultCharsetForPropertiesFiles(file)
```

控制流语句

如果 `if` 或 `when` 语句的条件有多行，那么在语句体外边总是使用大括号。 将该条件的每个后续行相对于条件语句起始处缩进 4 个空格。 将该条件的右圆括号与左花括号放在单独一行：

```
if (!component.isSyncing &&
    !hasAnyKotlinRuntimeInScope(module)
) {
    return createKotlinNotConfiguredPanel(module)
}
```

这有助于对齐条件与语句体。

将 `else`、 `catch`、 `finally` 关键字以及 `do-while` 循环的 `while` 关键字与之前的花括号放在相同的行上：

```
if (condition) {
    // 主体
} else {
    // else 部分
}

try {
    // 主体
} finally {
    // 清理
}
```

在 `when` 语句中，如果一个分支不止一行，可以考虑用空行将其与相邻的分支块分开：

```
private fun parsePropertyValue(propName: String, token: Token) {
    when (token) {
        is Token.ValueToken ->
            callback.visitValue(propName, token.value)

        Token.LBRACE -> { // ……
        }
    }
}
```

将短分支放在与条件相同的行上，无需花括号。

```
when (foo) {
    true -> bar() // 良好
    false -> { baz() } // 不良
}
```

方法调用格式化

在较长参数列表的左括号后添加一个换行符。按 4 个空格缩进参数。 将密切相关的多个参数分在同一行。

```
drawSquare(
    x = 10, y = 10,
    width = 100, height = 100,
    fill = true
)
```

在分隔参数名与值的 `=` 左右留空格。

链式调用换行

当对链式调用换行时，将 `.` 字符或者 `?.` 操作符放在下一行，并带有单倍缩进：

```
val anchor = owner
    ?.firstChild!!
    .siblings(forward = true)
    .dropWhile { it is PsiComment || it is PsiWhiteSpace }
```

调用链的第一个调用通常在换行之前，当然如果能让代码更有意义也可以忽略这点。

Lambda 表达式

在 lambda 表达式中，应该在花括号左右以及分隔参数与代码体的箭头左右留空格。 如果一个调用接受单个 lambda 表达式，尽可能将其放在圆括号外边传入。

```
list.filter { it > 10 }
```

如果为 lambda 表达式分配一个标签，那么不要在该标签与左花括号之间留空格：

```
fun foo() {
    ints.forEach lit@{
        // ……
    }
}
```

在多行的 lambda 表达式中声明参数名时，将参数名放在第一行，后跟箭头与换行符：

```
appendCommaSeparated(properties) { prop ->
    val propertyValue = prop.get(obj)  // ……
}
```

如果参数列表太长而无法放在一行上，请将箭头放在单独一行：

```
foo {
   context: Context,
   environment: Env
   ->
   context.configureEnv(environment)
}
```

尾部逗号

尾部逗号是一系列元素中最后一项之后的逗号符号：

```
class Person(
    val firstName: String,
    val lastName: String,
    val age: Int, // 尾部逗号
)
```

使用尾部逗号有以下几点收益：

- 使版本控制差异更清晰——因为所有焦点都集中在变更的值上。
- 使添加及重新排序元素更容易——操作元素时无需再添加或删除逗号。
- 简化了代码生成逻辑（例如对象初始化器）。 最后一个元素也可以有逗号。

尾部逗号完全是可选的——没有它们代码仍可以照常工作。 Kotlin 风格指南鼓励在声明处使用尾部逗号，而自行决定在调用处是否使用。

如需在 IntelliJ IDEA 格式化程序中启用尾部逗号，请转到 **Settings/Preferences | Editor | Code Style | Kotlin**， 打开 **Other** 选项卡并选择 **Use trailing comma** 选项。

枚举

```
enum class Direction {
    NORTH,
    SOUTH,
    WEST,
    EAST, // 尾部逗号
}
```

值实参

```
fun shift(x: Int, y: Int) { /*……*/ }
shift(
    25,
    20, // 尾部逗号
)
val colors = listOf(
    "red",
    "green",
    "blue", // 尾部逗号
)
```

类属性与形参

```
class Customer(
    val name: String,
    val lastName: String, // 尾部逗号
)
class Customer(
    val name: String,
    lastName: String, // 尾部逗号
)
```

函数值形参

```
fun powerOf(
    number: Int,
    exponent: Int, // 尾部逗号
) { /*……*/ }
constructor(
```

```
    x: Comparable<Number>,
    y: Iterable<Number>, // 尾部逗号
) {}
fun print(
    vararg quantity: Int,
    description: String, // 尾部逗号
) {}
```

类型可选的形参（包括 setter）

```
val sum: (Int, Int, Int) -> Int = fun(
    x,
    y,
    z, // 尾部逗号
): Int {
    return x + y + x
}
println(sum(8, 8, 8))
```

索引后缀

```
class Surface {
    operator fun get(x: Int, y: Int) = 2 * x + 4 * y - 10
}
fun getZValue(mySurface: Surface, xValue: Int, yValue: Int) =
    mySurface[
        xValue,
        yValue, // 尾部逗号
    ]
```

lambda 表达式形参

```
fun main() {
    val x = {
            x: Comparable<Number>,
            y: Iterable<Number>, // 尾部逗号
        ->
        println("1")
    }
    println(x)
}
```

when 条目

```
fun isReferenceApplicable(myReference: KClass<*>) = when (myReference) {
    Comparable::class,
    Iterable::class,
    String::class, // 尾部逗号
        -> true
    else -> false
}
```

（注解中的）集合字面值

```
annotation class ApplicableFor(val services: Array<String>)
@ApplicableFor([
    "serializer",
    "balancer",
    "database",
    "inMemoryCache", // 尾部逗号
])
fun run() {}
```

类型实参

```
fun <T1, T2> foo() {}
fun main() {
    foo<
            Comparable<Number>,
            Iterable<Number>, // 尾部逗号
            >()
}
```

类型形参

```
class MyMap<
        MyKey,
        MyValue, // 尾部逗号
        > {}
```

解构声明

```
data class Car(val manufacturer: String, val model: String, val year: Int)
val myCar = Car("Tesla", "Y", 2019)
val (
    manufacturer,
    model,
    year, // 尾部逗号
) = myCar
val cars = listOf<Car>()
fun printMeanValue() {
    var meanValue: Int = 0
    for ((
        _,
        _,
        year, // 尾部逗号
    ) in cars) {
        meanValue += year
    }
    println(meanValue/cars.size)
}
printMeanValue()
```

文档注释

对于较长的文档注释，将开头 `/**` 放在一个独立行中，并且后续每行都以星号开头：

```
/**
 * 这是一条多行
 * 文档注释。
 */
```

简短注释可以放在一行内：

```
/** 这是一条简短文档注释。 */
```

通常，避免使用 `@param` 与 `@return` 标记。而是将参数与返回值的描述直接合并到文档注释中，并在提到参数的任何地方加上参数链接。 只有当需要不适合放进主文本流程的冗长描述时才应使用 `@param` 与 `@return`。

```
// 避免这样：

/**
 * Returns the absolute value of the given number.
 * @param number The number to return the absolute value for.
 * @return The absolute value.
 */
fun abs(number: Int): Int { /*……*/ }

// 而要这样：

/**
 * Returns the absolute value of the given [number].
 */
fun abs(number: Int): Int { /*……*/ }
```

避免重复结构

一般来说，如果 Kotlin 中的某种语法结构是可选的并且被 IDE 高亮为冗余的，那么应该在代码中省略之。为了清楚起见，不要在代码中保留不必要的语法元素 。

Unit 返回类型

如果函数返回 Unit，那么应该省略返回类型：

```
fun foo() { // 这里省略了“: Unit”

}
```

分号

尽可能省略分号。

字符串模版

将简单变量传入到字符串模版中时不要使用花括号。只有用到更长表达式时才使用花括号。

```
println("$name has ${children.size} children")
```

语言特性的惯用法

不可变性

优先使用不可变（而不是可变）数据。初始化后未修改的局部变量与属性，总是将其声明为 `val` 而不是 `var`。

总是使用不可变集合接口（`Collection`, `List`, `Set`, `Map`）来声明无需改变的集合。使用工厂函数创建集合实例时，尽可能选用返回不可变集合类型的函数：

```
// 不良：使用可变集合类型作为无需改变的值
fun validateValue(actualValue: String, allowedValues: HashSet<String>) { …… }

// 良好：使用不可变集合类型
fun validateValue(actualValue: String, allowedValues: Set<String>) { …… }

// 不良：arrayListOf() 返回 ArrayList<T>，这是一个可变集合类型
val allowedValues = arrayListOf("a", "b", "c")

// 良好：listOf() 返回 List<T>
val allowedValues = listOf("a", "b", "c")
```

默认实参值

优先声明带有默认实参的函数而不是声明重载函数。

```
// 不良
fun foo() = foo("a")
fun foo(a: String) { /*……*/ }

// 良好
fun foo(a: String = "a") { /*……*/ }
```

类型别名

如果有一个在代码库中多次用到的函数类型或者带有类型参数的类型，那么最好为它定义一个类型别名：

```
typealias MouseClickHandler = (Any, MouseEvent) -> Unit
typealias PersonIndex = Map<String, Person>
```

如果使用一个 private 或者 internal 的类型别名来避免名称冲突，请优先使用包与导入中提到的 `import …… as ……`。

Lambda 表达式参数

在简短、非嵌套的 lambda 表达式中建议使用 `it` 用法而不是显式声明参数。而在有参数的嵌套 lambda 表达式中，始终显式声明参数。

在 lambda 表达式中返回

避免在 lambda 表达式中使用多个返回到标签。请考虑重新组织这样的 lambda 表达式使其只有单一退出点。 如果这无法做到或者不够清晰，请考虑将 lambda 表达式转换为匿名函数。

不要在 lambda 表达式的最后一条语句中使用返回到标签。

具名实参

当一个方法接受多个相同的原生类型参数或者多个 `Boolean` 类型参数时，请使用具名实参语法， 除非在上下文中的所有参数的含义都已绝对清楚。

```
drawSquare(x = 10, y = 10, width = 100, height = 100, fill = true)
```

条件语句

优先使用 `try`、`if` 与 `when` 的表达形式。

```
return if (x) foo() else bar()
```

```
return when(x) {
    0 -> "zero"
    else -> "nonzero"
}
```

优先选用上述代码而不是：

```
if (x)
    return foo()
else
    return bar()
```

```
when(x) {
    0 -> return "zero"
    else -> return "nonzero"
}
```

if 还是 when

二元条件优先使用 `if` 而不是 `when`。 例如，使用 `if` 的这种语法：

```
if (x == null) …… else ……
```

而不是使用 `when` 的这种：

```
when (x) {
    null -> // ……
    else -> // ……
}
```

如果有三个或多个选项时优先使用 `when`。

Guard conditions in when expression

Use parentheses when combining multiple boolean expressions in `when` expressions or statements with guard conditions:

```
when (status) {
    is Status.Ok if (status.info.isEmpty() || status.info.id == null) -> "no information"
}
```

Instead of:

```
when (status) {
    is Status.Ok if status.info.isEmpty() || status.info.id == null -> "no information"
}
```

在条件中的可空的布尔值

如果需要在条件语句中用到可空的 `Boolean`，使用 `if (value == true)` 或 `if (value == false)` 检测。

循环

优先使用高阶函数（`filter`、`map` 等）而不是循环。例外：`forEach`（优先使用常规的 `for` 循环， 除非 `forEach` 的接收者是可空的或者 `forEach` 用做长调用链的一部分。）

当在使用多个高阶函数的复杂表达式与循环之间进行选择时，请了解每种情况下所执行操作的开销并且记得考虑性能因素。

区间上循环

使用 `..<` 操作符在一个左闭右开区间上循环：

```
for (i in 0..n - 1) { /*……*/ }  // 不良
for (i in 0..<n) { /*……*/ }  // 良好
```

字符串

优先使用字符串模板而不是字符串拼接。

优先使用多行字符串而不是将 `\n` 转义序列嵌入到常规字符串字面值中。

如需在多行字符串中维护缩进，当生成的字符串不需要任何内部缩进时使用 `trimIndent`，而需要内部缩进时使用 `trimMargin`：

```
fun main() {
    println("""
     Not
     trimmed
     text
     """
    )

    println("""
     Trimmed
     text
     """.trimIndent()
    )

    println()

    val a = """Trimmed to margin text:
           |if(a > 1) {
           |    return a
           |}""".trimMargin()

    println(a)
}
```

了解 Java 与 Kotlin 多行字符串的区别。

函数还是属性

在某些场景中，不带参数的函数可与只读属性互换。 虽然语义相似，但是在某种程度上有一些风格上的约定。

底层算法优先使用属性而不是函数：

- 不会抛异常。
- 计算开销小（或者在首次运行时缓存）。
- 如果对象状态没有改变，那么多次调用都会返回相同结果。

扩展函数

放手去用扩展函数。每当你有一个主要用于某个对象的函数时，可以考虑使其成为一个以该对象为接收者的扩展函数。为了尽量减少 API 污染，尽可能地限制扩展函数的可见性。根据需要，使用局部扩展函数、成员扩展函数或者具有私有可视性的顶层扩展函数。

中缀函数

一个函数只有用于两个角色类似的对象时才将其声明为中缀（`infix`）函数。良好示例如：`and`、 `to`、`zip`。不良示例如：`add`。

如果一个方法会改动其接收者，那么不要声明为中缀（`infix`）形式。

工厂函数

如果为一个类声明一个工厂函数，那么不要让它与类自身同名。优先使用独特的名称， 该名称能表明为何该工厂函数的行为与众不同。只有当确实没有特殊的语义时， 才可以使用与该类相同的名称。

```
class Point(val x: Double, val y: Double) {
    companion object {
        fun fromPolar(angle: Double, radius: Double) = Point(……)
    }
}
```

如果一个对象有多个重载的构造函数，它们并非调用不同的超类构造函数，并且不能简化为具有默认实参值的单个构造函数，那么优先用工厂函数取代这些重载的构造函数。

平台类型

返回平台类型表达式的公有函数/方法必须显式声明其 Kotlin 类型：

```
fun apiCall(): String = MyJavaApi.getProperty("name")
```

任何使用平台类型表达式初始化的属性（包级别或类级别）必须显式声明其 Kotlin 类型：

```
class Person {
    val name: String = MyJavaApi.getProperty("name")
}
```

使用平台类型表达式初始化的局部值可以有也可以没有类型声明：

```
fun main() {
    val name = MyJavaApi.getProperty("name")
    println(name)
}
```

作用域函数 apply/with/run/also/let

Kotlin 提供了一系列用来在给定对象上下文中执行代码块的函数：`let`、 `run`、 `with`、 `apply` 以及 `also`。关于不同情况下选择正确作用域函数的准则，请参考作用域函数。

库的编码规范

在编写库时，建议遵循一组额外的规则以确保 API 的稳定性：

- 总是显式指定成员的可见性（以避免将声明意外暴露为公有 API ）。
- 总是显式指定函数返回类型以及属性类型（以避免当实现改变时意外更改返回类型）。
- 为所有公有成员提供 KDoc 注释，不需要任何新文档的覆盖成员除外 （以支持为该库生成文档）。

在库作者指南中详细了解为库编写 API 时需要考虑的最佳实践与意见。

基本类型

在 Kotlin 中，所有东西都是对象，在这个意义上讲可以在任何变量上调用成员函数与属性。 While certain types have an optimized internal representation as primitive values at runtime (such as numbers, characters, booleans and others), they appear and behave like regular classes to you.

本节会描述 Kotlin 中使用的基本类型：

- 数字及其无符号版
- 布尔
- 字符
- 字符串
- 数组

  提示： Learn how to perform type checks and casts in Kotlin.

To learn about other Kotlin types, such as `Nothing`, `Any`, and `Unit`, look through the Kotlin API reference:

- `Any`
- `Nothing`
- `Unit`

数字

整数类型

Kotlin 提供了一组表示数字的内置类型。 对于整数，有四种不同大小与取值范围的类型：

类型	大小（比特数）	最小值	最大值
`Byte`	8	-128	127
`Short`	16	-32768	32767
`Int`	32	-2,147,483,648 (-2^{31})	2,147,483,647 ($2^{31} - 1$)
`Long`	64	-9,223,372,036,854,775,808 (-2^{63})	9,223,372,036,854,775,807 ($2^{63} - 1$)

> **提示:** In addition to signed integer types, Kotlin also provides unsigned integer types. As unsigned integers are aimed at a different set of use cases, they are covered separately. See .

当初始化一个没有显式指定类型的变量时，编译器会自动推断为自 `Int` 起足以表示该值的最小类型。 如果不超过 `Int` 的取值范围，那么类型是 `Int`。 如果超出了该范围，那么类型是 `Long`。 如需显式指定 `Long` 值，请给该值追加后缀 `L`。 To use the `Byte` or `Short` type, specify it explicitly in the declaration. 显式指定类型会触发编译器检测该值不会超出指定类型的取值范围。

```
val one = 1 // Int
val threeBillion = 3000000000 // Long
val oneLong = 1L // Long
val oneByte: Byte = 1
```

浮点类型

对于实数，Kotlin 提供了浮点类型 `Float` 与 `Double` 类型，遵循 IEEE 754 标准。 `Float` 表达 IEEE 754 *单精度*，而 `Double` 表达*双精度*。

这两个类型的大小不同，并为两种不同精度的浮点数提供存储：

类型	大小（比特数）	有效数字比特数	指数比特数	十进制位数
`Float`	32	24	8	6-7
`Double`	64	53	11	15-16

可以仅使用带小数部分的数字初始化 `Double` 与 `Float` 变量。 小数部分与整数部分之间用句点（`.`）分隔

对于以小数初始化的变量，编译器会推断为 `Double` 类型：

```
val pi = 3.14          // Double

val one: Double = 1    // Int is inferred
// 初始化类型不匹配

val oneDouble = 1.0    // Double
```

如需将一个值显式指定为 `Float` 类型，请添加 `f` 或 `F` 后缀。 如果以这种方式提供的值包含多于 7 位十进制数，那么会将其舍入：

```
val e = 2.7182818284          // Double
val eFloat = 2.7182818284f    // Float, 实际值为 2.7182817
```

与一些其他语言不同，Kotlin 中的数字没有隐式拓宽转换。 例如，具有 `Double` 参数的函数只能对 `Double` 值调用，而不能对 `Float`、 `Int` 或者其他数字值调用：

```
fun main() {
    fun printDouble(x: Double) { print(x) }

    val x = 1.0
    val xInt = 1
    val xFloat = 1.0f

    printDouble(x)

    printDouble(xInt)
    // 参数类型不匹配

    printDouble(xFloat)
    // 参数类型不匹配
}
```

如需将数值转换为不同的类型，请使用显式转换。

数字字面常量

数值常量字面值有以下几种:

- 十进制：`123`
- Long 类型，以大写 `L` 结尾：`123L`
- 十六进制：`0x0F`
- 二进制：`0b00001011`

  注意: Kotlin 不支持八进制。

Kotlin 同样支持浮点数的常规表示方法:

- Doubles (default when the fractional part does not end with a letter): `123.5`, `123.5e10`

- Float，以字母 `f` 或 `F` 结尾：`123.5f`

你可以使用下划线使数字常量更易读：

```
val oneMillion = 1_000_000
val creditCardNumber = 1234_5678_9012_3456L
val socialSecurityNumber = 999_99_9999L
val hexBytes = 0xFF_EC_DE_5E
val bytes = 0b11010010_01101001_10010100_10010010
val bigFractional = 1_234_567.7182818284
```

> **提示:** 无符号整数字面值也有特殊后缀。
> 更多内容请参阅无符号整型字面值。

Boxing and caching numbers on the Java Virtual Machine

The way the JVM stores numbers can make your code behave counterintuitively because of the cache used by default for small (byte-sized) numbers.

The JVM stores numbers as primitive types: `int`, `double`, and so on. When you use generic types or create a nullable number reference such as `Int?`, numbers are boxed in Java classes such as `Integer` or `Double`.

The JVM applies a memory optimization technique to `Integer` and other objects that represent numbers between `-128` and `127`. All nullable references to such objects refer to the same cached object. For example, nullable objects in the following code are referentially equal:

```
fun main() {
    val a: Int = 100
    val boxedA: Int? = a
    val anotherBoxedA: Int? = a

    println(boxedA === anotherBoxedA) // true
}
```

For numbers outside this range, the nullable objects are different but structurally equal:

```
fun main() {
    val b: Int = 10000
    val boxedB: Int? = b
    val anotherBoxedB: Int? = b

    println(boxedB === anotherBoxedB) // false
    println(boxedB == anotherBoxedB) // true
}
```

For this reason, Kotlin warns about using referential equality with boxable numbers and literals with the following message: `"Identity equality for arguments of types ... and ... is`

`prohibited."` When comparing `Int`, `Short`, `Long`, and `Byte` types (as well as `Char` and `Boolean`), use structural equality checks to get consistent results.

显式数字转换

由于不同的表示方式，数字类型并*不是*彼此的子类型。 As a consequence, smaller types are *not* implicitly converted to bigger types and vice versa. For example, assigning a value of type `Byte` to an `Int` variable requires an explicit conversion:

```
fun main() {
    val byte: Byte = 1
    // OK, 字面值会静态检测

    val intAssignedByte: Int = byte
    // Initializer type mismatch

    val intConvertedByte: Int = byte.toInt()

    println(intConvertedByte)
}
```

所有数字类型都支持转换为其他类型：

- `toByte(): Byte` (deprecated for Float and Double)
- `toShort(): Short`
- `toInt(): Int`
- `toLong(): Long`
- `toFloat(): Float`
- `toDouble(): Double`

很多情况都不需要显式类型转换，因为类型会从上下文推断出来， 而算术运算符会有重载自动处理类型转换。例如：

```
fun main() {
    val l = 1L + 3       // Long + Int => Long
    println(l is Long)   // true
}
```

Reasoning against implicit conversions

Kotlin doesn't support implicit conversions because they can lead to unexpected behavior.

If numbers of different types were converted implicitly, we could sometimes lose equality and identity silently. For example, imagine if `Int` was a subtype of `Long`:

```
// Hypothetical code, does not actually compile:
val a: Int? = 1    // A boxed Int (java.lang.Integer)
```

```
val b: Long? = a   // Implicit conversion yields a boxed Long (java.lang.Long)
print(b == a)      // Prints "false" as Long.equals() checks not only the value but whether the other number is Long as well
```

数字运算

Kotlin支持数字运算的标准集：`+`、 `-`、 `*`、 `/`、 `%`。它们已定义为相应的类成员：

```
fun main() {
    println(1 + 2)
    println(2_500_000_000L - 1L)
    println(3.14 * 2.71)
    println(10.0 / 3)
}
```

可以在自定义数字类中覆盖这些操作符。 详情请参见操作符重载。

整数除法

整数间的除法总是返回整数。会丢弃任何小数部分。

```
fun main() {
    val x = 5 / 2
    println(x == 2.5)
    // Operator '==' cannot be applied to 'Int' and 'Double'

    println(x == 2)
    // true
}
```

对于任何两个整数类型之间的除法来说都是如此：

```
fun main() {
    val x = 5L / 2
    println (x == 2)
    // Error, as Long (x) cannot be compared to Int (2)

    println(x == 2L)
    // true
}
```

如需返回带小数部分的除法结果，请将其中的一个参数显式转换为浮点类型：

```
fun main() {
    val x = 5 / 2.toDouble()
    println(x == 2.5)
}
```

位运算

Kotlin 对整数提供了一组*位运算*。它们直接使用数字的比特表示在二进制级别进行操作。 位运算有可以通过中缀形式调用的函数表示。只能应用于 `Int` 与 `Long`：

```
fun main() {
    val x = 1
    val xShiftedLeft = (x shl 2)
    println(xShiftedLeft)
    // 4

    val xAnd = x and 0x000FF000
    println(xAnd)
    // 0
}
```

完整的位运算列表：

- `shl(bits)` – 有符号左移
- `shr(bits)` – 有符号右移
- `ushr(bits)` – 无符号右移
- `and(bits)` – 位**与**
- `or(bits)` – 位**或**
- `xor(bits)` – 位**异或**
- `inv()` – 位非

浮点数比较

本节讨论的浮点数操作如下：

- 相等性检测：`a == b` 与 `a != b`
- 比较操作符：`a < b`、 `a > b`、 `a <= b`、 `a >= b`
- 区间实例以及区间检测：`a..b`、 `x in a..b`、 `x !in a..b`

当其中的操作数 `a` 与 `b` 都是静态已知的 `Float` 或 `Double` 或者它们对应的可空类型（声明为该类型，或者推断为该类型，或者智能类型转换的结果是该类型），两数字所形成的操作或者区间遵循 IEEE 754 浮点运算标准。

然而，为了支持泛型场景并提供全序支持，对于**并非**静态类型就是浮点数的情况，行为是不同的。例如是 `Any`、 `Comparable<...>` 或者 `Collection<T>` 类型。 这种情况下，这些操作使用为 `Float` 与 `Double` 实现的 `equals` 与 `compareTo`。 因此：

- 认为 `NaN` 与其自身相等
- 认为 `NaN` 比包括正无穷大（`POSITIVE_INFINITY`）在内的任何其他元素都大
- 认为 `-0.0` 小于 `0.0`

以下示例显示了静态类型作为浮点数（`Double.NaN`）的操作数与静态类型**并非**作为浮点数的操作数（`listOf(T)`）之间的行为差异。

```
fun main() {
    // 静态类型作为浮点数的操作数
    println(Double.NaN == Double.NaN)                 // false

    // 静态类型并非作为浮点数的操作数
    // 所以 NaN 等于它本身
    println(listOf(Double.NaN) == listOf(Double.NaN)) // true

    // 静态类型作为浮点数的操作数
    println(0.0 == -0.0)                              // true

    // 静态类型并非作为浮点数的操作数
    // 所以 -0.0 小于 0.0
    println(listOf(0.0) == listOf(-0.0))              // false

    println(listOf(Double.NaN, Double.POSITIVE_INFINITY, 0.0, -0.0).sorted())
    // [-0.0, 0.0, Infinity, NaN]
}
```

无符号整型

除了整数类型，对于无符号整数，Kotlin 还提供了以下类型：

Type	Size (bits)	Min value	Max value
`UByte`	8	0	255
`UShort`	16	0	65,535
`UInt`	32	0	4,294,967,295 (2^{32} - 1)
`ULong`	64	0	18,446,744,073,709,551,615 (2^{64} - 1)

无符号类型支持其对应有符号类型的大多数操作。

> **注意:** 无符号数实现为具有单个存储属性的内联类， 该属性类型为与其宽度相同的对应有符号类型。 If you want to convert between unsigned and signed integer types, make sure you update your code so that any function calls and operations support the new type.

无符号数组与区间

> **警告:** 无符号数组及其对应操作处于 Beta 状态。 它们可能随时进行不兼容变更。 需要选择加入（详见下文）。

与原生类型相同，每个无符号类型都有表示相应类型数组的类型：

- `UByteArray`：无符号字节数组。
- `UShortArray`：无符号短整型数组。
- `UIntArray`：无符号整型数组。
- `ULongArray`：无符号长整型数组。

与有符号整型数组一样，它们提供了类似于 `Array` 类的 API 而没有装箱开销。

When you use unsigned arrays, you receive a warning that indicates that this feature is not stable yet. To remove the warning, opt-in with the `@ExperimentalUnsignedTypes` annotation. It's up to you to decide if your clients have to explicitly opt-in into usage of your API, but keep in mind that unsigned arrays are not a stable feature, so an API that uses them can be broken by changes in the language. Learn more about opt-in requirements.

区间与数列也支持 `UInt` 与 `ULong`（通过这些类 `UIntRange`、 `UIntProgression`、 `ULongRange`、 `ULongProgression`）。 Together with the unsigned integer types, these classes are stable.

无符号整数字面值

为使无符号整型更易用，可以在整型字面值后附加一个后缀来表示指定无符号类型（类似于 `Float` 用 `F` 或 `Long` 用 `L`）：

- `u` and `U` letters signify unsigned literals without specifying the exact type. If no expected type is provided, the compiler uses `UInt` or `ULong` depending on the size of the literal:

```
val b: UByte = 1u  // UByte，已提供预期类型
val s: UShort = 1u // UShort，已提供预期类型
val l: ULong = 1u  // ULong，已提供预期类型

val a1 = 42u // UInt：未提供预期类型，常量适于 UInt
val a2 = 0xFFFF_FFFF_FFFFu // ULong：未提供预期类型，常量不适于 UInt
```

- `uL` 与 `UL` 显式指定字面值应为无符号长整型：

```
val a = 1UL // ULong，即使未提供预期类型并且常量适于 UInt
```

使用场景

The main use case of unsigned numbers is utilizing the full bit range of an integer to represent positive values.
For example, to represent hexadecimal constants that do not fit in signed types such as color in 32-bit `AARRGGBB` format:

```
data class Color(val representation: UInt)

val yellow = Color(0xFFCC00CCu)
```

You can use unsigned numbers to initialize byte arrays without explicit `toByte()` literal casts:

```
val byteOrderMarkUtf8 = ubyteArrayOf(0xEFu, 0xBBu, 0xBFu)
```

Another use case is interoperability with native APIs. Kotlin allows representing native declarations that contain unsigned types in the signature. The mapping won't substitute unsigned integers with signed ones keeping the semantics unaltered.

非目标

While unsigned integers can only represent positive numbers and zero, it's not a goal to use them where application domain requires non-negative integers. For example, as a type of collection size or collection index value.

There are a couple of reasons:

- Using signed integers can help to detect accidental overflows and signal error conditions, such as `List.lastIndex` being -1 for an empty list.
- Unsigned integers cannot be treated as a range-limited version of signed ones because their range of values is not a subset of the signed integers range. Neither signed, nor unsigned integers are subtypes of each other.

布尔

`Boolean` 类型表示可以有 `true` 与 `false` 两个值的布尔对象。 `Boolean` has a nullable counterpart declared as `Boolean?`.

> **注意:** On the JVM, booleans stored as the primitive `boolean` type typically use 8 bits.

布尔值的内置运算有：

- `||` ——析取（逻辑*或*）
- `&&` ——合取（逻辑*与*）
- `!` ——否定（逻辑*非*）

For example:

```
fun main() {
    val myTrue: Boolean = true
    val myFalse: Boolean = false
    val boolNull: Boolean? = null

    println(myTrue || myFalse)
    // true
    println(myTrue && myFalse)
    // false
    println(!myTrue)
    // false
    println(boolNull)
    // null
}
```

The `||` and `&&` operators work lazily, which means:

- If the first operand is `true`, the `||` operator does not evaluate the second operand.
- If the first operand is `false`, the `&&` operator does not evaluate the second operand.

> **注意:** 在 JVM 平台，布尔对象的可空引用是装箱的 Java 类，类似于数字。

字符

字符用 `Char` 类型表示。 字符字面值用单引号括起来: `'1'`。

> **注意:** On the JVM, a character stored as primitive type: `char`, represents a 16-bit Unicode character.

特殊字符可以以转义反斜杠 `\` 开始。 支持这几个转义序列:

- `\t` —— 制表符
- `\b` —— 退格符
- `\n` —— 换行（LF）
- `\r` —— 回车（CR）
- `\'` —— 单引号
- `\"` —— 双引号
- `\\` —— 反斜杠
- `\$` —— 美元符

编码其他字符要用 Unicode 转义序列语法: `'\uFF00'`。

```
fun main() {
    val aChar: Char = 'a'

    println(aChar)
    println('\n') // 输出一个额外的换行符
    println('\uFF00')
}
```

如果字符变量的值是数字，那么可以使用 `digitToInt()` 函数将其显式转换为 `Int` 数字。

> **注意:** 在JVM 平台，当需要可空引用时字符会是装箱的 Java 类，类似于数字。 装箱操作不保留同一性。

字符串

Kotlin 中字符串用 `String` 类型表示。

> **注意:** On the JVM, an object of `String` type in UTF-16 encoding uses approximately 2 bytes per character.

通常，字符串值是双引号（`"`）中的字符序列：

```
val str = "abcd 123"
```

字符串的元素——字符可以使用索引运算符访问: `s[i]`。 可以使用 `for` 循环遍历这些字符：

```
fun main() {
    val str = "abcd"
    for (c in str) {
        println(c)
    }
}
```

字符串是不可变的。 一旦初始化了一个字符串，就不能改变它的值或者给它赋新值。 所有转换字符串的操作都以一个新的 `String` 对象来返回结果，而保持原始字符串不变：

```
fun main() {
    val str = "abcd"

    // 创建并输出一个新的 String 对象
    println(str.uppercase())
    // ABCD

    // 原始字符串保持不变
    println(str)
    // abcd
}
```

如需连接字符串，可以用 `+` 操作符。这也适用于连接字符串与其他类型的值， 只要表达式中的第一个元素是字符串：

```
fun main() {
    val s = "abc" + 1
    println(s + "def")
    // abc1def
}
```

> **注意:** 在大多数情况下，优先使用字符串模板或多行字符串而不是字符串连接。

字符串字面值

Kotlin 有两种类型的字符串字面值:

- 转义字符串
- 多行字符串

转义字符串

*转义字符串*可以包含转义字符。
以下是转义字符串的一个示例:

```
val s = "Hello, world!\n"
```

转义采用传统的反斜杠（`\`）方式。
所支持的转义序列请参见 字符 页。

多行字符串

*多行字符串*可以包含换行以及任意文本。 它使用三个引号（`"""`）分界符括起来，内部没有转义并且可以包含换行以及任何其他字符:

```
val text = """
    for (c in "foo")
        print(c)
    """
```

如需删掉多行字符串中的前导空格，请使用 `trimMargin()` 函数:

```
val text = """
    |Tell me and I forget.
    |Teach me and I remember.
    |Involve me and I learn.
    |(Benjamin Franklin)
    """.trimMargin()
```

默认以竖线 `|` 作为边界前缀，但你可以选择其他字符并作为参数传入，比如 `trimMargin(">")`。

字符串模板

字符串字面值可以包含*模板表达式*——一些小段代码，会求值并把结果合并到字符串中。 When a template expression is processed, Kotlin automatically calls the `.toString()` function on the expression's result to convert it into a string. 模板表达式以美元符（`$`）开头，要么由一个变量名构成:

```
fun main() {
    val i = 10
    println("i = $i")
```

```
    // i = 10

    val letters = listOf("a","b","c","d","e")
    println("Letters: $letters")
    // Letters: [a, b, c, d, e]
}
```

要么是用花括号括起来的表达式:

```
fun main() {
    val s = "abc"
    println("$s.length is ${s.length}")
    // abc.length is 3
}
```

在多行字符串及转义字符串中都可以使用模板。不过，多行字符串不支持反斜杠转义。 如需在多行字符串中允许出现在标识符（identifier）开头的任意符号之前插入美元符 `$`， 请使用以下语法:

```
val price = """
${'$'}_9.99
"""
```

> **注意:** To avoid `${'$'}` sequences in strings, you can use the Experimental multi-dollar string interpolation feature.

Multi-dollar string interpolation

> **警告:** The multi-dollar string interpolation is Experimental and opt-in is required (see details below).
>
> It may be changed at any time. We would appreciate your feedback in YouTrack.

Multi-dollar string interpolation allows you to specify how many consecutive dollar signs are required to trigger interpolation. Interpolation is the process of embedding variables or expressions directly into a string.

While you can escape literals for single-line strings, multiline strings in Kotlin don't support backslash escaping. To include dollar signs (`$`) as literal characters, you must use the `${'$'}` construct to prevent string interpolation. This approach can make code harder to read, especially when strings contain multiple dollar signs.

Multi-dollar string interpolation simplifies this by letting you treat dollar signs as literal characters in both single-line and multiline strings. For example:

```
val KClass<*>.jsonSchema : String
    get() = $$"""
    {
      "$schema": "https://json-schema.org/draft/2020-12/schema",
      "$id": "https://example.com/product.schema.json",
```

```
      "$dynamicAnchor": "meta"
      "title": "$${simpleName ?: qualifiedName ?: "unknown"}",
      "type": "object"
    }
    """
```

Here, the `$$` prefix specifies that two consecutive dollar signs are required to trigger string interpolation. Single dollar signs remain as literal characters.

You can adjust how many dollar signs trigger interpolation. For example, using three consecutive dollar signs (`$$$`) allows `$` and `$$` to remain as literals while enabling interpolation with `$$$`:

```
val productName = "carrot"
val requestedData =
    $$$"""{
      "currency": "$",
      "enteredAmount": "42.45 $$",
      "$$serviceField": "none",
      "product": "$$$productName"
    }
    """

println(requestedData)
//{
//    "currency": "$",
//    "enteredAmount": "42.45 $$",
//    "$$serviceField": "none",
//    "product": "carrot"
//}
```

Here, the `$$$` prefix allows the string to include `$` and `$$` without requiring the `${'$'}` construct for escaping.

To enable the feature, use the following compiler option in the command line:

```
kotlinc -Xmulti-dollar-interpolation main.kt
```

Alternatively, update the `compilerOptions {}` block of your Gradle build file:

```
// build.gradle.kts
kotlin {
    compilerOptions {
        freeCompilerArgs.add("-Xmulti-dollar-interpolation")
    }
}
```

This feature doesn't affect existing code that uses single-dollar string interpolation. You can continue using a single `$` as before and apply multi-dollar signs when you need to handle literal dollar signs in strings.

String formatting

> **注意:** String formatting with the `String.format()` function is only available in Kotlin/JVM.

To format a string to your specific requirements, use the `String.format()` function.

The `String.format()` function accepts a format string and one or more arguments. The format string contains one placeholder (indicated by `%`) for a given argument, followed by format specifiers. Format specifiers are formatting instructions for the respective argument, consisting of flags, width, precision, and conversion type. Collectively, format specifiers shape the output's formatting. Common format specifiers include `%d` for integers, `%f` for floating-point numbers, and `%s` for strings. You can also use the `argument_index$` syntax to reference the same argument multiple times within the format string in different formats.

> **注意:** For a detailed understanding and an extensive list of format specifiers, see Java's Class Formatter documentation.

Let's look at an example:

```
fun main() {
    // Formats an integer, adding leading zeroes to reach a length of seven characters
    val integerNumber = String.format("%07d", 31416)
    println(integerNumber)
    // 0031416

    // Formats a floating-point number to display with a + sign and four decimal places
    val floatNumber = String.format("%+.4f", 3.141592)
    println(floatNumber)
    // +3.1416

    // Formats two strings to uppercase, each taking one placeholder
    val helloString = String.format("%S %S", "hello", "world")
    println(helloString)
    // HELLO WORLD

    // Formats a negative number to be enclosed in parentheses, then repeats the same number
in a different format (without parentheses) using `argument_index$`.
    val negativeNumberInParentheses = String.format("%(d means %1\$d", -31416)
    println(negativeNumberInParentheses)
    //(31416) means -31416
}
```

The `String.format()` function provides similar functionality to string templates. However, the `String.format()` function is more versatile because there are more formatting options available.

In addition, you can assign the format string from a variable. This can be useful when the format string changes, for example, in localization cases that depend on the user locale.

Be careful when using the `String.format()` function because it can be easy to mismatch the number or position of the arguments with their corresponding placeholders.

数组

数组是一种保存固定数量相同类型或其子类型的值的数据结构。 Kotlin 中最常见的数组类型是对象类型数组，由 `Array` 类表示。

> **注意:** 如果在对象类型数组中使用原生类型，那么会对性能产生影响，因为原生值都装箱成了对象。 为了避免装箱开销，请改用原生类型数组。

何时使用数组

当需要满足特殊的底层需求时，可以在 Kotlin 中使用数组。 例如，如果性能要求超出了常规应用程序的需要，或者需要构建自定义的数据结构。 如果没有这些限制，那么请改用集合。

与数组相比，集合具有以下优点：

- 集合可以是只读的，这提供了更多的控制权而支持编写具有明确意图的健壮代码。
- 易于对集合增删元素。相比之下，数组大小是固定的。 对数组增删元素的唯一方式是每次创建一个新数组，效率很低：

```
fun main() {
    var riversArray = arrayOf("Nile", "Amazon", "Yangtze")

    // 使用 += 赋值操作创建了一个新的 riversArray，
    // 复制了原始元素并添加了“Mississippi”
    riversArray += "Mississippi"
    println(riversArray.joinToString())
    // Nile, Amazon, Yangtze, Mississippi
}
```

- 可以使用相等操作符（`==`）来检验两个集合是否在结构上相等。但不能对数组使用这个操作符。 相反，必须使用一个特殊函数，关于这点可以参阅比较数组了解更多信息。

关于集合的更多信息请参见集合概述。

创建数组

To create arrays in Kotlin, you can use:

- functions, such as `arrayOf()`, `arrayOfNulls()` or `emptyArray()`.
- the `Array` constructor.

This example uses the `arrayOf()` function and passes item values to it:

```
fun main() {
    // Creates an array with values [1, 2, 3]
    val simpleArray = arrayOf(1, 2, 3)
    println(simpleArray.joinToString())
    // 1, 2, 3
}
```

This example uses the `arrayOfNulls()` function to create an array of a given size filled with `null` elements:

```
fun main() {
    // Creates an array with values [null, null, null]
    val nullArray: Array<Int?> = arrayOfNulls(3)
    println(nullArray.joinToString())
    // null, null, null
}
```

This example uses the `emptyArray()` function to create an empty array :

```
    var exampleArray = emptyArray<String>()
```

> **注意:** You can specify the type of the empty array on the left-hand or right-hand side of the assignment due to Kotlin's type inference.
>
> For example:
>
> ```
> var exampleArray = emptyArray<String>()
>
> var exampleArray: Array<String> = emptyArray()
> ```

The `Array` constructor takes the array size and a function that returns values for array elements given its index:

```
fun main() {
    // Creates an Array<Int> that initializes with zeros [0, 0, 0]
    val initArray = Array<Int>(3) { 0 }
    println(initArray.joinToString())
    // 0, 0, 0

    // 创建一个 Array<String> 初始化为 ["0", "1", "4", "9", "16"]
    val asc = Array(5) { i -> (i * i).toString() }
    asc.forEach { print(it) }
    // 014916
}
```

> **注意:** Like in most programming languages, indices start from 0 in Kotlin.

嵌套数组

Arrays can be nested within each other to create multidimensional arrays:

```
fun main() {
    // Creates a two-dimensional array
    val twoDArray = Array(2) { Array<Int>(2) { 0 } }
    println(twoDArray.contentDeepToString())
    // [[0, 0], [0, 0]]

    // Creates a three-dimensional array
    val threeDArray = Array(3) { Array(3) { Array<Int>(3) { 0 } } }
    println(threeDArray.contentDeepToString())
    // [[[0, 0, 0], [0, 0, 0], [0, 0, 0]], [[0, 0, 0], [0, 0, 0], [0, 0, 0]], [[0, 0, 0],
[0, 0, 0], [0, 0, 0]]]
}
```

注意: Nested arrays don't have to be the same type or the same size.

访问与修改元素

Arrays are always mutable. To access and modify elements in an array, use the indexed access operator `[]`:

```
fun main() {
    val simpleArray = arrayOf(1, 2, 3)
    val twoDArray = Array(2) { Array<Int>(2) { 0 } }

    // Accesses the element and modifies it
    simpleArray[0] = 10
    twoDArray[0][0] = 2

    // Prints the modified element
    println(simpleArray[0].toString()) // 10
    println(twoDArray[0][0].toString()) // 2
}
```

Arrays in Kotlin are *invariant*. This means that Kotlin doesn't allow you to assign an `Array<String>` to an `Array<Any>` to prevent a possible runtime failure. Instead, you can use `Array<out Any>`. For more information, see Type Projections.

使用数组

In Kotlin, you can work with arrays by using them to pass a variable number of arguments to a function or perform operations on the arrays themselves. For example, comparing arrays, transforming their contents or converting them to collections.

向函数传入可变数量的实参

In Kotlin, you can pass a variable number of arguments to a function via the `vararg` parameter. This is useful when you don't know the number of arguments in advance, like when formatting a message or creating an SQL query.

To pass an array containing a variable number of arguments to a function, use the *spread* operator (`*`). The spread operator passes each element of the array as individual arguments to your chosen function:

```
fun main() {
    val lettersArray = arrayOf("c", "d")
    printAllStrings("a", "b", *lettersArray)
    // abcd
}

fun printAllStrings(vararg strings: String) {
    for (string in strings) {
        print(string)
    }
}
```

For more information, see Variable number of arguments (varargs).

比较数组

To compare whether two arrays have the same elements in the same order, use the `.contentEquals()` and `.contentDeepEquals()` functions:

```
fun main() {
    val simpleArray = arrayOf(1, 2, 3)
    val anotherArray = arrayOf(1, 2, 3)

    // Compares contents of arrays
    println(simpleArray.contentEquals(anotherArray))
    // true

    // Using infix notation, compares contents of arrays after an element
    // is changed
    simpleArray[0] = 10
    println(simpleArray contentEquals anotherArray)
    // false
}
```

> **警告:** Don't use equality (`==`) and inequality (`!=`) operators to compare the contents of arrays. These operators check whether the assigned variables point to the same object.
>
> To learn more about why arrays in Kotlin behave this way, see our blog post.

转换数组

Kotlin has many useful functions to transform arrays. This document highlights a few but this isn't an exhaustive list. For the full list of functions, see our API reference.

Sum

To return the sum of all elements in an array, use the `.sum()` function:

```
fun main() {
    val sumArray = arrayOf(1, 2, 3)

    // Sums array elements
    println(sumArray.sum())
    // 6
}
```

> **注意:** The `.sum()` function can only be used with arrays of numeric data types, such as `Int`.

Shuffle

To randomly shuffle the elements in an array, use the `.shuffle()` function:

```
fun main() {
    val simpleArray = arrayOf(1, 2, 3)

    // Shuffles elements [3, 2, 1]
    simpleArray.shuffle()
    println(simpleArray.joinToString())

    // Shuffles elements again [2, 3, 1]
    simpleArray.shuffle()
    println(simpleArray.joinToString())
}
```

将数组转换为集合

If you work with different APIs where some use arrays and some use collections, then you can convert your arrays to collections and vice versa.

转换为 List 或 Set

To convert an array to a `List` or `Set`, use the `.toList()` and `.toSet()` functions.

```
fun main() {
    val simpleArray = arrayOf("a", "b", "c", "c")

    // Converts to a Set
    println(simpleArray.toSet())
```

```
    // [a, b, c]

    // Converts to a List
    println(simpleArray.toList())
    // [a, b, c, c]
}
```

转换为 Map

To convert an array to a `Map`, use the `.toMap()` function.

Only an array of `Pair<K,V>` can be converted to a `Map`. The first value of a `Pair` instance becomes a key, and the second becomes a value. This example uses the infix notation to call the `to` function to create tuples of `Pair`:

```
fun main() {
    val pairArray = arrayOf("apple" to 120, "banana" to 150, "cherry" to 90, "apple" to 140)

    // Converts to a Map
    // The keys are fruits and the values are their number of calories
    // Note how keys must be unique, so the latest value of "apple"
    // overwrites the first
    println(pairArray.toMap())
    // {apple=140, banana=150, cherry=90}
}
```

原生类型数组

If you use the `Array` class with primitive values, these values are boxed into objects. As an alternative, you can use primitive-type arrays, which allow you to store primitives in an array without the side effect of boxing overhead:

Primitive-type array	Equivalent in Java
`BooleanArray`	`boolean[]`
`ByteArray`	`byte[]`
`CharArray`	`char[]`
`DoubleArray`	`double[]`
`FloatArray`	`float[]`
`IntArray`	`int[]`
`LongArray`	`long[]`
`ShortArray`	`short[]`

These classes have no inheritance relation to the `Array` class, but they have the same set of functions and properties.

This example creates an instance of the `IntArray` class:

```
fun main() {
    // Creates an array of Int of size 5 with the values initialized to zero
    val exampleArray = IntArray(5)
    println(exampleArray.joinToString())
    // 0, 0, 0, 0, 0
}
```

> **注意:** To convert primitive-type arrays to object-type arrays, use the `.toTypedArray()` function.
>
> To convert object-type arrays to primitive-type arrays, use `.toBooleanArray()`, `.toByteArray()`, `.toCharArray()`, and so on.

下一步做什么?

- To learn more about why we recommend using collections for most use cases, read our Collections overview.
- Learn about other basic types.
- If you are a Java developer, read our Java to Kotlin migration guide for Collections.

类型检测与类型转换

In Kotlin, you can perform type checks to check the type of an object at runtime. Type casts enable you to convert objects to a different type.

> **提示:** To learn specifically about **generics** type checks and casts, for example `List<T>`, `Map<K,V>`, see Generics type checks and casts.

is 与 !is 操作符

如需在运行时检测对象是否符合给定类型，使用 `is` 操作符或其否定形式 `!is`：

```
if (obj is String) {
    print(obj.length)
}

if (obj !is String) { // 与 !(obj is String) 相同
    print("Not a String")
} else {
    print(obj.length)
}
```

智能转换

大多数场景都不需要使用显式转换操作符，because the compiler automatically casts objects for you. This is called smart-casting. 编译器跟踪不可变值的类型检测以及显式转换， 并在必要时自动插入隐式安全的）转换：

```
fun demo(x: Any) {
    if (x is String) {
        print(x.length) // x 自动转换为字符串
    }
}
```

编译器甚至足够聪明，能够知道如果反向检测导致返回那么该转换是安全的：

```
if (x !is String) return

print(x.length) // x 自动转换为字符串
```

Control flow

Smart casts work not only for `if` conditional expressions but also for `when` expressions and `while` loops:

```
when (x) {
    is Int -> print(x + 1)
    is String -> print(x.length + 1)
    is IntArray -> print(x.sum())
}
```

If you declare a variable of `Boolean` type before using it in your `if`, `when`, or `while` condition, then any information collected by the compiler about the variable will be accessible in the corresponding block for smart-casting.

This can be useful when you want to do things like extract boolean conditions into variables. Then, you can give the variable a meaningful name, which will improve your code readability and make it possible to reuse the variable later in your code. For example:

```
class Cat {
    fun purr() {
        println("Purr purr")
    }
}

fun petAnimal(animal: Any) {
    val isCat = animal is Cat
    if (isCat) {
        // The compiler can access information about
        // isCat, so it knows that animal was smart-cast
        // to the type Cat.
        // Therefore, the purr() function can be called.
        animal.purr()
    }
}

fun main(){
    val kitty = Cat()
    petAnimal(kitty)
    // Purr purr
}
```

Logical operators

The compiler can perform smart casts on the right-hand side of `&&` or `||` operators if there is a type check (regular or negative) on the left-hand side:

```
// `||` 右侧的 x 自动转换为 String
if (x !is String || x.length == 0) return

// `&&` 右侧的 x 自动转换为 String
if (x is String && x.length > 0) {
    print(x.length) // x 自动转换为 String
}
```

If you combine type checks for objects with an `or` operator (`||`), a smart cast is made to their closest common supertype:

```
interface Status {
    fun signal() {}
}

interface Ok : Status
interface Postponed : Status
interface Declined : Status

fun signalCheck(signalStatus: Any) {
    if (signalStatus is Postponed || signalStatus is Declined) {
        // signalStatus is smart-cast to a common supertype Status
        signalStatus.signal()
    }
}
```

> **注意:** The common supertype is an **approximation** of a union type. Union types are not currently supported in Kotlin.

Inline functions

The compiler can smart-cast variables captured within lambda functions that are passed to inline functions.

Inline functions are treated as having an implicit `callsInPlace` contract. This means that any lambda functions passed to an inline function are called in place. Since lambda functions are called in place, the compiler knows that a lambda function can't leak references to any variables contained within its function body.

The compiler uses this knowledge, along with other analyses to decide whether it's safe to smart-cast any of the captured variables. For example:

```
interface Processor {
    fun process()
}

inline fun inlineAction(f: () -> Unit) = f()

fun nextProcessor(): Processor? = null

fun runProcessor(): Processor? {
    var processor: Processor? = null
    inlineAction {
        // The compiler knows that processor is a local variable and inlineAction()
        // is an inline function, so references to processor can't be leaked.
        // Therefore, it's safe to smart-cast processor.

        // If processor isn't null, processor is smart-cast
        if (processor != null) {
```

```
            // The compiler knows that processor isn't null, so no safe call
            // is needed
            processor.process()
        }

        processor = nextProcessor()
    }

    return processor
}
```

Exception handling

Smart cast information is passed on to `catch` and `finally` blocks. This makes your code safer as the compiler tracks whether your object has a nullable type. For example:

```
fun testString() {
    var stringInput: String? = null
    // stringInput is smart-cast to String type
    stringInput = ""
    try {
        // The compiler knows that stringInput isn't null
        println(stringInput.length)
        // 0

        // The compiler rejects previous smart cast information for
        // stringInput. Now stringInput has the String? type.
        stringInput = null

        // Trigger an exception
        if (2 > 1) throw Exception()
        stringInput = ""
    } catch (exception: Exception) {
        // The compiler knows stringInput can be null
        // so stringInput stays nullable.
        println(stringInput?.length)
        // null
    }
}
fun main() {
    testString()
}
```

Smart cast prerequisites

> **警告:** 请注意，当编译器能保证变量在检测及其使用之间不可改变时，智能转换才有效。

智能转换适用于以下情形：

`val` 局部变量	总是可以，局部委托属性除外。

`val` 属性	如果属性是 `private`、 `internal`，或者该检测在声明属性的同一模块中执行。 智能转换不能用于 `open` 的属性或者具有自定义 getter 的属性。
`var` 局部变量	如果变量在检测及其使用之间未修改、没有在会修改它的 lambda 中捕获、并且不是局部委托属性。
`var` 属性	决不可能，因为该变量可以随时被其他代码修改。

“不安全的”转换操作符

To explicitly cast an object to a non-nullable type, use the *unsafe* cast operator `as`:

```
val x: String = y as String
```

If the cast isn't possible, the compiler throws an exception. This is why it's called *unsafe*.

In the previous example, if `y` is `null`, the code above also throws an exception. This is because `null` can't be cast to `String`, as `String` isn't nullable. To make the example work for possible null values, use a nullable type on the right-hand side of the cast:

```
val x: String? = y as String?
```

“安全的”（可空）转换操作符

为了避免异常，可以使用*安全*转换操作符 `as?`，它可以在失败时返回 `null`：

```
val x: String? = y as? String
```

请注意，尽管事实上 `as?` 的右边是一个非空类型的 `String`，但是其转换的结果是可空的。

条件与循环

If 表达式

在 Kotlin 中，`if` 是一个表达式：它会返回一个值。 因此就不需要三元运算符（`条件 ? 然后 : 否则`），因为普通的 `if` 就能胜任这个角色。

```
fun main() {
    val a = 2
    val b = 3
    var max = a
    if (a < b) max = b

    // With else
    if (a > b) {
      max = a
    } else {
      max = b
    }

  // 作为表达式
 max = if (a > b) a else b

    // You can also use `else if` in expressions:
    val maxLimit = 1
    val maxOrLimit = if (maxLimit > a) maxLimit else if (a > b) a else b

    println("max is $max")
    // max is 3
    println("maxOrLimit is $maxOrLimit")
    // maxOrLimit is 3
}
```

`if` 表达式的分支可以是代码块，这种情况最后的表达式作为该块的值：

```
val max = if (a > b) {
    print("Choose a")
    a
} else {
    print("Choose b")
    b
}
```

If you're using `if` as an expression, for example, for returning its value or assigning it to a variable, the `else` branch is mandatory.

When expressions and statements

`when` is a conditional expression that runs code based on multiple possible values or conditions. It is similar to the `switch` statement in Java, C, and similar languages. For example:

```
fun main() {
    val x = 2
    when (x) {
        1 -> print("x == 1")
        2 -> print("x == 2")
        else -> print("x is neither 1 nor 2")
    }
    // x == 2
}
```

`when` 将它的参数与所有的分支条件顺序比较，直到某个分支满足条件。

You can use `when` in a few different ways. Firstly, you can use `when` either as an **expression** or as a **statement**. As an expression, `when` returns a value for later use in your code. As a statement, `when` completes an action without returning anything of further use:

Expression	Statement

`// Returns a string assigned to the`
`// text variable`
`val text = when (x) {`
`  1 -> "x == 1"`
`  2 -> "x == 2"`
`  else -> "x is neither 1 nor 2"`
`}`
	`// Returns nothing but triggers a`
	`// print statement`
	`when (x) {`
	`  1 -> print("x == 1")`
	`  2 -> print("x == 2")`
	`  else -> print("x is neither 1 nor 2")`
	`}`

Secondly, you can use `when` with or without a subject. Whether you use a subject with `when` or not, your expression or statement behaves the same. We recommend using `when` with a subject when possible, as it makes your code easier to read and maintain by clearly showing what you're checking.

With subject `x`	Without subject
`when(x) { ... }`	`when { ... }`

Depending on how you use `when`, there are different requirements for whether you need to cover all possible cases in your branches.

If you use `when` as a statement, you don't have to cover all possible cases. In this example, some cases aren't covered, so nothing happens. However, no error occurs:

```
fun main() {
    val x = 3
    when (x) {
        // Not all cases are covered
        1 -> print("x == 1")
        2 -> print("x == 2")
    }
}
```

In a `when` statement, the values of individual branches are ignored. Just like with `if`, each branch can be a block, and its value is the value of the last expression in the block.

If you use `when` as an expression, you have to cover all possible cases. In other words, it must be *exhaustive*. The value of the first matching branch becomes the value of the overall expression. If you don't cover all cases, the compiler throws an error.

If your `when` expression has a subject, you can use an `else` branch to make sure that all possible cases are covered, but it isn't mandatory. For example, if your subject is a `Boolean`, `enum` class, `sealed` class, or one of their nullable counterparts, you can cover all cases without an `else` branch:

```
enum class Bit {
    ZERO, ONE
}

val numericValue = when (getRandomBit()) {
    // No else branch is needed because all cases are covered
    Bit.ZERO -> 0
    Bit.ONE -> 1
}
```

If your `when` expression **doesn't** have a subject, you **must** have an `else` branch or the compiler throws an error. The `else` branch is evaluated when none of the other branch conditions are satisfied:

```
val message = when {
    a > b -> "a is greater than b"
    a < b -> "a is less than b"
    else -> "a is equal to b"
}
```

`when` expressions and statements offer different ways to simplify your code, handle multiple conditions, and perform type checks.

You can define a common behavior for multiple cases by combining their conditions in a single line with a comma:

```
when (x) {
    0, 1 -> print("x == 0 or x == 1")
```

```
    else -> print("otherwise")
}
```

可以用任意表达式（而不只是常量）作为分支条件：

```
when (x) {
    s.toInt() -> print("s encodes x")
    else -> print("s does not encode x")
}
```

You can also check whether a value is or isn't contained in a range or collection via the `in` or `!in` keywords:

```
when (x) {
    in 1..10 -> print("x is in the range")
    in validNumbers -> print("x is valid")
    !in 10..20 -> print("x is outside the range")
    else -> print("none of the above")
}
```

Additionally, you can check that a value is or isn't a particular type via the `is` or `!is` keywords. 注意： 由于智能转换，可以访问该类型的成员函数与属性而无需任何额外的检测。

```
fun hasPrefix(x: Any) = when(x) {
    is String -> x.startsWith("prefix")
    else -> false
}
```

可以用 `when` 来取代 `if-else if` 链。 如果没有主语（subject，译注：指 `when` 所判断的表达式），所有的分支条件都是简单的布尔表达式。第一个条件为 `true` 的分支会执行：

```
when {
    x.isOdd() -> print("x is odd")
    y.isEven() -> print("y is even")
    else -> print("x+y is odd")
}
```

可以使用以下语法将主语捕获到变量中：

```
fun Request.getBody() =
    when (val response = executeRequest()) {
        is Success -> response.body
        is HttpError -> throw HttpException(response.status)
    }
```

作为主语引入的变量的作用域仅限于 `when` 表达式或语句的主体。

Guard conditions in when expressions

> **警告:** Guard conditions are an experimental feature that may be changed at any time. We would appreciate your feedback in YouTrack.

Guard conditions allow you to include more than one condition to the branches of a `when` expression, making complex control flow more explicit and concise. You can use guard conditions in `when` expressions or statements with a subject.

To include a guard condition in a branch, place it after the primary condition, separated by `if`:

```
sealed interface Animal {
    data class Cat(val mouseHunter: Boolean) : Animal
    data class Dog(val breed: String) : Animal
}

fun feedAnimal(animal: Animal) {
    when (animal) {
        // Branch with only primary condition. Calls `feedDog()` when `animal` is `Dog`
        is Animal.Dog -> feedDog()
        // Branch with both primary and guard conditions. Calls `feedCat()` when `animal` is
`Cat` and not `mouseHunter`
        is Animal.Cat if !animal.mouseHunter -> feedCat()
        // Prints "Unknown animal" if none of the above conditions match
        else -> println("Unknown animal")
    }
}
```

In a single `when` expression, you can combine branches with and without guard conditions. The code in a branch with a guard condition runs only if both the primary condition and the guard condition evaluate to true. If the primary condition does not match, the guard condition is not evaluated.

If you use guard conditions in `when` statements without an `else` branch, and none of the conditions matches, none of the branches is executed.

Otherwise, if you use guard conditions in `when` expressions without an `else` branch, the compiler requires you to declare all the possible cases to avoid runtime errors.

Additionally, guard conditions support `else if`:

```
when (animal) {
    // Checks if `animal` is `Dog`
    is Animal.Dog -> feedDog()
    // Guard condition that checks if `animal` is `Cat` and not `mouseHunter`
    is Animal.Cat if !animal.mouseHunter -> feedCat()
    // Calls giveLettuce() if none of the above conditions match and animal.eatsPlants is tr
ue
    else if animal.eatsPlants -> giveLettuce()
    // Prints "Unknown animal" if none of the above conditions match
```

```
    else -> println("Unknown animal")
}
```

Combine multiple guard conditions within a single branch using the boolean operators `&&` (AND) or `||` (OR). Use parentheses around the boolean expressions to avoid confusion:

```
when (animal) {
    is Animal.Cat if (!animal.mouseHunter && animal.hungry) -> feedCat()
}
```

You can use guard conditions in any `when` expression or statement with a subject, except the case when you have multiple conditions separated by a comma. For example, `0, 1 -> print("x == 0 or x == 1")`.

> **注意:** To enable guard conditions in CLI, run the following command:
>
> `kotlinc -Xwhen-guards main.kt`
>
> To enable guard conditions in Gradle, add the following line to the `build.gradle.kts` file:
>
> `kotlin.compilerOptions.freeCompilerArgs.add("-Xwhen-guards")`

For 循环

`for` 循环可以对任何提供迭代器（iterator）的对象进行遍历，这相当于像 C# 这样的语言中的 `foreach` 循环。 `for` 的语法如下所示：

```
for (item in collection) print(item)
```

`for` 循环体可以是一个代码块。

```
for (item: Int in ints) {
    // ……
}
```

如上所述， `for` 可以循环遍历任何提供了迭代器的对象。这意味着：

- 有一个成员函数或者扩展函数 `iterator()` 返回 `Iterator<>`，其中：
  - 有一个成员函数或者扩展函数 `next()`
  - 有一个成员函数或者扩展函数 `hasNext()` 返回 `Boolean`。

这三个函数都需要标记为 `operator`。

如需在数字区间上迭代，请使用区间表达式：

```
fun main() {
    for (i in 1..3) {
        print(i)
    }
    for (i in 6 downTo 0 step 2) {
        print(i)
    }
    // 1236420
}
```

对区间或者数组的 `for` 循环会被编译为并不创建迭代器的基于索引的循环。

如果你想要通过索引遍历一个数组或者一个 list，你可以这么做：

```
fun main() {
val array = arrayOf("a", "b", "c")
    for (i in array.indices) {
        print(array[i])
    }
    // abc
}
```

或者你可以用库函数 `withIndex`：

```
fun main() {
    val array = arrayOf("a", "b", "c")
    for ((index, value) in array.withIndex()) {
        println("the element at $index is $value")
    }
    // the element at 0 is a
    // the element at 1 is b
    // the element at 2 is c
}
```

while 循环

`while` 和 `do-while` 当循环条件满足时会持续处理它们的主体。 它们之间的区别在于条件检查的时间：

- `while` 先检查条件，如果满足，则处理主体，然后再返回到条件检查。
- `do-while` 先处理主体，然后检查条件。如果满足，则循环重复。所以 `do-while` 的主体至少运行一次，不管条件如何。

```
while (x > 0) {
    x--
}

do {
  val y = retrieveData()
} while (y != null) // y 在此处可见
```

循环中的 break 与 continue

在循环中 Kotlin 支持传统的 `break` 与 `continue` 操作符。参见返回与跳转。

返回与跳转

Kotlin 有三种结构化跳转表达式：

- `return` 默认从最直接包围它的函数或者匿名函数返回。
- `break` 终止最直接包围它的循环。
- `continue` 继续下一次最直接包围它的循环。

所有这些表达式都可以用作更大表达式的一部分：

```
val s = person.name ?: return
```

这些表达式的类型是 Nothing 类型。

Break 与 Continue 标签

在 Kotlin 中任何表达式都可以用标签来标记。 标签的格式为标识符后跟 `@` 符号，例如：`abc@`、`fooBar@`。 要为一个表达式加标签，我们只要在其前加标签即可。

```
loop@ for (i in 1..100) {
    // ……
}
```

现在，可以用标签限定 `break` 或者 `continue`：

```
loop@ for (i in 1..100) {
    for (j in 1..100) {
        if (……) break@loop
    }
}
```

标签限定的 `break` 跳转到刚好位于该标签指定的循环后面的执行点。 `continue` 继续标签指定的循环的下一次迭代。

> **注意：** In some cases, you can apply `break` and `continue` *non-locally* without explicitly defining labels. Such non-local usages are valid in lambda expressions used in enclosing inline functions.

返回到标签

Kotlin 中函数可以使用函数字面量、局部函数与对象表达式实现嵌套。 标签限定的 `return` 让你可能从外层函数返回。

最重要的一个用途就是从 lambda 表达式中返回。 To return from a lambda expression, label it and qualify the `return`:

```
fun foo() {
    listOf(1, 2, 3, 4, 5).forEach lit@{
        if (it == 3) return@lit // 局部返回到该 lambda 表达式的调用者—forEach 循环
        print(it)
    }
    print(" done with explicit label")
}
fun main() {
    foo()
}
```

现在，它只会从 lambda 表达式中返回。通常情况下使用*隐式*标签更方便，因为该标签与接受该 lambda 的函数同名。

```
fun foo() {
    listOf(1, 2, 3, 4, 5).forEach {
        if (it == 3) return@forEach // 局部返回到该 lambda 表达式的调用者—forEach 循环
        print(it)
    }
    print(" done with implicit label")
}
fun main() {
    foo()
}
```

或者，我们用一个匿名函数替代 lambda 表达式。 匿名函数内部的 `return` 语句将从该匿名函数自身返回

```
fun foo() {
    listOf(1, 2, 3, 4, 5).forEach(fun(value: Int) {
        if (value == 3) return  // 局部返回到匿名函数的调用者—forEach 循环
        print(value)
    })
    print(" done with anonymous function")
}
fun main() {
    foo()
}
```

请注意，前文三个示例中使用的局部返回类似于在常规循环中使用 `continue`。

并没有 `break` 的直接等价形式，不过可以通过增加另一层嵌套 lambda 表达式并从其中非局部返回来模拟：

```
fun foo() {
    run loop@{
        listOf(1, 2, 3, 4, 5).forEach {
            if (it == 3) return@loop // 从传入 run 的 lambda 表达式非局部返回
            print(it)
        }
    }
```

```
    print(" done with nested loop")
}
fun main() {
    foo()
}
```

当要返一个回值的时候，解析器优先选用标签限定的返回：

```
return@a 1
```

这意味着“返回 `1` 到 `@a`”，而不是“返回一个标签标注的表达式 `(@a 1)`”。

> **注意：** In some cases, you can return from a lambda expression without using labels. Such *non-local* returns are located in a lambda but exit the enclosing inline function.

包与导入

源文件通常以包声明开头：

```
package org.example

fun printMessage() { /*……*/ }
class Message { /*……*/ }

// ……
```

源文件所有内容（无论是类还是函数）都包含在该包内。 所以上例中 `printMessage()` 的全名是 `org.example.printMessage`， 而 `Message` 的全名是 `org.example.Message`。

如果没有指明包，该文件的内容属于无名字的*默认*包。

默认导入

有多个包会默认导入到每个 Kotlin 文件中：

- kotlin.*
- kotlin.annotation.*
- kotlin.collections.*
- kotlin.comparisons.*
- kotlin.io.*
- kotlin.ranges.*
- kotlin.sequences.*
- kotlin.text.*

根据目标平台还会导入额外的包：

- JVM:
    - java.lang.*
    - kotlin.jvm.*
- JS:
    - kotlin.js.*

导入

除了默认导入之外，每个文件可以包含它自己的导入（`import`）指令。

可以导入一个单个名称：

```
import org.example.Message // 现在 Message 可以不用限定符访问
```

也可以导入一个作用域下的所有内容：包、类、对象等：

```
import org.example.* // “org.example”中的一切都可访问
```

如果出现名字冲突，可以使用 `as` 关键字在本地重命名冲突项来消歧义：

```
import org.example.Message // Message 可访问
import org.test.Message as TestMessage // TestMessage 代表“org.test.Message”
```

关键字 `import` 并不仅限于导入类；也可用它来导入其他声明：

- 顶层函数及属性
- 在对象声明中声明的函数和属性
- 枚举常量

顶层声明的可见性

如果顶层声明是 `private` 的，它是声明它的文件所私有的（参见 可见性修饰符）。

类

Kotlin 中使用关键字 `class` 声明类

```
class Person { /*……*/ }
```

类声明由类名、类头（指定其类型参数、主构造函数等）以及由花括号包围的类体构成。类头与类体都是可选的； 如果一个类没有类体，可以省略花括号。

```
class Empty
```

构造函数

在 Kotlin 中的一个类有一个*主构造函数*并可能有一个或多个*次构造函数*。主构造函数在类头中声明，它跟在类名与可选的类型参数后。

```
class Person constructor(firstName: String) { /*……*/ }
```

如果主构造函数没有任何注解或者可见性修饰符，可以省略这个 `constructor` 关键字。

```
class Person(firstName: String) { /*……*/ }
```

The primary constructor initializes a class instance and its properties in the class header. The class header can't contain any runnable code. If you want to run some code during object creation, use *initializer blocks* inside the class body. Initializer blocks are declared with the `init` keyword followed by curly braces. Write any code that you want to run within the curly braces.

在实例初始化期间，初始化块按照它们出现在类体中的顺序执行，与属性初始化器交织在一起：

```
class InitOrderDemo(name: String) {
    val firstProperty = "First property: $name".also(::println)

    init {
        println("First initializer block that prints $name")
    }

    val secondProperty = "Second property: ${name.length}".also(::println)

    init {
        println("Second initializer block that prints ${name.length}")
    }
}
fun main() {
    InitOrderDemo("hello")
}
```

主构造的参数可以在初始化块中使用。它们也可以在类体内声明的属性初始化器中使用：

```
class Customer(name: String) {
    val customerKey = name.uppercase()
}
```

Kotlin has a concise syntax for declaring properties and initializing them from the primary constructor:

```
class Person(val firstName: String, val lastName: String, var age: Int)
```

Such declarations can also include default values of the class properties:

```
class Person(val firstName: String, val lastName: String, var isEmployed: Boolean = true)
```

声明类属性时，可以使用尾部逗号：

```
class Person(
    val firstName: String,
    val lastName: String,
    var age: Int, // 尾部逗号
) { /*……*/ }
```

与普通属性一样，主构造函数中声明的属性可以是可变的（`var`）或只读的（`val`）。

如果构造函数有注解或可见性修饰符，这个 `constructor` 关键字是必需的，并且这些修饰符在它前面：

```
class Customer public @Inject constructor(name: String) { /*……*/ }
```

Learn more about visibility modifiers.

次构造函数

类也可以声明前缀有 `constructor` 的*次构造函数*：

```
class Person(val pets: MutableList<Pet> = mutableListOf())

class Pet {
    constructor(owner: Person) {
        owner.pets.add(this) // adds this pet to the list of its owner's pets
    }
}
```

如果类有一个主构造函数，每个次构造函数需要委托给主构造函数， 可以直接委托或者通过别的次构造函数间接委托。委托到同一个类的另一个构造函数用 `this` 关键字即可：

```
class Person(val name: String) {
    val children: MutableList<Person> = mutableListOf()
```

```
    constructor(name: String, parent: Person) : this(name) {
        parent.children.add(this)
    }
}
```

请注意，初始化块中的代码实际上会成为主构造函数的一部分。对主构造函数的委托发生在访问次构造函数的第一条语句时，因此所有初始化块与属性初始化器中的代码都会在次构造函数体之前执行。

即使该类没有主构造函数，这种委托仍会隐式发生，并且仍会执行初始化块：

```
class Constructors {
    init {
        println("Init block")
    }

    constructor(i: Int) {
        println("Constructor $i")
    }
}
fun main() {
    Constructors(1)
}
```

如果一个非抽象类没有声明任何（主或次）构造函数，它会有一个生成的不带参数的主构造函数。构造函数的可见性是 public。

如果你不希望你的类有一个公有构造函数，那么声明一个带有非默认可见性的空的主构造函数：

```
class DontCreateMe private constructor() { /*……*/ }
```

> **注意：** 在 JVM 上，如果主构造函数的所有的参数都有默认值，编译器会生成一个额外的无参构造函数，它将使用默认值。这使得 Kotlin 更易于使用像 Jackson 或者 JPA 这样的通过无参构造函数创建类的实例的库。
>
> ```
> class Customer(val customerName: String = "")
> ```

创建类的实例

创建一个类的实例，只需像普通函数一样调用构造函数。You can assign the created instance to a variable:

```
val invoice = Invoice()

val customer = Customer("Joe Smith")
```

> **注意：** Kotlin does not have a `new` keyword.

创建嵌套类、内部类与匿名内部类的类实例的过程在嵌套类中有述。

类成员

类可以包含:

- 构造函数与初始化块
- 函数
- 属性
- 嵌套类与内部类
- 对象声明

继承

Classes can be derived from each other and form inheritance hierarchies. Learn more about inheritance in Kotlin.

抽象类

类以及其中的某些或全部成员可以声明为 `abstract`。 抽象成员在本类中可以不用实现。 并不需要用 `open` 标注抽象类或者函数。

```
abstract class Polygon {
    abstract fun draw()
}

class Rectangle : Polygon() {
    override fun draw() {
        // draw the rectangle
    }
}
```

可以用一个抽象成员覆盖一个非抽象的开放成员。

```
open class Polygon {
    open fun draw() {
        // some default polygon drawing method
    }
}

abstract class WildShape : Polygon() {
    // Classes that inherit WildShape need to provide their own
    // draw method instead of using the default on Polygon
    abstract override fun draw()
}
```

伴生对象

如果你需要写一个可以无需用一个类的实例来调用、但需要访问类内部的函数（例如，工厂方法），你可以把它写成该类内对象声明中的一员。

更具体地讲，如果在你的类内声明了一个伴生对象， 你就可以访问其成员，只是以类名作为限定符。

继承

在 Kotlin 中所有类都有一个共同的超类 `Any`，对于没有超类型声明的类它是默认超类：

```
class Example // 从 Any 隐式继承
```

`Any` 有三个方法：`equals()`、 `hashCode()` 与 `toString()`。因此，为所有 Kotlin 类都定义了这些方法。

默认情况下，Kotlin 类是最终（final）的——它们不能被继承。 要使一个类可继承，请用 `open` 关键字标记它：

```
open class Base // 该类开放继承
```

如需声明一个显式的超类型，请在类头中把超类型放到冒号之后：

```
open class Base(p: Int)

class Derived(p: Int) : Base(p)
```

如果派生类有一个主构造函数，其基类可以（并且必须）根据其参数在该主构造函数中初始化。

如果派生类没有主构造函数，那么每个次构造函数必须使用 `super` 关键字初始化其基类型，或委托给另一个做到这点的构造函数。 请注意，在这种情况下，不同的次构造函数可以调用基类型的不同的构造函数：

```
class MyView : View {
    constructor(ctx: Context) : super(ctx)

    constructor(ctx: Context, attrs: AttributeSet) : super(ctx, attrs)
}
```

覆盖方法

Kotlin 对于可覆盖的成员以及覆盖后的成员需要显式修饰符：

```
open class Shape {
    open fun draw() { /*……*/ }
    fun fill() { /*……*/ }
}

class Circle() : Shape() {
    override fun draw() { /*……*/ }
}
```

`Circle.draw()` 函数上必须加上 `override` 修饰符。如果没写，编译器会报错。 如果函数没有标注 `open` 如 `Shape.fill()`，那么子类中不允许定义相同签名的函数， 无论加不加 `override`。将 `open` 修饰符添加到

final 类（即没有 `open` 的类） 的成员上不起作用。

标记为 `override` 的成员本身是开放的，因此可以在子类中覆盖。如果你想禁止再次覆盖， 使用 `final` 关键字：

```
open class Rectangle() : Shape() {
    final override fun draw() { /*……*/ }
}
```

覆盖属性

属性与方法的覆盖机制相同。在超类中声明然后在派生类中重新声明的属性必须以 `override` 开头，并且它们必须具有兼容的类型。 每个声明的属性可以由具有初始化器的属性或者具有 `get` 方法的属性覆盖：

```
open class Shape {
    open val vertexCount: Int = 0
}

class Rectangle : Shape() {
    override val vertexCount = 4
}
```

你也可以用一个 `var` 属性覆盖一个 `val` 属性，但反之则不行。 这是允许的，因为一个 `val` 属性本质上声明了一个 `get` 方法， 而将其覆盖为 `var` 只是在子类中额外声明一个 `set` 方法。

请注意，你可以在主构造函数中使用 `override` 关键字作为属性声明的一部分：

```
interface Shape {
    val vertexCount: Int
}

class Rectangle(override val vertexCount: Int = 4) : Shape // 总是有 4 个顶点

class Polygon : Shape {
    override var vertexCount: Int = 0  // 以后可以设置为任何数
}
```

派生类初始化顺序

在构造派生类的新实例的过程中，第一步完成其基类的初始化 （在之前只有对基类构造函数参数的求值），这意味着它发生在派生类的初始化逻辑运行之前。

```
open class Base(val name: String) {

    init { println("Initializing a base class") }

    open val size: Int =
        name.length.also { println("Initializing size in the base class: $it") }
```

```
}

class Derived(
    name: String,
    val lastName: String,
) : Base(name.replaceFirstChar { it.uppercase() }.also { println("Argument for the base clas
s: $it") }) {

    init { println("Initializing a derived class") }

    override val size: Int =
        (super.size + lastName.length).also { println("Initializing size in the derived clas
s: $it") }
}
fun main() {
    println("Constructing the derived class(\"hello\", \"world\")")
    Derived("hello", "world")
}
```

这意味着，基类构造函数执行时，派生类中声明或覆盖的属性都还没有初始化。在基类初始化逻辑中（直接或者通过另一个覆盖的 `open` 成员的实现间接）使用任何一个这种属性，都可能导致不正确的行为或运行时故障。 设计一个基类时，应该避免在构造函数、属性初始化器或者 `init` 块中使用 `open` 成员。

调用超类实现

派生类中的代码可以使用 `super` 关键字调用其超类的函数与属性访问器的实现：

```
open class Rectangle {
    open fun draw() { println("Drawing a rectangle") }
    val borderColor: String get() = "black"
}

class FilledRectangle : Rectangle() {
    override fun draw() {
        super.draw()
        println("Filling the rectangle")
    }

    val fillColor: String get() = super.borderColor
}
```

在一个内部类中访问外部类的超类，可以使用由外部类名限定的 `super` 关键字来实现：`super@Outer`：

```
open class Rectangle {
    open fun draw() { println("Drawing a rectangle") }
    val borderColor: String get() = "black"
}
class FilledRectangle: Rectangle() {
    override fun draw() {
        val filler = Filler()
        filler.drawAndFill()
    }
```

```
    inner class Filler {
        fun fill() { println("Filling") }
        fun drawAndFill() {
            super@FilledRectangle.draw() // 调用 Rectangle 的 draw() 实现
            fill()
            println("Drawn a filled rectangle with color ${super@FilledRectangle.borderColo
r}") // 使用 Rectangle 所实现的 borderColor 的 get()
        }
    }
}
fun main() {
    val fr = FilledRectangle()
        fr.draw()
}
```

覆盖规则

在 Kotlin 中，实现继承由下述规则规定：如果一个类从它的直接超类继承相同成员的多个实现， 它必须覆盖这个成员并提供其自己的实现（也许用继承来的其中之一）。

如需表示采用从哪个超类型继承的实现，请使用由尖括号中超类型名限定的 `super`，如 `super<Base>`：

```
open class Rectangle {
    open fun draw() { /* …… */ }
}

interface Polygon {
    fun draw() { /* …… */ } // 接口成员默认就是“open”的
}

class Square() : Rectangle(), Polygon {
    // 编译器要求覆盖 draw():
    override fun draw() {
        super<Rectangle>.draw() // 调用 Rectangle.draw()
        super<Polygon>.draw() // 调用 Polygon.draw()
    }
}
```

可以同时继承 `Rectangle` 与 `Polygon`， 但是二者都有各自的 `draw()` 实现，所以必须在 `Square` 中覆盖 `draw()`， 并为其提供一个单独的实现以消除歧义。

属性

声明属性

Kotlin 类中的属性既可以用关键字 `var` 声明为可变的， 也可以用关键字 `val` 声明为只读的。

```
class Address {
    var name: String = "Holmes, Sherlock"
    var street: String = "Baker"
    var city: String = "London"
    var state: String? = null
    var zip: String = "123456"
}
```

使用一个属性，以其名称引用它即可：

```
fun copyAddress(address: Address): Address {
    val result = Address() // Kotlin 中没有“new”关键字
    result.name = address.name // 将调用访问器
    result.street = address.street
    // ……
    return result
}
```

Getter 与 Setter

声明一个属性的完整语法如下：

```
var <propertyName>[: <PropertyType>] [= <property_initializer>]
    [<getter>]
    [<setter>]
```

其初始器（initializer）、getter 和 setter 都是可选的。属性类型如果可以从初始器， 或其 getter 的返回值（如下文所示）中推断出来，也可以省略：

```
var initialized = 1 // 类型 Int、默认 getter 和 setter
// var allByDefault // 错误：需要显式初始化器，隐含默认 getter 和 setter
```

一个只读属性的语法和一个可变的属性的语法有两方面的不同： 1、只读属性的用 `val` 而不是 `var` 声明 2、只读属性不允许 setter

```
val simple: Int? // 类型 Int、默认 getter、必须在构造函数中初始化
val inferredType = 1 // 类型 Int 、默认 getter
```

可以为属性定义自定义的访问器。如果定义了一个自定义的 getter，那么每次访问该属性时都会调用它 （这让可以实现计算出的属性）。以下是一个自定义 getter 的示例：

```
class Rectangle(val width: Int, val height: Int) {
    val area: Int // property type is optional since it can be inferred from the getter's re
turn type
        get() = this.width * this.height
}
fun main() {
    val rectangle = Rectangle(3, 4)
    println("Width=${rectangle.width}, height=${rectangle.height}, area=${rectangle.area}")
}
```

如果可以从 getter 推断出属性类型，则可以省略它：

```
val area get() = this.width * this.height
```

如果定义了一个自定义的 setter，那么每次给属性赋值时都会调用它, except its initialization. 一个自定义的 setter 如下所示：

```
var stringRepresentation: String
    get() = this.toString()
    set(value) {
        setDataFromString(value) // 解析字符串并赋值给其他属性
    }
```

按照惯例，setter 参数的名称是 `value`，但是如果你喜欢你可以选择一个不同的名称。

如果需要改变对一个访问器进行注解或者改变其可见性，但是不希望改变默认的实现， 那么可以定义访问器而不定义其实现：

```
var setterVisibility: String = "abc"
    private set // 此 setter 是私有的并且有默认实现

var setterWithAnnotation: Any? = null
    @Inject set // 用 Inject 注解此 setter
```

幕后字段

在 Kotlin 中，字段仅作为属性的一部分在内存中保存其值时使用。字段不能直接声明。 然而，当一个属性需要一个幕后字段时，Kotlin 会自动提供。这个幕后字段可以使用 `field` 标识符在访问器中引用：

```
var counter = 0 // 这个初始器直接为幕后字段赋值
    set(value) {
        if (value >= 0)
            field = value
            // counter = value // ERROR StackOverflow: Using actual name 'counter' would mak
e setter recursive
    }
```

`field` 标识符只能用在属性的访问器内。

如果属性至少一个访问器使用默认实现， 或者自定义访问器通过 `field` 引用幕后字段，将会为该属性生成一个幕后字段。

例如，以下情况下就没有幕后字段：

```
val isEmpty: Boolean
    get() = this.size == 0
```

幕后属性

如果你的需求不符合这套*隐式的幕后字段*方案， 那么总可以使用 *幕后属性（backing property）*：

```
private var _table: Map<String, Int>? = null
public val table: Map<String, Int>
    get() {
        if (_table == null) {
            _table = HashMap() // 类型参数已推断出
        }
        return _table ?: throw AssertionError("Set to null by another thread")
    }
```

> **注意:** 对于 JVM 平台：通过默认 getter 和 setter 访问私有属性会被优化以避免函数调用开销。

编译期常量

如果只读属性的值在编译期是已知的，那么可以使用 `const` 修饰符将其标记为*编译期常量*。 这种属性需要满足以下要求：

- 必须位于顶层或者是 `object` 声明 或*伴生对象*的一个成员
- 必须以 `String` 或原生类型值初始化
- 不能有自定义 getter

The compiler will inline usages of the constant, replacing the reference to the constant with its actual value. However, the field will not be removed and therefore can be interacted with using reflection.

这些属性也可以用在注解中：

```
const val SUBSYSTEM_DEPRECATED: String = "This subsystem is deprecated"

@Deprecated(SUBSYSTEM_DEPRECATED) fun foo() { …… }
```

延迟初始化属性与变量

一般地，属性声明为非空类型必须在构造函数中初始化。 然而，这经常不方便。例如：属性可以通过依赖注入来初始化， 或者在单元测试的 setup 方法中初始化。 这种情况下，你不能在构造函数内提供一个非空初始器。 但你仍然想在类体中引用该属性时避免空检测。

为处理这种情况，你可以用 `lateinit` 修饰符标记该属性：

```
public class MyTest {
    lateinit var subject: TestSubject

    @SetUp fun setup() {
        subject = TestSubject()
    }

    @Test fun test() {
        subject.method()  // 直接解引用
    }
}
```

该修饰符只能用于在类体中的属性（不是在主构造函数中声明的 `var` 属性， 并且仅当该属性没有自定义 getter 或 setter 时），也用于顶层属性与局部变量。 该属性或变量必须为非空类型，并且不能是原生类型。

在初始化前访问一个 `lateinit` 属性会抛出一个特定异常，该异常明确标识该属性被访问及它没有初始化的事实。

检测一个 lateinit var 是否已初始化

要检测一个 `lateinit var` 是否已经初始化过，请在该属性的引用上使用 `.isInitialized`：

```
if (foo::bar.isInitialized) {
    println(foo.bar)
}
```

此检测仅对可词法级访问的属性可用，当声明位于同一个类型内、位于其中一个外围类型中或者位于相同文件的顶层的属性时。

覆盖属性

参见覆盖属性

委托属性

最常见的一类属性就是简单地从幕后字段中读取（以及可能的写入）， 但是使用自定义 getter 和 setter 可以实现属性的任何行为。 介于最简单的第一类与多样的第二类之间，属性可做的事情有一些常见的模式。一些示例：惰性值、 通过键值从映射（map）读取、访问数据库、访问时通知侦听器。

这些常见行为可以通过使用委托属性实现为库。

接口

Kotlin 的接口可以既包含抽象方法的声明也包含实现。与抽象类不同的是，接口无法保存状态。它可以有属性但必须声明为抽象或提供访问器实现。

使用关键字 `interface` 来定义接口：

```
interface MyInterface {
    fun bar()
    fun foo() {
      // 可选的方法体
    }
}
```

实现接口

一个类或者对象可以实现一个或多个接口：

```
class Child : MyInterface {
    override fun bar() {
        // 方法体
    }
}
```

接口中的属性

可以在接口中定义属性。在接口中声明的属性要么是抽象的，要么提供访问器的实现。在接口中声明的属性不能有幕后字段（backing field），因此接口中声明的访问器不能引用它们：

```
interface MyInterface {
    val prop: Int // 抽象的

    val propertyWithImplementation: String
        get() = "foo"

    fun foo() {
        print(prop)
    }
}

class Child : MyInterface {
    override val prop: Int = 29
}
```

接口继承

一个接口可以从其他接口派生，意味着既能提供基类型成员的实现也能声明新的函数与属性。很自然地，实现这样接口的类只需定义所缺少的实现：

```
interface Named {
    val name: String
}

interface Person : Named {
    val firstName: String
    val lastName: String

    override val name: String get() = "$firstName $lastName"
}

data class Employee(
    // 不必实现“name”
    override val firstName: String,
    override val lastName: String,
    val position: Position
) : Person
```

解决覆盖冲突

实现多个接口时，可能会遇到同一方法继承多个实现的问题：

```
interface A {
    fun foo() { print("A") }
    fun bar()
}

interface B {
    fun foo() { print("B") }
    fun bar() { print("bar") }
}

class C : A {
    override fun bar() { print("bar") }
}

class D : A, B {
    override fun foo() {
        super<A>.foo()
        super<B>.foo()
    }

    override fun bar() {
        super<B>.bar()
    }
}
```

上例中，接口 *A* 和 *B* 都定义了方法 *foo()* 和 *bar()*。 两者都实现了 *foo()*, 但是只有 *B* 实现了 *bar()* (*bar()* 在 *A* 中没有标记为抽象， 因为在接口中没有方法体时默认为抽象)。 现在，如果实现 *A* 的一个具体类 *C*，那么必须要重写 *bar()* 并实现这个抽象方法。

然而，如果从 *A* 和 *B* 派生 *D*，需要实现从多个接口继承的所有方法，并指明 *D* 应该如何实现它们。这一规则既适用于继承单个实现（*bar()*）的方法也适用于继承多个实现（*foo()*）的方法。

函数式（SAM）接口

只有一个抽象成员函数的接口称为*函数式接口*或 *单一抽象方法（SAM）*接口。函数式接口可以有多个非抽象成员函数，但只能有一个抽象成员函数。

可以用 `fun` 修饰符在 Kotlin 中声明一个函数式接口。

```
fun interface KRunnable {
   fun invoke()
}
```

SAM 转换

对于函数式接口，可以通过 lambda 表达式实现 SAM 转换，从而使代码更简洁、更有可读性。

使用 lambda 表达式可以替代手动创建实现函数式接口的类。 通过 SAM 转换， Kotlin can convert any lambda expression whose signature matches the signature of the interface's single method into the code, which dynamically instantiates the interface implementation.

例如，有这样一个 Kotlin 函数式接口：

```
fun interface IntPredicate {
   fun accept(i: Int): Boolean
}
```

如果不使用 SAM 转换，那么你需要像这样编写代码：

```
// 创建一个类的实例
val isEven = object : IntPredicate {
   override fun accept(i: Int): Boolean {
       return i % 2 == 0
   }
}
```

通过利用 Kotlin 的 SAM 转换，可以改为以下等效代码：

```
// 通过 lambda 表达式创建一个实例
val isEven = IntPredicate { it % 2 == 0 }
```

可以通过更短的 lambda 表达式替换所有不必要的代码。

```
fun interface IntPredicate {
   fun accept(i: Int): Boolean
}

val isEven = IntPredicate { it % 2 == 0 }
```

```
fun main() {
   println("Is 7 even? - ${isEven.accept(7)}")
}
```

你也可以使用 Java 接口上的 SAM 转换。

Migration from an interface with constructor function to a functional interface

Starting from 1.6.20, Kotlin supports callable references to functional interface constructors, which adds a source-compatible way to migrate from an interface with a constructor function to a functional interface. Consider the following code:

```
interface Printer {
    fun print()
}

fun Printer(block: () -> Unit): Printer = object : Printer { override fun print() = block()
}
```

With callable references to functional interface constructors enabled, this code can be replaced with just a functional interface declaration:

```
fun interface Printer {
    fun print()
}
```

Its constructor will be created implicitly, and any code using the `::Printer` function reference will compile. For example:

```
documentsStorage.addPrinter(::Printer)
```

Preserve the binary compatibility by marking the legacy function `Printer` with the `@Deprecated` annotation with `DeprecationLevel.HIDDEN`:

```
@Deprecated(message = "Your message about the deprecation", level = DeprecationLevel.HIDDEN)
fun Printer(...) {...}
```

函数式接口与类型别名比较

You can also simply rewrite the above using a type alias for a functional type:

```
typealias IntPredicate = (i: Int) -> Boolean

val isEven: IntPredicate = { it % 2 == 0 }
```

```
fun main() {
   println("Is 7 even? - ${isEven(7)}")
}
```

However, 函数式接口和类型别名用途并不相同。 类型别名只是现有类型的名称——它们不会创建新的类型，而函数式接口却会创建新类型。 You can provide extensions that are specific to a particular functional interface to be inapplicable for plain functions or their type aliases.

类型别名只能有一个成员，而函数式接口可以有多个非抽象成员函数以及一个抽象成员函数。 函数式接口还可以实现以及继承其他接口。

函数式接口比类型别名更灵活并且提供了更多的功能, but they can be more costly both syntactically and at runtime because they can require conversions to a specific interface. When you choose which one to use in your code, consider your needs:

- If your API needs to accept a function (any function) with some specific parameter and return types – use a simple functional type or define a type alias to give a shorter name to the corresponding functional type.
- If your API accepts a more complex entity than a function – for example, it has non-trivial contracts and/or operations on it that can't be expressed in a functional type's signature – declare a separate functional interface for it.

可见性修饰符

类、对象、接口、构造函数、方法与属性及其 setter 都可以有*可见性修饰符*。 getter 总是与属性有着相同的可见性。

在 Kotlin 中有这四个可见性修饰符：`private`、 `protected`、 `internal` 和 `public`。 默认可见性是 `public`。

在本页可以学到这些修饰符如何应用到不同类型的声明作用域。

包

函数、属性和类、对象和接口可以直接在包内的顶层声明：

```
// 文件名：example.kt
package foo

fun baz() { …… }
class Bar { …… }
```

- 如果你不使用任何可见性修饰符，默认为 `public`，这意味着你的声明将随处可见。
- 如果你声明为 `private`，它只会在声明它的文件内可见。
- 如果你声明为 `internal`，它会在相同模块内随处可见。
- `protected` 修饰符不适用于顶层声明。

  注意： 要使用另一包中可见的顶层声明，需要将其导入进来。

例如：

```
// 文件名：example.kt
package foo

private fun foo() { …… } // 在 example.kt 内可见

public var bar: Int = 5 // 该属性随处可见
    private set         // setter 只在 example.kt 内可见

internal val baz = 6    // 相同模块内可见
```

类成员

对于类内部声明的成员：

- `private` 意味着只该成员在这个类内部（包含其所有成员）可见；
- `protected` 意味着该成员具有与 `private` 一样的可见性，但也在子类中可见。

- `internal` 意味着能见到类声明的*本模块内*的任何客户端都可见其 `internal` 成员。
- `public` 意味着能见到类声明的任何客户端都可见其 `public` 成员。

> **注意:** 在 Kotlin 中，外部类不能访问内部类的 private 成员。

如果你覆盖一个 `protected` 或 `internal` 成员并且没有显式指定其可见性，该成员还会具有与原始成员相同的可见性。

例子:

```
open class Outer {
    private val a = 1
    protected open val b = 2
    internal open val c = 3
    val d = 4  // 默认 public

    protected class Nested {
        public val e: Int = 5
    }
}

class Subclass : Outer() {
    // a 不可见
    // b、c、d 可见
    // Nested 和 e 可见

    override val b = 5   // “b”为 protected
    override val c = 7   // 'c' is internal
}

class Unrelated(o: Outer) {
    // o.a、o.b 不可见
    // o.c 和 o.d 可见（相同模块）
    // Outer.Nested 不可见，Nested::e 也不可见
}
```

构造函数

使用以下语法来指定一个类的的主构造函数的可见性:

> **注意:** You need to add an explicit `constructor` keyword.

```
class C private constructor(a: Int) { …… }
```

这里的构造函数是私有（`private`）的。默认情况下，所有构造函数都是公开（`public`）的，这实际上等于类可见的地方它就可见（即 一个 `internal` 类的构造函数只能在相同模块内可见).

For sealed classes, constructors are `protected` by default. For more information, see Sealed classes.

局部声明

局部变量、函数和类不能有可见性修饰符。

模块

可见性修饰符 `internal` 意味着该成员只在相同模块内可见。更具体地说， 一个模块是编译在一起的一套 Kotlin 文件，例如：

- 一个 IntelliJ IDEA 模块
- 一个 Maven 项目
- 一个 Gradle 源代码集（例外是 `test` 源代码集可以访问 `main` 的 internal 声明）
- 一次 `<kotlinc>` Ant 任务执行所编译的一套文件

扩展

Kotlin 能够对一个类或接口扩展新功能而无需继承该类或者使用像*装饰者*这样的设计模式。 这通过叫做*扩展*的特殊声明完成。

例如，你可以为一个你不能修改的、来自第三方库中的类或接口编写一个新的函数。 这个新增的函数就像那个原始类本来就有的函数一样，可以用寻常方式调用。 这种机制称为*扩展函数*。此外，也有*扩展属性*， 允许你为一个已经存在的类添加新的属性。

扩展函数

声明一个扩展函数需用一个*接收者类型*也就是被扩展的类型来作为他的前缀。 下面代码为 `MutableList<Int>` 添加一个 `swap` 函数：

```
fun MutableList<Int>.swap(index1: Int, index2: Int) {
    val tmp = this[index1] // “this”对应该列表
    this[index1] = this[index2]
    this[index2] = tmp
}
```

这个 `this` 关键字在扩展函数内部对应到接收者对象（传过来的在点符号前的对象） 现在，可以对任意 `MutableList<Int>` 调用该函数了：

```
val list = mutableListOf(1, 2, 3)
list.swap(0, 2) // “swap()”内部的“this”会保存“list”的值
```

这个函数对任何 `MutableList<T>` 起作用，可以泛化它：

```
fun <T> MutableList<T>.swap(index1: Int, index2: Int) {
    val tmp = this[index1] // “this”对应该列表
    this[index1] = this[index2]
    this[index2] = tmp
}
```

为了在接收者类型表达式中使用泛型，需要在函数名前声明泛型参数。 For more information about generics, 参见泛型函数。

扩展是*静态解析的*

扩展不能真正的修改他们所扩展的类。通过定义一个扩展，并没有在一个类中插入新成员， 只不过是可以通过该类型的变量用点表达式去调用这个新函数。

Extension functions are dispatched *statically*. So which extension function is called is already known at compile time based on the receiver type. For example:

```
fun main() {
    open class Shape
    class Rectangle: Shape()

    fun Shape.getName() = "Shape"
    fun Rectangle.getName() = "Rectangle"

    fun printClassName(s: Shape) {
        println(s.getName())
    }

    printClassName(Rectangle())
}
```

这个例子会输出 *Shape*，因为调用的扩展函数只取决于参数 `s` 的声明类型，该类型是 `Shape` 类。

如果一个类定义有一个成员函数与一个扩展函数，而这两个函数又有相同的接收者类型、 相同的名字，并且都适用给定的参数，这种情况*总是取成员函数*。 例如：

```
fun main() {
    class Example {
        fun printFunctionType() { println("Class method") }
    }

    fun Example.printFunctionType() { println("Extension function") }

    Example().printFunctionType()
}
```

这段代码输出 *Class method*。

当然，扩展函数重载同样名字但不同签名成员函数也完全可以：

```
fun main() {
    class Example {
        fun printFunctionType() { println("Class method") }
    }

    fun Example.printFunctionType(i: Int) { println("Extension function #$i") }

    Example().printFunctionType(1)
}
```

可空接收者

注意可以为可空的接收者类型定义扩展。这样的扩展可以在对象变量上调用， 即使其值为 null。 If the receiver is `null`, then `this` is also `null`. So when defining an extension with a nullable receiver

type, we recommend performing a `this == null` check inside the function body to avoid compiler errors.

可以在没有检测 null 的时候调用 Kotlin 中的 `toString()`：检测已发生在扩展函数的内部：

```
fun Any?.toString(): String {
    if (this == null) return "null"
    // 空检测之后，“this”会自动转换为非空类型，所以下面的 toString()
    // 解析为 Any 类的成员函数
    return toString()
}
```

扩展属性

与扩展函数类似，Kotlin 支持扩展属性：

```
val <T> List<T>.lastIndex: Int
    get() = size - 1
```

> **注意：**由于扩展没有实际的将成员插入类中，因此对扩展属性来说幕后字段是无效的。这就是为什么*扩展属性不能有初始化器*。他们的行为只能由显式提供的 getter/setter 定义。

例如：

```
val House.number = 1 // 错误：扩展属性不能有初始化器
```

伴生对象的扩展

如果一个类定义有一个伴生对象，你也可以为伴生对象定义扩展函数与属性。就像伴生对象的常规成员一样，可以只使用类名作为限定符来调用伴生对象的扩展成员：

```
class MyClass {
    companion object { }  // 将被称为 "Companion"
}

fun MyClass.Companion.printCompanion() { println("companion") }

fun main() {
    MyClass.printCompanion()
}
```

扩展的作用域

大多数情况都在顶层定义扩展——直接在包里：

```
package org.example.declarations

fun List<String>.getLongestString() { /*……*/}
```

如需使用所定义包之外的一个扩展，只需在调用方导入它：

```
package org.example.usage

import org.example.declarations.getLongestString

fun main() {
    val list = listOf("red", "green", "blue")
    list.getLongestString()
}
```

更多信息参见导入

扩展声明为成员

可以在一个类内部为另一个类声明扩展。在这样的扩展内部，有多个*隐式接收者*—— 其中的对象成员可以无需通过限定符访问。扩展声明所在的类的实例称为*分发接收者*，扩展方法调用所在的接收者类型的实例称为*扩展接收者*。

```
class Host(val hostname: String) {
    fun printHostname() { print(hostname) }
}

class Connection(val host: Host, val port: Int) {
    fun printPort() { print(port) }

    fun Host.printConnectionString() {
         printHostname()   // 调用 Host.printHostname()
        print(":")
         printPort()   // 调用 Connection.printPort()
    }

    fun connect() {
         /*……*/
         host.printConnectionString()   // 调用扩展函数
    }
}

fun main() {
    Connection(Host("kotl.in"), 443).connect()
    //Host("kotl.in").printConnectionString()  // 错误，该扩展函数在 Connection 外不可用
}
```

对于分发接收者与扩展接收者的成员名字冲突的情况，扩展接收者优先。要引用分发接收者的成员你可以使用限定的 `this` 语法。

```
class Connection {
    fun Host.getConnectionString() {
        toString()         // 调用 Host.toString()
        this@Connection.toString()  // 调用 Connection.toString()
    }
}
```

声明为成员的扩展可以声明为 `open` 并在子类中覆盖。这意味着这些函数的分发对于分发接收者类型是虚拟的，但对于扩展接收者类型是静态的。

```
open class Base { }

class Derived : Base() { }

open class BaseCaller {
    open fun Base.printFunctionInfo() {
        println("Base extension function in BaseCaller")
    }

    open fun Derived.printFunctionInfo() {
        println("Derived extension function in BaseCaller")
    }

    fun call(b: Base) {
        b.printFunctionInfo()   // 调用扩展函数
    }
}

class DerivedCaller: BaseCaller() {
    override fun Base.printFunctionInfo() {
        println("Base extension function in DerivedCaller")
    }

    override fun Derived.printFunctionInfo() {
        println("Derived extension function in DerivedCaller")
    }
}

fun main() {
    BaseCaller().call(Base())   // “Base extension function in BaseCaller”
    DerivedCaller().call(Base())  // “Base extension function in DerivedCaller”——分发接收者虚拟解析
    DerivedCaller().call(Derived())  // “Base extension function in DerivedCaller”——扩展接收者静态解析
}
```

关于可见性的说明

Extensions utilize the same visibility modifiers as regular functions declared in the same scope would. For example:

- 在文件顶层声明的扩展可以访问同一文件中的其他 `private` 顶层声明。

- 如果扩展是在其接收者类型外部声明的，那么它不能访问接收者的 `private` 或 `protected` 成员。

数据类

Data classes in Kotlin are primarily used to hold data. For each data class, the compiler automatically generates additional member functions that allow you to print an instance to readable output, compare instances, copy instances, and more. Data classes are marked with `data`:

```
data class User(val name: String, val age: Int)
```

编译器自动从主构造函数中声明的所有属性导出以下成员：

- `equals()`/`hashCode()` 对。
- `toString()` 格式是 `"User(name=John, age=42)"`。
- `componentN()` 函数 按声明顺序对应于所有属性。
- `copy()` 函数（见下文）

为了确保生成的代码的一致性以及有意义的行为，数据类必须满足以下要求：

- 主构造函数必须至少有一个参数。
- 主构造函数的所有参数必须标记为 `val` 或 `var`。
- 数据类不能是抽象、开放、密封或者内部的。

此外，数据类成员的生成遵循关于成员继承的这些规则：

- 如果在数据类体中有显式实现 `equals()`、 `hashCode()` 或者 `toString()`，或者这些函数在父类中有 `final` 实现，那么不会生成这些函数，而会使用现有函数。
- 如果超类型具有 `open` 的 `componentN()` 函数并且返回兼容的类型， 那么会为数据类生成相应的函数，并覆盖超类的实现。如果超类型的这些函数由于签名不兼容或者是 final 而导致无法覆盖，那么会报错。
- 不允许为 `componentN()` 以及 `copy()` 函数提供显式实现。

数据类可以扩展其他类（示例请参见密封类）。

> **注意:** 在 JVM 中，如果生成的类需要含有一个无参的构造函数，那么属性必须指定默认值。（参见构造函数）：
>
> ```
> data class User(val name: String = "", val age: Int = 0)
> ```

在类体中声明的属性

请注意，对于那些自动生成的函数，编译器只使用在主构造函数内部定义的属性。 如需在生成的实现中排除一个属性，请将其声明在类体中：

```
data class Person(val name: String) {
    var age: Int = 0
}
```

In the example below, only the `name` property is used by default inside the `toString()`, `equals()`, `hashCode()`, and `copy()` implementations, and there is only one component function, `component1()`. The `age` property is declared inside the class body and is excluded. Therefore, two `Person` objects with the same `name` but different `age` values are considered equal since `equals()` only evaluates properties from the primary constructor:

```
data class Person(val name: String) {
    var age: Int = 0
}
fun main() {
    val person1 = Person("John")
    val person2 = Person("John")
    person1.age = 10
    person2.age = 20

    println("person1 == person2: ${person1 == person2}")
    // person1 == person2: true

    println("person1 with age ${person1.age}: ${person1}")
    // person1 with age 10: Person(name=John)

    println("person2 with age ${person2.age}: ${person2}")
    // person2 with age 20: Person(name=John)
}
```

复制

Use the `copy()` function to copy an object, allowing you to alter *some* of its properties while keeping the rest unchanged. The implementation of this function for the `User` class above would be as follows:

```
fun copy(name: String = this.name, age: Int = this.age) = User(name, age)
```

然后可以这样写：

```
val jack = User(name = "Jack", age = 1)
val olderJack = jack.copy(age = 2)
```

数据类与解构声明

为数据类生成的 *component 函数* 使它们可在解构声明中使用：

```
val jane = User("Jane", 35)
val (name, age) = jane
println("$name, $age years of age")
// Jane, 35 years of age
```

标准数据类

标准库提供了 `Pair` 与 `Triple` 类。尽管在很多情况下具名数据类是更好的设计选择， 因为它们通过为属性提供有意义的名称使代码更具可读性。

泛型：in、out、where

Kotlin 中的类可以有类型参数，与 Java 类似：

```
class Box<T>(t: T) {
    var value = t
}
```

创建这样类的实例只需提供类型参数即可：

```
val box: Box<Int> = Box<Int>(1)
```

但是如果类型参数可以推断出来，例如从构造函数的参数或者从其他途径， 就可以省略类型参数：

```
val box = Box(1) // 1 具有类型 Int，所以编译器推算出它是 Box<Int>
```

型变

Java 类型系统中最棘手的部分之一是通配符类型（参见 Java Generics FAQ）。 而 Kotlin 中没有。 相反，Kotlin 有声明处型变（declaration-site variance）与类型投影（type projections）。

Variance and wildcards in Java

我们来思考下为什么 Java 需要这些神秘的通配符。 首先，Java 中的泛型是*不型变的*， 这意味着 `List<String>` 并*不是* `List<Object>` 的子类型。 如果 `List` 不是*不型变的*，它就没比 Java 的数组好到哪去，因为如下代码会通过编译但是导致运行时异常：

```
// Java
List<String> strs = new ArrayList<String>();

// Java reports a type mismatch here at compile-time.
List<Object> objs = strs;

// What if it didn't?
// We would be able to put an Integer into a list of Strings.
objs.add(1);

// And then at runtime, Java would throw
// a ClassCastException: Integer cannot be cast to String
String s = strs.get(0);
```

Java 禁止这样的事情以保证运行时的安全。但这样会有一些影响。例如， 考虑 `Collection` 接口中的 `addAll()` 方法。该方法的签名应该是什么？直觉上， 需要这样写：

```java
// Java
interface Collection<E> …… {
    void addAll(Collection<E> items);
}
```

但随后，就无法做到以下这样（完全安全的）的事：

```java
// Java

// The following would not compile with the naive declaration of addAll:
// Collection<String> is not a subtype of Collection<Object>
void copyAll(Collection<Object> to, Collection<String> from) {
    to.addAll(from);
}
```

这就是为什么 `addAll()` 的实际签名是以下这样：

```java
// Java
interface Collection<E> …… {
    void addAll(Collection<? extends E> items);
}
```

通配符类型参数 `? extends E` 表示此方法接受 `E` *或者 E 的一个子类型*对象的集合，而不只是 `E` 自身。 这意味着我们可以安全地从其中（该集合中的元素是 E 的子类的实例）*读取* `E`，但*不能写入*， 因为我们不知道什么对象符合那个未知的 `E` 的子类型。 反过来，该限制可以得到想要的行为：`Collection<String>` 表示为 `Collection<? extends Object>` 的子类型。 简而言之，带 *extends* 限定（*上界*）的通配符类型使得类型是*协变的（covariant）*。

理解为什么这能够工作的关键相当简单：如果只能从集合中获取元素， 那么使用 `String` 的集合， 并且从其中读取 `Object` 也没问题 。反过来，如果只能向集合中 *放入* 元素 ， 就可以用 `Object` 集合并向其中放入 `String`：in Java there is `List<? super String>`, which accepts `String`s or any of its supertypes.

后者称为*逆变性（contravariance）*，并且对于 `List <? super String>` 你只能调用接受 `String` 作为参数的方法 （例如，你可以调用 `add(String)` 或者 `set(int, String)`），如果调用函数返回 `List<T>` 中的 `T`， 你得到的并非一个 `String` 而是一个 `Object`。

Joshua Bloch 在其著作《Effective Java》第三版 中很好地解释了该问题 （第 31 条：“利用有限制通配符来提升 API 的灵活性”）。 他称那些你只能*从中读取*的对象为*生产者*， 并称那些只能*向其写入*的对象为*消费者*。他建议：

> “为了灵活性最大化，在表示生产者或消费者的输入参数上使用通配符类型。”

他还提出了以下助记符：*PECS* 代表*生产者-Extends*、*消费者-Super（Producer-Extends, Consumer-Super）*。

> **注意:** 如果你使用一个生产者对象，如 `List<? extends Foo>`，在该对象上不允许调用 `add()` 或 `set()`， 但这并不意味着它是*不可变的*：例如，没有什么阻止你调用 `clear()` 从列表中删除所有

元素，因为 `clear()` 根本无需任何参数。

通配符（或其他类型的型变）保证的唯一的事情是*类型安全*。不可变性完全是另一回事。

声明处型变

假设有一个泛型接口 `Source<T>`，该接口中不存在任何以 `T` 作为参数的方法，只是方法返回 `T` 类型值：

```
// Java
interface Source<T> {
    T nextT();
}
```

那么，在 `Source <Object>` 类型的变量中存储 `Source <String>` 实例的引用是极为安全的—— 没有消费者-方法可以调用。但是 Java 并不知道这一点，并且仍然禁止这样操作：

```
// Java
void demo(Source<String> strs) {
  Source<Object> objects = strs; // ！！！在 Java 中不允许
  // ……
}
```

为了修正这一点，我们必须声明对象的类型为 `Source<? extends Object>`。这么做毫无意义， 因为我们可以像以前一样在该对象上调用所有相同的方法，所以更复杂的类型并没有带来价值。 但编译器并不知道。

在 Kotlin 中，有一种方法向编译器解释这种情况。这称为*声明处型变*： 可以标注 `Source` 的*类型参数* `T` 来确保它仅从 `Source<T>` 成员中*返回*（生产），并从不被消费。 为此请使用 `out` 修饰符：

```
interface Source<out T> {
    fun nextT(): T
}

fun demo(strs: Source<String>) {
    val objects: Source<Any> = strs // 这个没问题，因为 T 是一个 out-参数
    // ……
}
```

一般原则是：当一个类 `C` 的类型参数 `T` 被声明为 `out` 时，它就只能出现在 `C` 的成员的*输出*-位置， 但回报是 `C<Base>` 可以安全地作为 `C<Derived>` 的超类。

简而言之，可以说类 `C` 是在参数 `T` 上是*协变的*，或者说 `T` 是一个*协变的*类型参数。 可以认为 `C` 是 `T` 的*生产者*，而不是 `T` 的*消费者*。

`out` 修饰符称为*型变注解*，并且由于它在类型参数声明处提供， 所以它提供了*声明处型变*。 这与 Java 的*使用处型变*相反，其类型用途通配符使得类型协变。

另外除了 `out`，Kotlin 又补充了一个型变注解：`in`。它使得一个类型参数*逆变*，即只可以消费而不可以生产。逆变类型的一个很好的例子是 `Comparable`：

```
interface Comparable<in T> {
    operator fun compareTo(other: T): Int
}

fun demo(x: Comparable<Number>) {
    x.compareTo(1.0) // 1.0 拥有类型 Double，它是 Number 的子类型
    // 因此，可以将 x 赋给类型为 Comparable <Double> 的变量
    val y: Comparable<Double> = x // OK!
}
```

in 和 *out* 两词看起来是自解释的（因为它们已经在 C# 中成功使用很长时间了），因此上面提到的助记符不是真正需要的。可以将其改写为更高级的抽象：

存在性（The Existential） **变换：消费者 in, 生产者 out!** :-)

类型投影

使用处型变：类型投影

将类型参数 `T` 声明为 `out` 非常简单，并且能避免使用处子类型化的麻烦，但是有些类实际上*不能*限制为只返回 `T`！一个很好的例子是 `Array`：

```
class Array<T>(val size: Int) {
    operator fun get(index: Int): T { …… }
    operator fun set(index: Int, value: T) { …… }
}
```

该类在 `T` 上既不能是协变的也不能是逆变的。这造成了一些不灵活性。考虑下述函数：

```
fun copy(from: Array<Any>, to: Array<Any>) {
    assert(from.size == to.size)
    for (i in from.indices)
        to[i] = from[i]
}
```

这个函数应该将项目从一个数组复制到另一个数组。让我们尝试在实践中应用它：

```
val ints: Array<Int> = arrayOf(1, 2, 3)
val any = Array<Any>(3) { "" }
copy(ints, any)
//   ^ 其类型为 Array<Int> 但此处期望 Array<Any>
```

这里我们遇到同样熟悉的问题：`Array <T>` 在 `T` 上是*不型变的*，因此 `Array <Int>` 与 `Array <Any>` 都不是另一个的子类型。为什么？再次重复，因为 `copy` *可能*有非预期行为，例如它可能尝试写一个 `String` 到 `from`，并且如果我们实际上传递一个 `Int` 的数组，以后会抛 `ClassCastException` 异常。

To prohibit the `copy` function from *writing* to `from`, you can do the following:

```
fun copy(from: Array<out Any>, to: Array<Any>) { …… }
```

这就是*类型投影*：意味着 `from` 不仅仅是一个数组，而是一个受限制的（*投影的*）数组。 只可以调用返回类型为类型参数 `T` 的方法，如上，这意味着只能调用 `get()`。 这就是*使用处型变*的用法，并且是对应于 Java 的 `Array<? extends Object>`、 但更简单。

你也可以使用 `in` 投影一个类型：

```
fun fill(dest: Array<in String>, value: String) { …… }
```

`Array<in String>` 对应于 Java 的 `Array<? super String>`，也就是说，你可以传递一个 `String` 数组、`CharSequence` 数组或 `Object` 数组给 `fill()` 函数。

星投影

有时你想说，你对类型参数一无所知，但仍然希望以安全的方式使用它。 这里的安全方式是定义泛型类型的这种投影，该泛型类型的每个具体实例化都会是该投影的子类型。

Kotlin 为此提供了所谓的*星投影*语法：

- 对于 `Foo <out T : TUpper>`，其中 `T` 是一个具有上界 `TUpper` 的协变类型参数，`Foo <*>` 等价于 `Foo <out TUpper>`。 意味着当 `T` 未知时，你可以安全地从 `Foo <*>` *读取* `TUpper` 的值。
- 对于 `Foo <in T>`，其中 `T` 是一个逆变类型参数，`Foo <*>` 等价于 `Foo <in Nothing>`。 意味着当 `T` 未知时， 没有什么可以以安全的方式*写入* `Foo <*>`。
- 对于 `Foo <T : TUpper>`，其中 `T` 是一个具有上界 `TUpper` 的不型变类型参数，`Foo<*>` 对于读取值时等价于 `Foo<out TUpper>` 而对于写值时等价于`Foo<in Nothing>`。

如果泛型类型具有多个类型参数，则每个类型参数都可以单独投影。 例如，如果类型被声明为 `interface Function <in T, out U>`，可以使用以下星投影：

- `Function<*, String>` 表示 `Function<in Nothing, String>`。
- `Function<Int, *>` 表示 `Function<Int, out Any?>`。
- `Function<*, *>` 表示 `Function<in Nothing, out Any?>`。

  注意: 星投影非常像 Java 的原始类型，但是安全。

泛型函数

不仅类可以有类型参数。函数也可以有。类型参数要放在函数名称*之前*：

```
fun <T> singletonList(item: T): List<T> {
    // ……
}

fun <T> T.basicToString(): String { // 扩展函数
```

```
    // ……
}
```

要调用泛型函数，在调用处函数名*之后*指定类型参数即可：

```
val l = singletonList<Int>(1)
```

可以省略能够从上下文中推断出来的类型参数，所以以下示例同样适用：

```
val l = singletonList(1)
```

泛型约束

能够替换给定类型参数的所有可能类型的集合可以由*泛型约束*限制。

上界

最常见的约束类型是*上界*，与 Java 的 `extends` 关键字对应：

```
fun <T : Comparable<T>> sort(list: List<T>) {  …… }
```

冒号之后指定的类型是*上界*，表明只有 `Comparable<T>` 的子类型可以替代 `T`。 例如：

```
sort(listOf(1, 2, 3)) // OK。Int 是 Comparable<Int> 的子类型
sort(listOf(HashMap<Int, String>())) // 错误：HashMap<Int, String> 不是 Comparable<HashMap<In
t, String>> 的子类型
```

默认的上界（如果没有声明）是 `Any?`。在尖括号中只能指定一个上界。 如果同一类型参数需要多个上界，需要一个单独的 *where*-子句：

```
fun <T> copyWhenGreater(list: List<T>, threshold: T): List<String>
    where T : CharSequence,
          T : Comparable<T> {
    return list.filter { it > threshold }.map { it.toString() }
}
```

所传递的类型必须同时满足 `where` 子句的所有条件。在上述示例中，类型 `T` 必须 *既*实现了 `CharSequence` *也*实现了 `Comparable`。

Definitely non-nullable types

To make interoperability with generic Java classes and interfaces easier, Kotlin supports declaring a generic type parameter as **definitely non-nullable**.

To declare a generic type `T` as definitely non-nullable, declare the type with `& Any`. For example: `T & Any`.

A definitely non-nullable type must have a nullable upper bound.

The most common use case for declaring definitely non-nullable types is when you want to override a Java method that contains `@NotNull` as an argument. For example, consider the `load()` method:

```
import org.jetbrains.annotations.*;

public interface Game<T> {
    public T save(T x) {}
    @NotNull
    public T load(@NotNull T x) {}
}
```

To override the `load()` method in Kotlin successfully, you need `T1` to be declared as definitely non-nullable:

```
interface ArcadeGame<T1> : Game<T1> {
    override fun save(x: T1): T1
    // T1 is definitely non-nullable
    override fun load(x: T1 & Any): T1 & Any
}
```

When working only with Kotlin, it's unlikely that you will need to declare definitely non-nullable types explicitly because Kotlin's type inference takes care of this for you.

类型擦除

Kotlin 为泛型声明用法执行的类型安全检测在编译期进行。 运行时泛型类型的实例不保留关于其类型实参的任何信息。 其类型信息称为被*擦除*。例如， `Foo<Bar>` 与 `Foo<Baz?>` 的实例都会被擦除为 `Foo<*>`。

泛型类型检测与类型转换

由于类型擦除，并没有通用的方法在运行时检测一个泛型类型的实例是否通过指定类型参数所创建 ，并且编译器禁止这种 `is` 检测，例如 `ints is List<Int>` or `list is T` (type parameter). 当然，你可以对一个实例检测星投影的类型：

```
if (something is List<*>) {
    something.forEach { println(it) } // 每一项的类型都是 `Any?`
}
```

类似地， 当已经让一个实例的类型参数（在编译期）静态检测， 就可以对涉及非泛型部分做 `is` 检测或者类型转换。请注意， 在这种情况下，会省略尖括号：

```
fun handleStrings(list: MutableList<String>) {
    if (list is ArrayList) {
        // `list` 智能转换为 `ArrayList<String>`
    }
}
```

省略类型参数的这种语法可用于不考虑类型参数的类型转换：`list as ArrayList`。

泛型函数调用的类型参数也同样只在编译期检测。在函数体内部， 类型参数不能用于类型检测，并且类型转换为类型参数（`foo as T`）也是非受检的。 The only exclusion is inline functions with reified type parameters, which have their actual type arguments inlined at each call site. This enables type checks and casts for the type parameters. However, the restrictions described above still apply for instances of generic types used inside checks or casts. For example, in the type check `arg is T`, if `arg` is an instance of a generic type itself, its type arguments are still erased.

```
inline fun <reified A, reified B> Pair<*, *>.asPairOf(): Pair<A, B>? {
    if (first !is A || second !is B) return null
    return first as A to second as B
}

val somePair: Pair<Any?, Any?> = "items" to listOf(1, 2, 3)

val stringToSomething = somePair.asPairOf<String, Any>()
val stringToInt = somePair.asPairOf<String, Int>()
val stringToList = somePair.asPairOf<String, List<*>>()
val stringToStringList = somePair.asPairOf<String, List<String>>() // Compiles but breaks ty
pe safety!
// Expand the sample for more details
fun main() {
    println("stringToSomething = " + stringToSomething)
    println("stringToInt = " + stringToInt)
    println("stringToList = " + stringToList)
    println("stringToStringList = " + stringToStringList)
    //println(stringToStringList?.second?.forEach() {it.length}) // This will throw ClassCas
tException as list items are not String
}
```

非受检类型转换

类型转换为带有具体类型参数的泛型类型，如 `foo as List<String>` 无法在运行时检测。 当高级程序逻辑隐含了类型转换的类型安全而无法直接通过编译器推断时， 可以使用这种非受检类型转换。 See the example below.

```
fun readDictionary(file: File): Map<String, *> = file.inputStream().use {
    TODO("Read a mapping of strings to arbitrary elements.")
}

// 我们已将存有一些 `Int` 的映射保存到这个文件
val intsFile = File("ints.dictionary")
```

```
// Warning: Unchecked cast: `Map<String, *>` to `Map<String, Int>`
val intsDictionary: Map<String, Int> = readDictionary(intsFile) as Map<String, Int>
```

最后一行的类型转换会出现一个警告。编译器无法在运行时完全检测该类型转换，并且不能保证映射中的值是“Int”。

为避免未受检类型转换，可以重新设计程序结构。在上例中，可以使用具有类型安全实现的不同接口 `DictionaryReader<T>` 与 `DictionaryWriter<T>`。 可以引入合理的抽象，将未受检的类型转换从调用处移动到实现细节中。 正确使用泛型型变也有帮助。

对于泛型函数，使用具体化的类型参数可以使形如 `arg as T` 这样的类型转换受检，除非 `arg` 对应类型的*自身*类型参数已被擦除。

可以通过在产生警告的语句或声明上用注解 `@Suppress("UNCHECKED_CAST")` 标注来禁止未受检类型转换警告：

```
inline fun <reified T> List<*>.asListOfType(): List<T>? =
    if (all { it is T })
        @Suppress("UNCHECKED_CAST")
        this as List<T> else
        null
```

> **对于 JVM 平台**：数组类型（`Array<Foo>`）会保留关于其元素被擦除类型的信息，并且类型转换为一个数组类型可以部分受检： 元素类型的可空性与类型实参仍然会被擦除。例如， 如果 `foo` 是一个保存了任何 `List<*>`（无论可不可空）的数组的话，类型转换 `foo as Array<List<String>?>` 都会成功。 **注意：**

Underscore operator for type arguments

The underscore operator `_` can be used for type arguments. Use it to automatically infer a type of the argument when other types are explicitly specified:

```
abstract class SomeClass<T> {
    abstract fun execute() : T
}

class SomeImplementation : SomeClass<String>() {
    override fun execute(): String = "Test"
}

class OtherImplementation : SomeClass<Int>() {
    override fun execute(): Int = 42
}

object Runner {
    inline fun <reified S: SomeClass<T>, T> run() : T {
        return S::class.java.getDeclaredConstructor().newInstance().execute()
    }
}
```

```
fun main() {
    // T is inferred as String because SomeImplementation derives from SomeClass<String>
    val s = Runner.run<SomeImplementation, _>()
    assert(s == "Test")

    // T is inferred as Int because OtherImplementation derives from SomeClass<Int>
    val n = Runner.run<OtherImplementation, _>()
    assert(n == 42)
}
```

嵌套类与内部类

类可以嵌套在其他类中：

```
class Outer {
    private val bar: Int = 1
    class Nested {
        fun foo() = 2
    }
}

val demo = Outer.Nested().foo() // == 2
```

还可以使用带有嵌套的接口。所有类与接口的组合都是可能的：可以将接口嵌套在类中、将类嵌套在接口中、将接口嵌套在接口中。

```
interface OuterInterface {
    class InnerClass
    interface InnerInterface
}

class OuterClass {
    class InnerClass
    interface InnerInterface
}
```

内部类

标记为 `inner` 的嵌套类能够访问其外部类的成员。内部类会带有一个对外部类的对象的引用：

```
class Outer {
    private val bar: Int = 1
    inner class Inner {
        fun foo() = bar
    }
}

val demo = Outer().Inner().foo() // == 1
```

参见限定的 `this` 表达式以了解内部类中的 `this` 的消歧义用法。

匿名内部类

使用对象表达式创建匿名内部类实例：

```
window.addMouseListener(object : MouseAdapter() {

    override fun mouseClicked(e: MouseEvent) { …… }

    override fun mouseEntered(e: MouseEvent) { …… }
})
```

注意: 对于 JVM 平台, 如果对象是函数式 Java 接口（即具有单个抽象方法的 Java 接口）的实例，可以使用带接口类型前缀的 lambda 表达式创建它：

```
val listener = ActionListener { println("clicked") }
```

枚举类

枚举类的最基本的应用场景是实现类型安全的枚举：

```
enum class Direction {
    NORTH, SOUTH, WEST, EAST
}
```

每个枚举常量都是一个对象。枚举常量以逗号分隔。

因为每一个枚举都是枚举类的实例，所以可以这样初始化：

```
enum class Color(val rgb: Int) {
    RED(0xFF0000),
    GREEN(0x00FF00),
    BLUE(0x0000FF)
}
```

匿名类

枚举常量可以声明其带有相应方法以及覆盖了基类方法的自身匿名类 。

```
enum class ProtocolState {
    WAITING {
        override fun signal() = TALKING
    },

    TALKING {
        override fun signal() = WAITING
    };

    abstract fun signal(): ProtocolState
}
```

如果枚举类定义任何成员，那么使用分号将成员定义与常量定义分隔开。

在枚举类中实现接口

一个枚举类可以实现接口（但不能从类继承），可以为所有条目提供统一的接口成员实现，也可以在相应匿名类中为每个条目提供各自的实现。 只需将想要实现的接口添加到枚举类声明中即可，如下所示：

```
import java.util.function.BinaryOperator
import java.util.function.IntBinaryOperator
enum class IntArithmetics : BinaryOperator<Int>, IntBinaryOperator {
    PLUS {
        override fun apply(t: Int, u: Int): Int = t + u
```

```
    },
    TIMES {
        override fun apply(t: Int, u: Int): Int = t * u
    };

    override fun applyAsInt(t: Int, u: Int) = apply(t, u)
}
fun main() {
    val a = 13
    val b = 31
    for (f in IntArithmetics.entries) {
        println("$f($a, $b) = ${f.apply(a, b)}")
    }
}
```

All enum classes implement the Comparable interface by default. Constants in the enum class are defined in the natural order. For more information, see Ordering.

使用枚举常量

Kotlin 中的枚举类有合成的属性与方法用于列出定义的枚举常量以及通过其名称获取枚举常量。这些方法的签名如下（假设枚举类的名称是 `EnumClass`）：

```
EnumClass.valueOf(value: String): EnumClass
EnumClass.entries: EnumEntries<EnumClass> // specialized List<EnumClass>
```

Below is an example of them in action:

```
enum class RGB { RED, GREEN, BLUE }

fun main() {
    for (color in RGB.entries) println(color.toString()) // prints RED, GREEN, BLUE
    println("The first color is: ${RGB.valueOf("RED")}") // prints "The first color is: RED"
}
```

如果指定的名称与类中定义的任何枚举常量均不匹配， `valueOf()` 方法会抛出 `IllegalArgumentException` 异常。

Prior to the introduction of `entries` in Kotlin 1.9.0, the `values()` function was used to retrieve an array of enum constants.

每个枚举常量也都具有这两个属性： `name` 与 `ordinal`， 用于在枚举类声明中获取其名称与（自 0 起的）位置：

```
enum class RGB { RED, GREEN, BLUE }

fun main() {
    println(RGB.RED.name)    // prints RED
```

```
    println(RGB.RED.ordinal) // prints 0
}
```

可以使用 `enumValues<T>()` 与 `enumValueOf<T>()` 函数以泛型的方式访问枚举类中的常量。 In Kotlin 2.0.0, the `enumEntries<T>()` function is introduced as a replacement for the `enumValues<T>()` function. The `enumEntries<T>()` function returns a list of all enum entries for the given enum type `T`.

The `enumValues<T>()` function is still supported, but we recommend that you use the `enumEntries<T>()` function instead because it has less performance impact. Every time you call `enumValues<T>()` a new array is created, whereas whenever you call `enumEntries<T>()` the same list is returned each time, which is far more efficient.

For example:

```
enum class RGB { RED, GREEN, BLUE }

inline fun <reified T : Enum<T>> printAllValues() {
    println(enumEntries<T>().joinToString { it.name })
}

printAllValues<RGB>()
// RED, GREEN, BLUE
```

> For more information about inline functions and reified type parameters, see Inline functions.

Inline value classes

Sometimes it is useful to wrap a value in a class to create a more domain-specific type. 然而，由于额外的堆内存分配问题， 它会引入运行时的性能开销。此外，如果被包装的类型是原生类型，the performance hit is significant, 因为原生类型通常在运行时就进行了大量优化，然而他们的包装器却没有得到任何特殊的处理。

为了解决这类问题，Kotlin 引入了一种被称为*内联类*的特殊类。 Inline classes are a subset of value-based classes. They don't have an identity and can only hold values.

To declare an inline class, use the `value` modifier before the name of the class:

```
value class Password(private val s: String)
```

To declare an inline class for the JVM backend, use the `value` modifier along with the `@JvmInline` annotation before the class declaration:

```
// For JVM backends
@JvmInline
value class Password(private val s: String)
```

内联类必须含有唯一的一个属性在主构造函数中初始化。在运行时， 将使用这个唯一属性来表示内联类的实例（关于运行时的内部表达请参阅下文）：

```
// 不存在 'Password' 类的真实实例对象
// 在运行时，'securePassword' 仅仅包含 'String'
val securePassword = Password("Don't try this in production")
```

这就是内联类的主要特性，它灵感来源于 *inline* 这个名称：类的数据被*内联*到该类使用的地方（类似于内联函数中的代码被内联到该函数调用的地方）。

成员

内联类支持普通类中的一些功能。特别是，内联类可以声明属性与函数, have an `init` block and secondary constructors:

```
@JvmInline
value class Person(private val fullName: String) {
    init {
        require(fullName.isNotEmpty()) {
            "Full name shouldn't be empty"
        }
    }

    constructor(firstName: String, lastName: String) : this("$firstName $lastName") {
```

```
        require(lastName.isNotBlank()) {
            "Last name shouldn't be empty"
        }
    }

    val length: Int
        get() = fullName.length

    fun greet() {
        println("Hello, $fullName")
    }
}

fun main() {
    val name1 = Person("Kotlin", "Mascot")
    val name2 = Person("Kodee")
    name1.greet() // greet` 函数会作为一个静态方法被调用
    println(name2.length) // 属性的 getter 会作为一个静态方法被调用
}
```

Inline class properties cannot have backing fields. They can only have simple computable properties (no `lateinit`/delegated properties).

继承

内联类允许去继承接口

```
interface Printable {
    fun prettyPrint(): String
}

@JvmInline
value class Name(val s: String) : Printable {
    override fun prettyPrint(): String = "Let's $s!"
}

fun main() {
    val name = Name("Kotlin")
    println(name.prettyPrint()) // 仍然会作为一个静态方法被调用
}
```

禁止内联类参与到类的继承关系结构中。这就意味着内联类不能继承其他的类而且始终是 `final` 的。

表示方式

在生成的代码中，Kotlin 编译器为每个内联类保留一个包装器。内联类的实例可以在运行时表示为包装器或者基础类型。这就类似于 `Int` 可以表示为原生类型 `int` 或者包装器 `Integer`。

为了生成性能最优的代码，Kotlin 编译更倾向于使用基础类型而不是包装器。 然而，有时候使用包装器是必要的。一般来说，只要将内联类用作另一种类型， 它们就会被装箱。

```
interface I

@JvmInline
value class Foo(val i: Int) : I

fun asInline(f: Foo) {}
fun <T> asGeneric(x: T) {}
fun asInterface(i: I) {}
fun asNullable(i: Foo?) {}

fun <T> id(x: T): T = x

fun main() {
    val f = Foo(42)

    asInline(f)    // 拆箱操作: 用作 Foo 本身
    asGeneric(f)   // 装箱操作: 用作泛型类型 T
    asInterface(f) // 装箱操作: 用作类型 I
    asNullable(f)  // 装箱操作: 用作不同于 Foo 的可空类型 Foo?

    // 在下面这里例子中，'f' 首先会被装箱（当它作为参数传递给 'id' 函数时）然后又被拆箱（当它从'id'函数中被返回时）
    // 最后， 'c' 中就包含了被拆箱后的内部表达(也就是 '42')， 和 'f' 一样
    val c = id(f)
}
```

因为内联类既可以表示为基础类型有可以表示为包装器，引用相等对于内联类而言毫无意义，因此这也是被禁止的。

Inline classes can also have a generic type parameter as the underlying type. In this case, the compiler maps it to `Any?` or, generally, to the upper bound of the type parameter.

```
@JvmInline
value class UserId<T>(val value: T)

fun compute(s: UserId<String>) {} // compiler generates fun compute-<hashcode>(s: Any?)
```

名字修饰

由于内联类被编译为其基础类型，因此可能会导致各种模糊的错误，例如意想不到的平台签名冲突：

```
@JvmInline
value class UInt(val x: Int)

// 在 JVM 平台上被表示为'public final void compute(int x)'
fun compute(x: Int) { }

// 同理，在 JVM 平台上也被表示为'public final void compute(int x)'!
fun compute(x: UInt) { }
```

为了缓解这种问题，一般会通过在函数名后面拼接一些稳定的哈希码来*修饰*函数名。 因此，`fun compute(x: UInt)` 将会被表示为 `public final void compute-<hashcode>(int x)`，以此来解决冲突的问题。

Calling from Java code

You can call functions that accept inline classes from Java code. To do so, you should manually disable mangling: add the `@JvmName` annotation before the function declaration:

```
@JvmInline
value class UInt(val x: Int)

fun compute(x: Int) { }

@JvmName("computeUInt")
fun compute(x: UInt) { }
```

内联类与类型别名

初看起来，内联类似乎与类型别名非常相似。实际上，两者似乎都引入了一种新的类型，并且都在运行时表示为基础类型。

然而，关键的区别在于类型别名与其基础类型（以及具有相同基础类型的其他类型别名）是*赋值兼容*的，而内联类却不是这样。

换句话说，内联类引入了一个真实的新类型，与类型别名正好相反，类型别名仅仅是为现有的类型取了个新的替代名称 （别名）：

```
typealias NameTypeAlias = String

@JvmInline
value class NameInlineClass(val s: String)

fun acceptString(s: String) {}
fun acceptNameTypeAlias(n: NameTypeAlias) {}
fun acceptNameInlineClass(p: NameInlineClass) {}

fun main() {
    val nameAlias: NameTypeAlias = ""
    val nameInlineClass: NameInlineClass = NameInlineClass("")
    val string: String = ""

    acceptString(nameAlias) // 正确：传递别名类型的实参替代函数中基础类型的形参
    acceptString(nameInlineClass) // 错误：不能传递内联类的实参替代函数中基础类型的形参

    // And vice versa:
    acceptNameTypeAlias(string) // 正确：传递基础类型的实参替代函数中别名类型的形参
    acceptNameInlineClass(string) // 错误：不能传递基础类型的实参替代函数中内联类类型的形参
}
```

Inline classes and delegation

Implementation by delegation to inlined value of inlined class is allowed with interfaces:

```
interface MyInterface {
    fun bar()
    fun foo() = "foo"
}

@JvmInline
value class MyInterfaceWrapper(val myInterface: MyInterface) : MyInterface by myInterface

fun main() {
    val my = MyInterfaceWrapper(object : MyInterface {
        override fun bar() {
            // body
        }
    })
    println(my.foo()) // prints "foo"
}
```

对象声明与对象表达式

In Kotlin, objects allow you to define a class and create an instance of it in a single step. This is useful when you need either a reusable singleton instance or a one-time object. To handle these scenarios, Kotlin provides two key approaches: *object declarations* for creating singletons and *object expressions* for creating anonymous, one-time objects.

> **提示:** A singleton ensures that a class has only one instance and provides a global point of access to it.

Object declarations and object expressions are best used for scenarios when:

- **Using singletons for shared resources:** You need to ensure that only one instance of a class exists throughout the application. For example, managing a database connection pool.
- **Creating factory methods:** You need a convenient way to create instances efficiently. Companion objects allow you to define class-level functions and properties tied to a class, simplifying the creation and management of these instances.
- **Modifying existing class behavior temporarily:** You want to modify the behavior of an existing class without the need to create a new subclass. For example, adding temporary functionality to an object for a specific operation.
- **Type-safe design is required:** You require one-time implementations of interfaces or abstract classes using object expressions. This can be useful for scenarios like a button click handler.

对象声明

You can create single instances of objects in Kotlin using object declarations, which always have a name following the `object` keyword. This allows you to define a class and create an instance of it in a single step, which is useful for implementing singletons:

```
// Declares a Singleton object to manage data providers
object DataProviderManager {
    private val providers = mutableListOf<DataProvider>()

    // Registers a new data provider
    fun registerDataProvider(provider: DataProvider) {
        providers.add(provider)
    }

    // Retrieves all registered data providers
    val allDataProviders: Collection<DataProvider>
        get() = providers
}
// Example data provider interface
interface DataProvider {
```

```
    fun provideData(): String
}

// Example data provider implementation
class ExampleDataProvider : DataProvider {
    override fun provideData(): String {
        return "Example data"
    }
}

fun main() {
    // Creates an instance of ExampleDataProvider
    val exampleProvider = ExampleDataProvider()

    // To refer to the object, use its name directly
    DataProviderManager.registerDataProvider(exampleProvider)

    // Retrieves and prints all data providers
    println(DataProviderManager.allDataProviders.map { it.provideData() })
    // [Example data]
}
```

提示: 对象声明的初始化过程是线程安全的并且在首次访问时进行。

如需引用该对象（`object`），直接使用其名称即可：

```
DataProviderManager.registerDataProvider(exampleProvider)
```

Object declarations can also have supertypes, similar to how anonymous objects can inherit from existing classes or implement interfaces:

```
object DefaultListener : MouseAdapter() {
    override fun mouseClicked(e: MouseEvent) { …… }

    override fun mouseEntered(e: MouseEvent) { …… }
}
```

Like variable declarations, object declarations are not expressions, so they cannot be used on the right-hand side of an assignment statement:

```
// Syntax error: An object expression cannot bind a name.
val myObject = object MySingleton {
    val name = "Singleton"
}
```

对象声明不能在局部作用域，这意味着它们不能直接嵌套在函数内部。 不过，它们可以嵌套在其他对象声明或非内部类中。

Data objects

When printing a plain object declaration in Kotlin, the string representation contains both its name and the hash of the `object`:

```
object MyObject

fun main() {
    println(MyObject)
    // MyObject@hashcode
}
```

However, by marking an object declaration with the `data` modifier, you can instruct the compiler to return the actual name of the object when calling `toString()`, the same way it works for data classes:

```
data object MyDataObject {
    val number: Int = 3
}

fun main() {
    println(MyDataObject)
    // MyDataObject
}
```

Additionally, the compiler generates several functions for your `data object`:

- `toString()` returns the name of the data object
- `equals()`/`hashCode()` enables equality checks and hash-based collections

    You can't provide a custom `equals` or `hashCode` implementation for a `data object`.

The `equals()` function for a `data object` ensures that all objects that have the type of your `data object` are considered equal. In most cases, you will only have a single instance of your `data object` at runtime, since a `data object` declares a singleton. However, in the edge case where another object of the same type is generated at runtime (for example, by using platform reflection with `java.lang.reflect` or a JVM serialization library that uses this API under the hood), this ensures that the objects are treated as being equal.

> **警告:** Make sure that you only compare `data objects` structurally (using the `==` operator) and never by reference (using the `===` operator). This helps you to avoid pitfalls when more than one instance of a data object exists at runtime.

```
import java.lang.reflect.Constructor

data object MySingleton

fun main() {
```

```
    val evilTwin = createInstanceViaReflection()

    println(MySingleton)
    // MySingleton

    println(evilTwin)
    // MySingleton

    // Even when a library forcefully creates a second instance of MySingleton,
    // its equals() function returns true:
    println(MySingleton == evilTwin)
    // true

    // Don't compare data objects using ===
    println(MySingleton === evilTwin)
    // false
}

fun createInstanceViaReflection(): MySingleton {
    // Kotlin reflection does not permit the instantiation of data objects.
    // This creates a new MySingleton instance "by force" (using Java platform reflection)
    // Don't do this yourself!
    return (MySingleton.javaClass.declaredConstructors[0].apply { isAccessible = true } as C
onstructor<MySingleton>).newInstance()
}
```

The generated `hashCode()` function has a behavior that is consistent with the `equals()` function, so that all runtime instances of a `data object` have the same hash code.

Differences between data objects and data classes

While `data object` and `data class` declarations are often used together and have some similarities, there are some functions that are not generated for a `data object`:

- No `copy()` function. Because a `data object` declaration is intended to be used as singletons, no `copy()` function is generated. Singletons restrict the instantiation of a class to a single instance, which would be violated by allowing copies of the instance to be created.
- No `componentN()` function. Unlike a `data class`, a `data object` does not have any data properties. Since attempting to destructure such an object without data properties wouldn't make sense, no `componentN()` functions are generated.

Use data objects with sealed hierarchies

Data object declarations are particularly useful for sealed hierarchies like sealed classes or sealed interfaces. They allow you to maintain symmetry with any data classes you may have defined alongside the object.

In this example, declaring `EndOfFile` as a `data object` instead of a plain `object` means that it will get the `toString()` function without the need to override it manually:

```
sealed interface ReadResult
data class Number(val number: Int) : ReadResult
data class Text(val text: String) : ReadResult
data object EndOfFile : ReadResult

fun main() {
    println(Number(7))
    // Number(number=7)
    println(EndOfFile)
    // EndOfFile
}
```

伴生对象

Companion objects allow you to define class-level functions and properties. This makes it easy to create factory methods, hold constants, and access shared utilities.

类内部的对象声明可以用 `companion` 关键字标记：

```
class MyClass {
    companion object Factory {
        fun create(): MyClass = MyClass()
    }
}
```

该伴生对象（`companion object`）的成员可通过只使用类名作为限定符来调用：

```
class User(val name: String) {
    // Defines a companion object that acts as a factory for creating User instances
    companion object Factory {
        fun create(name: String): User = User(name)
    }
}

fun main(){
    // Calls the companion object's factory method using the class name as the qualifier.
    // Creates a new User instance
    val userInstance = User.create("John Doe")
    println(userInstance.name)
    // John Doe
}
```

可以省略伴生对象的名称，在这种情况下使用名称 `Companion`：

```
class User(val name: String) {
    // Defines a companion object without a name
    companion object { }
}

// Accesses the companion object
val companionUser = User.Companion
```

Class members can access `private` members of their corresponding `companion object`:

```
class User(val name: String) {
    companion object {
        private val defaultGreeting = "Hello"
    }

    fun sayHi() {
        println(defaultGreeting)
    }
}
User("Nick").sayHi()
// Hello
```

When a class name is used by itself, it acts as a reference to the companion object of the class, regardless of whether the companion object is named or not:

```
class User1 {
    // Defines a named companion object
    companion object Named {
        fun show(): String = "User1's Named Companion Object"
    }
}

// References the companion object of User1 using the class name
val reference1 = User1

class User2 {
    // Defines an unnamed companion object
    companion object {
        fun show(): String = "User2's Companion Object"
    }
}

// References the companion object of User2 using the class name
val reference2 = User2
fun main() {
    // Calls the show() function from the companion object of User1
    println(reference1.show())
    // User1's Named Companion Object

    // Calls the show() function from the companion object of User2
    println(reference2.show())
    // User2's Companion Object
}
```

虽然 Kotlin 中伴生对象的成员看起来像其他语言的静态成员， 但是它们实际上是伴生对象的实例成员，这意味着它们属于对象本身。 这让伴生对象可以实现接口：

```
interface Factory<T> {
    fun create(name: String): T
}
```

```
class User(val name: String) {
    // Defines a companion object that implements the Factory interface
    companion object : Factory<User> {
        override fun create(name: String): User = User(name)
    }
}

fun main() {
    // Uses the companion object as a Factory
    val userFactory: Factory<User> = User
    val newUser = userFactory.create("Example User")
    println(newUser.name)
    // Example User
}
```

当然，在 JVM 平台，如果使用 `@JvmStatic` 注解，你可以将伴生对象的成员生成为真正的静态方法和字段。更详细信息请参见Java 互操作性一节 。

对象表达式

Object expressions declare a class and create an instance of that class, but without naming either of them. These classes are useful for one-time use. They can either be created from scratch, inherit from existing classes, or implement interfaces. Instances of these classes are also called *anonymous objects* because they are defined by an expression, not a name.

Create anonymous objects from scratch

Object expressions start with the `object` keyword.

If the object doesn't extend any classes or implement interfaces, you can define an object's members directly inside curly braces after the `object` keyword:

```
fun main() {
    val helloWorld = object {
        val hello = "Hello"
        val world = "World"
        // Object expressions extend the Any class, which already has a toString() function,
        // so it must be overridden
        override fun toString() = "$hello $world"
    }

    print(helloWorld)
    // Hello World
}
```

Inherit anonymous objects from supertypes

如需创建一个继承自某个（或某些）类型的匿名对象， specify this type after `object` and a colon `:`. Then implement or override the members of this class as if you were inheriting from it:

```
window.addMouseListener(object : MouseAdapter() {
    override fun mouseClicked(e: MouseEvent) { /*...*/ }

    override fun mouseEntered(e: MouseEvent) { /*...*/ }
})
```

如果超类型有一个构造函数，那么传递适当的构造函数参数给它。 可以指定多个超类型，在冒号后用逗号分隔：

```
// Creates an open class BankAccount with a balance property
open class BankAccount(initialBalance: Int) {
    open val balance: Int = initialBalance
}

// Defines an interface Transaction with an execute() function
interface Transaction {
    fun execute()
}

// A function to perform a special transaction on a BankAccount
fun specialTransaction(account: BankAccount) {
    // Creates an anonymous object that inherits from the BankAccount class and implements t
he Transaction interface
    // The balance of the provided account is passed to the BankAccount superclass construct
or
    val temporaryAccount = object : BankAccount(account.balance), Transaction {

        override val balance = account.balance + 500  // Temporary bonus

        // Implements the execute() function from the Transaction interface
        override fun execute() {
            println("Executing special transaction. New balance is $balance.")
        }
    }
    // Executes the transaction
    temporaryAccount.execute()
}
fun main() {
    // Creates a BankAccount with an initial balance of 1000
    val myAccount = BankAccount(1000)
    // Performs a special transaction on the created account
    specialTransaction(myAccount)
    // Executing special transaction. New balance is 1500.
}
```

Use anonymous objects as return and value types

When you return an anonymous object from a local or `private` function or property, all the members of that anonymous object are accessible through that function or property:

```
class UserPreferences {
    private fun getPreferences() = object {
```

```
        val theme: String = "Dark"
        val fontSize: Int = 14
    }

    fun printPreferences() {
        val preferences = getPreferences()
        println("Theme: ${preferences.theme}, Font Size: ${preferences.fontSize}")
    }
}
fun main() {
    val userPreferences = UserPreferences()
    userPreferences.printPreferences()
    // Theme: Dark, Font Size: 14
}
```

This allows you to return an anonymous object with specific properties, offering a simple way to encapsulate data or behavior without creating a separate class.

If a function or property that returns an anonymous object has `public`, `protected`, or `internal` visibility, its actual type is:

- `Any` if the anonymous object doesn't have a declared supertype.
- The declared supertype of the anonymous object, if there is exactly one such type.
- The explicitly declared type if there is more than one declared supertype.

In all these cases, members added in the anonymous object are not accessible. Overridden members are accessible if they are declared in the actual type of the function or property. For example:

```
interface Notification {
    // Declares notifyUser() in the Notification interface
    fun notifyUser()
}

interface DetailedNotification

class NotificationManager {
    // The return type is Any. The message property is not accessible.
    // When the return type is Any, only members of the Any class are accessible.
    fun getNotification() = object {
        val message: String = "General notification"
    }

    // The return type is Notification because the anonymous object implements only one inte
rface
    // The notifyUser() function is accessible because it is part of the Notification interf
ace
    // The message property is not accessible because it is not declared in the Notification
interface
    fun getEmailNotification() = object : Notification {
        override fun notifyUser() {
            println("Sending email notification")
        }
```

```
        val message: String = "You've got mail!"
    }

    // The return type is DetailedNotification. The notifyUser() function and the message pr
operty are not accessible
    // Only members declared in the DetailedNotification interface are accessible
    fun getDetailedNotification(): DetailedNotification = object : Notification, DetailedNot
ification {
        override fun notifyUser() {
            println("Sending detailed notification")
        }
        val message: String = "Detailed message content"
    }
}
fun main() {
    // This produces no output
    val notificationManager = NotificationManager()

    // The message property is not accessible here because the return type is Any
    // This produces no output
    val notification = notificationManager.getNotification()

    // The notifyUser() function is accessible
    // The message property is not accessible here because the return type is Notification
    val emailNotification = notificationManager.getEmailNotification()
    emailNotification.notifyUser()
    // Sending email notification

    // The notifyUser() function and message property are not accessible here because the re
turn type is DetailedNotification
    // This produces no output
    val detailedNotification = notificationManager.getDetailedNotification()
}
```

Access variables from anonymous objects

对象表达式主体内的代码可以访问来自包含它的作用域的变量：

```
import java.awt.event.MouseAdapter
import java.awt.event.MouseEvent

fun countClicks(window: JComponent) {
    var clickCount = 0
    var enterCount = 0

    // MouseAdapter provides default implementations for mouse event functions
    // Simulates MouseAdapter handling mouse events
    window.addMouseListener(object : MouseAdapter() {
        override fun mouseClicked(e: MouseEvent) {
            clickCount++
        }

        override fun mouseEntered(e: MouseEvent) {
            enterCount++
        }
```

```
    })
    // The clickCount and enterCount variables are accessible within the object expression
}
```

对象声明与表达式之间的行为差异

对象声明与对象表达式在初始化行为方面存在差异：

- 对象表达式是在使用他们的地方*立即*执行（及初始化）的。
- 对象声明是在第一次被访问到时*延迟*初始化的。
- 伴生对象的初始化是在相应的类被加载（解析）时，与 Java 静态初始化器的语义相匹配 。

委托

委托模式已经证明是实现继承的一个很好的替代方式， 而 Kotlin 可以零样板代码地原生支持它。

`Derived` 类可以通过将其所有公有成员都委托给指定对象来实现一个接口 `Base`：

```
interface Base {
    fun print()
}

class BaseImpl(val x: Int) : Base {
    override fun print() { print(x) }
}

class Derived(b: Base) : Base by b

fun main() {
    val base = BaseImpl(10)
    Derived(base).print()
}
```

`Derived` 的超类型列表中的 `by`-子句表示 `b` 将会在 `Derived` 中内部存储， 并且编译器将生成转发给 `b` 的所有 `Base` 的方法。

覆盖由委托实现的接口成员

覆盖符合预期：编译器会使用 `override` 覆盖的实现而不是委托对象中的。如果将 `override fun printMessage() { print("abc") }` 添加到 `Derived`，那么当调用 `printMessage` 时程序会输出 *abc* 而不是 *10*：

```
interface Base {
    fun printMessage()
    fun printMessageLine()
}

class BaseImpl(val x: Int) : Base {
    override fun printMessage() { print(x) }
    override fun printMessageLine() { println(x) }
}

class Derived(b: Base) : Base by b {
    override fun printMessage() { print("abc") }
}

fun main() {
    val base = BaseImpl(10)
    Derived(base).printMessage()
    Derived(base).printMessageLine()
}
```

但请注意，以这种方式重写的成员不会在委托对象的成员中调用，委托对象的成员只能访问其自身对接口成员实现：

```
interface Base {
    val message: String
    fun print()
}

class BaseImpl(x: Int) : Base {
    override val message = "BaseImpl: x = $x"
    override fun print() { println(message) }
}

class Derived(b: Base) : Base by b {
    // 在 b 的 `print` 实现中不会访问到这个属性
    override val message = "Message of Derived"
}

fun main() {
    val b = BaseImpl(10)
    val derived = Derived(b)
    derived.print()
    println(derived.message)
}
```

Learn more about delegated properties.

属性委托

With some common kinds of properties, even though you can implement them manually every time you need them, it is more helpful to implement them once, add them to a library, and reuse them later. For example:

- 延迟属性（*lazy* properties）：其值只在首次访问时计算。
- 可观察属性（*observable* properties）：监听器会收到有关此属性变更的通知。
- 把多个属性储存在一个映射（*map*）中，而不是每个存在单独的字段中。

为了涵盖这些（以及其他）情况，Kotlin 支持 *委托属性*:

```
class Example {
    var p: String by Delegate()
}
```

语法是： `val/var <属性名>: <类型> by <表达式>`。在 `by` 后面的表达式是该 *委托*， 因为属性对应的 `get()`（与 `set()`）会被委托给它的 `getValue()` 与 `setValue()` 方法。 属性的委托不必实现接口，但是需要提供一个 `getValue()` 函数（对于 `var` 属性还有 `setValue()`）。

例如:

```
import kotlin.reflect.KProperty

class Delegate {
    operator fun getValue(thisRef: Any?, property: KProperty<*>): String {
        return "$thisRef, thank you for delegating '${property.name}' to me!"
    }

    operator fun setValue(thisRef: Any?, property: KProperty<*>, value: String) {
        println("$value has been assigned to '${property.name}' in $thisRef.")
    }
}
```

当从委托到一个 `Delegate` 实例的 `p` 读取时，将调用 `Delegate` 中的 `getValue()` 函数。 它的第一个参数是读出 `p` 的对象、第二个参数保存了对 `p` 自身的描述 （例如可以取它的名称）。

```
val e = Example()
println(e.p)
```

输出结果:

```
Example@33a17727, thank you for delegating 'p' to me!
```

类似地， 当我们给 `p` 赋值时，将调用 `setValue()` 函数。前两个参数相同， 第三个参数保存将要被赋予的值:

```
e.p = "NEW"
```

输出结果：

```
NEW has been assigned to 'p' in Example@33a17727.
```

委托对象的要求规范可以在下文找到。

可以在函数或代码块中声明一个委托属性；它不一定是类的成员。 你可以在下文找到其示例。

标准委托

Kotlin 标准库为几种有用的委托提供了工厂方法。

延迟属性 Lazy properties

`lazy()` 是接受一个 lambda 并返回一个 `Lazy <T>` 实例的函数，返回的实例可以作为实现延迟属性的委托。第一次调用 `get()` 会执行已传递给 `lazy()` 的 lambda 表达式并记录结果。 后续调用 `get()` 只是返回记录的结果。

```
val lazyValue: String by lazy {
    println("computed!")
    "Hello"
}

fun main() {
    println(lazyValue)
    println(lazyValue)
}
```

默认情况下，对于 lazy 属性的求值是*同步锁的（synchronized）*：该值只在一个线程中计算，但所有线程都会看到相同的值。如果初始化委托的同步锁不是必需的，这样可以让多个线程同时执行，那么将 `LazyThreadSafetyMode.PUBLICATION` 作为参数传给 `lazy()`。

如果你确定初始化将总是发生在与属性使用位于相同的线程， 那么可以使用 `LazyThreadSafetyMode.NONE` 模式。它不会有任何线程安全的保证以及相关的开销。

可观察属性 Observable properties

`Delegates.observable()` 接受两个参数：初始值与修改时处理程序（handler）。

每当我们给属性赋值时会调用该处理程序（在赋值*后*执行）。它有三个参数：被赋值的属性、旧值与新值：

```
import kotlin.properties.Delegates

class User {
    var name: String by Delegates.observable("<no name>") {
```

```
        prop, old, new ->
        println("$old -> $new")
    }
}

fun main() {
    val user = User()
    user.name = "first"
    user.name = "second"
}
```

如果你想截获赋值并*否决*它们，那么使用 `vetoable()` 取代 `observable()`。 在属性被赋新值*之前*会调用传递给 `vetoable` 的处理程序。

委托给另一个属性

一个属性可以把它的 getter 与 setter 委托给另一个属性。这种委托对于顶层和类的属性（成员和扩展）都可用。该委托属性可以为：

- 顶层属性
- 同一个类的成员或扩展属性
- 另一个类的成员或扩展属性

为将一个属性委托给另一个属性，应在委托名称中使用 `::` 限定符，例如，`this::delegate` 或 `MyClass::delegate`。

```
var topLevelInt: Int = 0
class ClassWithDelegate(val anotherClassInt: Int)

class MyClass(var memberInt: Int, val anotherClassInstance: ClassWithDelegate) {
    var delegatedToMember: Int by this::memberInt
    var delegatedToTopLevel: Int by ::topLevelInt

    val delegatedToAnotherClass: Int by anotherClassInstance::anotherClassInt
}
var MyClass.extDelegated: Int by ::topLevelInt
```

这是很有用的，例如，当想要以一种向后兼容的方式重命名一个属性的时候：引入一个新的属性、 使用 `@Deprecated` 注解来注解旧的属性、并委托其实现。

```
class MyClass {
   var newName: Int = 0
   @Deprecated("Use 'newName' instead", ReplaceWith("newName"))
   var oldName: Int by this::newName
}
fun main() {
   val myClass = MyClass()
   // 通知: 'oldName: Int' is deprecated.
   // Use 'newName' instead
   myClass.oldName = 42
```

```
    println(myClass.newName) // 42
}
```

将属性储存在映射中

一个常见的用例是在一个映射（map）里存储属性的值。 这经常出现在像解析 JSON 或者执行其他“动态”任务的应用中。 在这种情况下，你可以使用映射实例自身作为委托来实现委托属性。

```
class User(val map: Map<String, Any?>) {
    val name: String by map
    val age: Int     by map
}
```

在这个例子中，构造函数接受一个映射参数：

```
val user = User(mapOf(
    "name" to "John Doe",
    "age"  to 25
))
```

Delegated properties take values from this map through string keys, which are associated with the names of properties:

```
class User(val map: Map<String, Any?>) {
    val name: String by map
    val age: Int     by map
}

fun main() {
    val user = User(mapOf(
        "name" to "John Doe",
        "age"  to 25
    ))
    println(user.name) // Prints "John Doe"
    println(user.age)  // Prints 25
}
```

这也适用于 `var` 属性，如果把只读的 `Map` 换成 `MutableMap` 的话：

```
class MutableUser(val map: MutableMap<String, Any?>) {
    var name: String by map
    var age: Int     by map
}
```

局部委托属性

你可以将局部变量声明为委托属性。 例如，你可以使一个局部变量惰性初始化：

```
fun example(computeFoo: () -> Foo) {
    val memoizedFoo by lazy(computeFoo)

    if (someCondition && memoizedFoo.isValid()) {
        memoizedFoo.doSomething()
    }
}
```

`memoizedFoo` 变量只会在第一次访问时计算。 如果 `someCondition` 失败，那么该变量根本不会计算。

属性委托要求

对于一个*只读*属性（即 `val` 声明的），委托必须提供一个操作符函数 `getValue()`，该函数具有以下参数：

- `thisRef` 必须与*属性所有者*类型（对于扩展属性必须是被扩展的类型）相同或者是其超类型。
- `property` 必须是类型 `KProperty<*>` 或其超类型。

`getValue()` 必须返回与属性相同的类型（或其子类型）。

```
class Resource

class Owner {
    val valResource: Resource by ResourceDelegate()
}

class ResourceDelegate {
    operator fun getValue(thisRef: Owner, property: KProperty<*>): Resource {
        return Resource()
    }
}
```

对于一个**可变**属性（即 `var` 声明的），委托必须额外提供一个操作符函数 `setValue()`， 该函数具有以下参数：

- `thisRef` 必须与*属性所有者*类型（对于扩展属性必须是被扩展的类型）相同或者是其超类型。
- `property` 必须是类型 `KProperty<*>` 或其超类型。
- `value` 必须与属性类型相同（或者是其超类型）。

```
class Resource

class Owner {
    var varResource: Resource by ResourceDelegate()
}

class ResourceDelegate(private var resource: Resource = Resource()) {
    operator fun getValue(thisRef: Owner, property: KProperty<*>): Resource {
        return resource
    }
    operator fun setValue(thisRef: Owner, property: KProperty<*>, value: Any?) {
        if (value is Resource) {
```

```
            resource = value
        }
    }
}
```

`getValue()` 或/与 `setValue()` 函数可以通过委托类的成员函数提供或者由扩展函数提供。 当你需要委托属性到原本未提供的这些函数的对象时后者会更便利。 两函数都需要用 `operator` 关键字来进行标记。

You can create delegates as anonymous objects without creating new classes, by using the interfaces `ReadOnlyProperty` and `ReadWriteProperty` from the Kotlin standard library. They provide the required methods: `getValue()` is declared in `ReadOnlyProperty`; `ReadWriteProperty` extends it and adds `setValue()`. This means you can pass a `ReadWriteProperty` whenever a `ReadOnlyProperty` is expected.

```
fun resourceDelegate(resource: Resource = Resource()): ReadWriteProperty<Any?, Resource> =
    object : ReadWriteProperty<Any?, Resource> {
        var curValue = resource
        override fun getValue(thisRef: Any?, property: KProperty<*>): Resource = curValue
        override fun setValue(thisRef: Any?, property: KProperty<*>, value: Resource) {
            curValue = value
        }
    }

val readOnlyResource: Resource by resourceDelegate()  // ReadWriteProperty as val
var readWriteResource: Resource by resourceDelegate()
```

Translation rules for delegated properties

在底层，Kotlin 编译器会为某些类型的委托属性生成辅助属性并委托给它们。

> **注意:** For optimization purposes, the compiler *does not* generate auxiliary properties in several cases. Learn about the optimization on the example of delegating to another property.

例如，对于属性 `prop`，生成隐藏属性 `prop$delegate`，而访问器的代码只是简单地委托给这个附加属性：

```
class C {
    var prop: Type by MyDelegate()
}

// 这段是由编译器生成的相应代码：
class C {
    private val prop$delegate = MyDelegate()
    var prop: Type
        get() = prop$delegate.getValue(this, this::prop)
        set(value: Type) = prop$delegate.setValue(this, this::prop, value)
}
```

Kotlin 编译器在参数中提供了关于 `prop` 的所有必要信息：第一个参数 `this` 引用到外部类 `C` 的实例，而 `this::prop` 是 `KProperty` 类型的反射对象，该对象描述 `prop` 自身。

Optimized cases for delegated properties

The `$delegate` field will be omitted if a delegate is:

- A referenced property:

```
class C<Type> {
    private var impl: Type = ...
    var prop: Type by ::impl
}
```

- A named object:

```
object NamedObject {
    operator fun getValue(thisRef: Any?, property: KProperty<*>): String = ...
}

val s: String by NamedObject
```

- A final `val` property with a backing field and a default getter in the same module:

```
val impl: ReadOnlyProperty<Any?, String> = ...

class A {
    val s: String by impl
}
```

- A constant expression, enum entry, `this`, `null`. The example of `this`:

```
class A {
    operator fun getValue(thisRef: Any?, property: KProperty<*>) ...

    val s by this
}
```

Translation rules when delegating to another property

When delegating to another property, the Kotlin compiler generates immediate access to the referenced property. This means that the compiler doesn't generate the field `prop$delegate`. This optimization helps save memory.

Take the following code, for example:

```
class C<Type> {
    private var impl: Type = ...
```

```
    var prop: Type by ::impl
}
```

Property accessors of the `prop` variable invoke the `impl` variable directly, skipping the delegated property's `getValue` and `setValue` operators, and thus the `KProperty` reference object is not needed.

For the code above, the compiler generates the following code:

```
class C<Type> {
    private var impl: Type = ...

    var prop: Type
        get() = impl
        set(value) {
            impl = value
        }

    fun getProp$delegate(): Type = impl // This method is needed only for reflection
}
```

提供委托

通过定义 `provideDelegate` 操作符，可以扩展创建属性实现所委托对象的逻辑。 如果 `by` 右侧所使用的对象将 `provideDelegate` 定义为成员或扩展函数， 那么会调用该函数来创建属性委托实例。

One of the possible use cases of `provideDelegate` is to check the consistency of the property upon its initialization.

例如，如需在绑定之前检测属性名称，可以这样写：

```
class ResourceDelegate<T> : ReadOnlyProperty<MyUI, T> {
    override fun getValue(thisRef: MyUI, property: KProperty<*>): T { ... }
}

class ResourceLoader<T>(id: ResourceID<T>) {
    operator fun provideDelegate(
            thisRef: MyUI,
            prop: KProperty<*>
    ): ReadOnlyProperty<MyUI, T> {
        checkProperty(thisRef, prop.name)
        // 创建委托
        return ResourceDelegate()
    }

    private fun checkProperty(thisRef: MyUI, name: String) { …… }
}

class MyUI {
    fun <T> bindResource(id: ResourceID<T>): ResourceLoader<T> { …… }

    val image by bindResource(ResourceID.image_id)
```

```
    val text by bindResource(ResourceID.text_id)
}
```

`provideDelegate` 的参数与 `getValue` 的相同：

- `thisRef` 必须与 *属性所有者* 类型（对于扩展属性必须是被扩展的类型）相同或者是它的超类型；
- `property` 必须是类型 `KProperty<*>` 或其超类型。

在创建 `MyUI` 实例期间，为每个属性调用 `provideDelegate` 方法，并立即执行必要的验证。

如果没有这种拦截属性与其委托之间的绑定的能力，为了实现相同的功能， 你必须显式传递属性名，这不是很方便：

```
// 检测属性名称而不使用“provideDelegate”功能
class MyUI {
    val image by bindResource(ResourceID.image_id, "image")
    val text by bindResource(ResourceID.text_id, "text")
}

fun <T> MyUI.bindResource(
        id: ResourceID<T>,
        propertyName: String
): ReadOnlyProperty<MyUI, T> {
    checkProperty(this, propertyName)
   // 创建委托
}
```

在生成的代码中，会调用 `provideDelegate` 方法来初始化辅助的 `prop$delegate` 属性。 比较对于属性声明 `val prop: Type by MyDelegate()` 生成的代码与上面（当 `provideDelegate` 方法不存在时）生成的代码：

```
class C {
    var prop: Type by MyDelegate()
}

// 这段代码是当“provideDelegate”功能可用时
// 由编译器生成的代码：
class C {
    // 调用“provideDelegate”来创建额外的“delegate”属性
    private val prop$delegate = MyDelegate().provideDelegate(this, this::prop)
    var prop: Type
        get() = prop$delegate.getValue(this, this::prop)
        set(value: Type) = prop$delegate.setValue(this, this::prop, value)
}
```

请注意， `provideDelegate` 方法只影响辅助属性的创建，并不会影响为 getter 或 setter 生成的代码。

With the `PropertyDelegateProvider` interface from the standard library, you can create delegate providers without creating new classes.

```
val provider = PropertyDelegateProvider { thisRef: Any?, property ->
    ReadOnlyProperty<Any?, Int> {_, property -> 42 }
```

```
}
val delegate: Int by provider
```

类型别名

类型别名为现有类型提供替代名称。 如果类型名称太长，你可以另外引入较短的名称，并使用新的名称替代原类型名。

它有助于缩短较长的泛型类型。 例如，通常缩减集合类型是很有吸引力的：

```
typealias NodeSet = Set<Network.Node>

typealias FileTable<K> = MutableMap<K, MutableList<File>>
```

你可以为函数类型提供另外的别名：

```
typealias MyHandler = (Int, String, Any) -> Unit

typealias Predicate<T> = (T) -> Boolean
```

你可以为内部类和嵌套类创建新名称：

```
class A {
    inner class Inner
}
class B {
    inner class Inner
}

typealias AInner = A.Inner
typealias BInner = B.Inner
```

类型别名不会引入新类型。 它们等效于相应的底层类型。 当你在代码中添加 `typealias Predicate<T>` 并使用 `Predicate<Int>` 时，Kotlin 编译器总是把它扩展为 `(Int) -> Boolean`。 因此，当你需要泛型函数类型时，你可以传递该类型的变量，反之亦然：

```
typealias Predicate<T> = (T) -> Boolean

fun foo(p: Predicate<Int>) = p(42)

fun main() {
    val f: (Int) -> Boolean = { it > 0 }
    println(foo(f)) // 输出 "true"

    val p: Predicate<Int> = { it > 0 }
    println(listOf(1, -2).filter(p)) // 输出 "[1]"
}
```

函数

Kotlin 函数使用 `fun` 关键字声明：

```
fun double(x: Int): Int {
    return 2 * x
}
```

函数用法

使用常规方式来调用函数：

```
val result = double(2)
```

调用成员函数使用点表示法：

```
Stream().read() // 创建类 Stream 实例并调用 read()
```

参数

函数参数使用 Pascal 表示法定义——*name*: *type*。参数用逗号隔开， 每个参数必须有显式类型：

```
fun powerOf(number: Int, exponent: Int): Int { /*……*/ }
```

在声明函数参数的时候，你也可以使用尾部逗号：

```
fun powerOf(
    number: Int,
    exponent: Int, // 尾部逗号
) { /*……*/ }
```

默认实参

函数参数可以有默认值，当省略相应的参数时使用默认值。这可以减少重载数量：

```
fun read(
    b: ByteArray,
    off: Int = 0,
    len: Int = b.size,
) { /*……*/ }
```

通过在类型的后面添加 `=` 符号来设置参数的默认值。

覆盖方法总是使用与基类型方法相同的默认实参值。 当覆盖一个有默认实参值的方法时，必须从签名中省略默认实参值：

```
open class A {
    open fun foo(i: Int = 10) { /*……*/ }
}

class B : A() {
    override fun foo(i: Int) { /*……*/ }  // 不能有默认值
}
```

如果一个有默认值参数在一个无默认值的参数之前，那么该默认值只能通过使用具名实参调用该函数来使用：

```
fun foo(
    bar: Int = 0,
    baz: Int,
) { /*……*/ }

foo(baz = 1) // 使用默认值 bar = 0
```

如果在默认实参之后的最后一个参数是 lambda 表达式，那么它既可以作为具名实参在括号内传入，也可以在括号外传入：

```
fun foo(
    bar: Int = 0,
    baz: Int = 1,
    qux: () -> Unit,
) { /*……*/ }

foo(1) { println("hello") }     // 使用默认值 baz = 1
foo(qux = { println("hello") }) // 使用两个默认值 bar = 0 与 baz = 1
foo { println("hello") }        // 使用两个默认值 bar = 0 与 baz = 1
```

具名实参

在调用函数的时候可以指明一个或多个参数，这对于包含多个参数的函数是十分有帮助的，尤其是在传入 `null` 或者布尔值的情况下很难联想到参数的含义。

在调用函数的过程中使用具名实参时，你可以随意修改参数的顺序。如果想要使用某些参数的默认值，只需要在传参的时候省略掉这些参数即可。

以 `reformat()` 函数为例，该函数包含 4 个有默认值的参数。

```
fun reformat(
    str: String,
    normalizeCase: Boolean = true,
    upperCaseFirstLetter: Boolean = true,
    divideByCamelHumps: Boolean = false,
    wordSeparator: Char = ' ',
) { /*……*/ }
```

当调用这个函数时，不需要让其所有参数都具名：

```
reformat(
    "String!",
    false,
    upperCaseFirstLetter = false,
    divideByCamelHumps = true,
    '_'
)
```

你也可以省略所有有默认值的参数：

```
reformat("This is a long String!")
```

除了省略掉所有有默认值的参数，你也可以选择只省略某些特定的有默认值的参数。 但是你需要在跳过第一个参数后，对后续的所有参数都使用具名实参：

```
reformat("This is a short String!", upperCaseFirstLetter = false, wordSeparator = '_')
```

可以通过具名实参与*展开*操作符（数组前加 `*`）来传入可变实参（`vararg`）：

```
fun foo(vararg strings: String) { /*……*/ }

foo(strings = *arrayOf("a", "b", "c"))
```

> **注意：** 在 JVM 平台中调用 Java 函数时不能使用具名实参语法，因为 Java 字节码并不总是保留函数参数的名称。

返回 Unit 的函数

如果一个函数并不返回有用的值，其返回类型是 `Unit`。`Unit` 是一种只有一个值——`Unit` 的类型。 这个值不需要显式返回：

```
fun printHello(name: String?): Unit {
    if (name != null)
        println("Hello $name")
    else
        println("Hi there!")
    // `return Unit` 或者 `return` 是可选的
}
```

`Unit` 返回类型声明也是可选的。上面的代码等同于：

```
fun printHello(name: String?) { …… }
```

单表达式函数

当函数体由单个表达式构成时，可以省略花括号并且在 `=` 符号之后指定代码体即可：

```
fun double(x: Int): Int = x * 2
```

当返回值类型可由编译器推断时，显式声明返回类型是可选的：

```
fun double(x: Int) = x * 2
```

显式返回类型

具有块代码体的函数必须始终显式指定返回类型，除非他们旨在返回 `Unit`， 在这种情况下指定返回值类型是可选的。

Kotlin 不推断具有块代码体的函数的返回类型，因为这样的函数在代码体中可能有复杂的控制流， 并且返回类型对于读者（有时甚至对于编译器）是不明显的。

可变数量的参数（varargs）

函数的参数（通常是最后一个）可以用 `vararg` 修饰符标记：

```
fun <T> asList(vararg ts: T): List<T> {
    val result = ArrayList<T>()
    for (t in ts) // ts 是一个数组
        result.add(t)
    return result
}
```

在本例中，可以将可变数量的参数传递给函数：

```
val list = asList(1, 2, 3)
```

在函数内部，类型 `T` 的 `vararg` 参数的可见方式是作为 `T` 数组，如上例中的 `ts` 变量具有类型 `Array <out T>`。

只有一个参数可以标注为 `vararg`。如果 `vararg` 参数不是列表中的最后一个参数， 必须使用具名实参语法传递其后的参数的值，或者，如果参数具有函数类型，则通过在括号外部传一个 lambda。

当调用 `vararg`-函数时，可以逐个传参，例如 `asList(1, 2, 3)`。如果已经有一个数组并希望将其内容传给该函数，那么使用展开操作符（在数组前面加 `*`）：

```
val a = arrayOf(1, 2, 3)
val list = asList(-1, 0, *a, 4)
```

如果你想传入一个原生类型数组到 `vararg` 中，你需要先通过 `toTypedArray()` 函数将其转换为常规的类型化数组：

```
val a = intArrayOf(1, 2, 3) // IntArray 是一种原生类型数组
val list = asList(-1, 0, *a.toTypedArray(), 4)
```

中缀表示法

标有 `infix` 关键字的函数也可以使用中缀表示法（忽略该调用的点与圆括号）调用。 中缀函数必须满足以下要求：

- 它们必须是成员函数或扩展函数。
- 它们必须只有一个参数。
- 其参数不得接受可变数量的参数且不能有默认值。

```
infix fun Int.shl(x: Int): Int { …… }

// 用中缀表示法调用该函数
1 shl 2

// 等同于这样
1.shl(2)
```

> **注意：** 中缀函数调用的优先级低于算术操作符、类型转换以及 `rangeTo` 操作符。 以下表达式是等价的：
>
> - `1 shl 2 + 3` 等价于 `1 shl (2 + 3)`
> - `0 until n * 2` 等价于 `0 until (n * 2)`
> - `xs union ys as Set<*>` 等价于 `xs union (ys as Set<*>)`
>
> 另一方面，中缀函数调用的优先级高于布尔操作符 `&&` 与 `||`、`is-` 与 `in-` 检测以及其他一些操作符。这些表达式也是等价的：
>
> - `a && b xor c` 等价于 `a && (b xor c)`
> - `a xor b in c` 等价于 `(a xor b) in c`

请注意，中缀函数总是要求指定接收者与参数。当使用中缀表示法在当前接收者上调用方法时，需要显式使用 `this`。这是确保非模糊解析所必需的。

```
class MyStringCollection {
    infix fun add(s: String) { /*……*/ }

    fun build() {
        this add "abc"   // 正确
        add("abc")       // 正确
        //add "abc"        // 错误：必须指定接收者
    }
}
```

函数作用域

Kotlin 函数可以在文件顶层声明，这意味着你不需要像一些语言如 Java、C# 与 Scala (在 Scala 3 之后新增了顶层作用域的支持) 那样需要创建一个类来保存一个函数。此外除了顶层函数，Kotlin 中函数也可以声明在

局部作用域、作为成员函数以及扩展函数。

局部函数

Kotlin 支持局部函数，即一个函数在另一个函数内部：

```
fun dfs(graph: Graph) {
    fun dfs(current: Vertex, visited: MutableSet<Vertex>) {
        if (!visited.add(current)) return
        for (v in current.neighbors)
            dfs(v, visited)
    }

    dfs(graph.vertices[0], HashSet())
}
```

局部函数可以访问外部函数（闭包）的局部变量。在上例中，`visited` 可以是局部变量：

```
fun dfs(graph: Graph) {
    val visited = HashSet<Vertex>()
    fun dfs(current: Vertex) {
        if (!visited.add(current)) return
        for (v in current.neighbors)
            dfs(v)
    }

    dfs(graph.vertices[0])
}
```

成员函数

成员函数是在类或对象内部定义的函数：

```
class Sample {
    fun foo() { print("Foo") }
}
```

成员函数以点表示法调用：

```
Sample().foo() // 创建类 Sample 实例并调用 foo
```

关于类和覆盖成员的更多信息参见类和继承。

泛型函数

函数可以有泛型参数，通过在函数名前使用尖括号指定：

```
fun <T> singletonList(item: T): List<T> { /*……*/ }
```

关于泛型函数的更多信息，请参见泛型。

尾递归函数

Kotlin 支持一种称为尾递归的函数式编程风格。 对于某些使用循环的算法，可以使用尾递归替代而不会有堆栈溢出的风险。 当一个函数用 `tailrec` 修饰符标记并满足所需的形式条件时，编译器会优化该递归， 留下一个快速而高效的基于循环的版本：

```
val eps = 1E-10 // "good enough", could be 10^-15

tailrec fun findFixPoint(x: Double = 1.0): Double =
    if (Math.abs(x - Math.cos(x)) < eps) x else findFixPoint(Math.cos(x))
```

这段代码计算余弦的不动点（`fixpoint` of cosine），这是一个数学常数。 它只是重复地从 `1.0` 开始调用 `Math.cos`， 直到结果不再改变，对于这里指定的 `eps` 精度会产生 `0.7390851332151611` 的结果。最终代码相当于这种更传统风格的代码：

```
val eps = 1E-10 // "good enough", could be 10^-15

private fun findFixPoint(): Double {
    var x = 1.0
    while (true) {
        val y = Math.cos(x)
        if (Math.abs(x - y) < eps) return x
        x = Math.cos(x)
    }
}
```

要符合 `tailrec` 修饰符的条件的话，函数必须将其自身调用作为它执行的最后一个操作。在递归调用后有更多代码时， 不能使用尾递归，不能用在 `try/catch/finally` 块中，也不能用于 open 的函数。 目前在 Kotlin for the JVM 与 Kotlin/Native 中支持尾递归。

另见：

- 内联函数
- 扩展函数
- 高阶函数与 Lambda 表达式

高阶函数与 lambda 表达式

Kotlin 函数都是头等的，这意味着它们可以存储在变量与数据结构中，并可以作为参数传给其他高阶函数以及从其他高阶函数返回。可以像操作任何其他非函数值一样对函数进行操作。

为促成这点，作为一门静态类型编程语言的 Kotlin 使用一系列函数类型来表示函数并提供一组特定的语言结构，例如 lambda 表达式。

高阶函数

高阶函数是将函数用作参数或返回值的函数。

高阶函数的一个不错的示例是集合的函数式风格的 `fold`， 它接受一个初始累积值与一个接合函数，并通过将当前累积值与每个集合元素连续接合起来代入累积值来构建返回值：

```
fun <T, R> Collection<T>.fold(
    initial: R,
    combine: (acc: R, nextElement: T) -> R
): R {
    var accumulator: R = initial
    for (element: T in this) {
        accumulator = combine(accumulator, element)
    }
    return accumulator
}
```

在上述代码中，参数 `combine` 具有函数类型 `(R, T) -> R`，因此 `fold` 接受一个函数作为参数， 该函数接受类型分别为 `R` 与 `T` 的两个参数并返回一个 `R` 类型的值。 在 `for` 循环内部调用该函数，然后将其返回值赋值给 `accumulator`。

为了调用 `fold`，需要传给它一个函数类型的实例作为参数， 而在高阶函数调用处，（下文详述的）lambda 表达 式广泛用于此目的。

```
fun main() {
    val items = listOf(1, 2, 3, 4, 5)

    // Lambdas 表达式是花括号括起来的代码块。
    items.fold(0, {
        // 如果一个 lambda 表达式有参数，前面是参数，后跟“->”
        acc: Int, i: Int ->
        print("acc = $acc, i = $i, ")
        val result = acc + i
        println("result = $result")
        // lambda 表达式中的最后一个表达式是返回值：
        result
    })
```

```
    // lambda 表达式的参数类型是可选的，如果能够推断出来的话：
    val joinedToString = items.fold("Elements:", { acc, i -> acc + " " + i })

    // 函数引用也可以用于高阶函数调用：
    val product = items.fold(1, Int::times)
    println("joinedToString = $joinedToString")
    println("product = $product")
}
```

函数类型

Kotlin 使用类似 `(Int) -> String` 的函数类型来处理函数的声明： `val onClick: () -> Unit = ……`。

这些类型具有与函数签名相对应的特殊表示法——它们的参数和返回值：

- 所有函数类型都有一个圆括号括起来的参数类型列表以及一个返回类型：`(A, B) -> C` 表示接受类型分别为 `A` 与 `B` 两个参数并返回一个 `C` 类型值的函数类型。 参数类型列表可以为空，如 `() -> A`。`Unit` 返回类型不可省略。
- 函数类型可以有一个额外的*接收者*类型，它在表示法中的点之前指定： 类型 `A.(B) -> C` 表示可以在 `A` 的接收者对象上以一个 `B` 类型参数来调用并返回一个 `C` 类型值的函数。 带有接收者的函数字面值通常与这些类型一起使用。
- 挂起函数属于函数类型的特殊种类，它的表示法中有一个 `suspend` 修饰符 ，例如 `suspend () -> Unit` 或者 `suspend A.(B) -> C`。

函数类型表示法可以选择性地包含函数的参数名：`(x: Int, y: Int) -> Point`。 这些名称可用于表明参数的含义。

如需将函数类型指定为可空，请使用圆括号，如下所示： `((Int, Int) -> Int)?`。

函数类型还可以使用圆括号进行接合：`(Int) -> ((Int) -> Unit)`。

> **注意:** 箭头表示法是右结合的，`(Int) -> (Int) -> Unit` 与前述示例等价，但不等于 `((Int) -> (Int)) -> Unit`。

还可以通过使用类型别名给函数类型起一个别称：

```
typealias ClickHandler = (Button, ClickEvent) -> Unit
```

函数类型实例化

有几种方法可以获得函数类型的实例：

- 使用函数字面值的代码块，采用以下形式之一：
  - lambda 表达式: `{ a, b -> a + b }`，
  - 匿名函数: `fun(s: String): Int { return s.toIntOrNull() ?: 0 }`

带有接收者的函数字面值可用作带有接收者的函数类型的值。

- 使用已有声明的可调用引用：

  - 顶层、局部、成员、扩展函数：`::isOdd`、`String::toInt`，
  - 顶层、成员、扩展属性：`List<Int>::size`，
  - 构造函数：`::Regex`

  这包括指向特定实例成员的绑定的可调用引用：`foo::toString`。

- 使用实现函数类型接口的自定义类的实例：

```
class IntTransformer: (Int) -> Int {
    override operator fun invoke(x: Int): Int = TODO()
}

val intFunction: (Int) -> Int = IntTransformer()
```

如果有足够信息，编译器可以推断变量的函数类型：

```
val a = { i: Int -> i + 1 } // 推断出的类型是 (Int) -> Int
```

带与不带接收者的函数类型*非字面*值可以互换，其中接收者可以替代第一个参数，反之亦然。例如，`(A, B) -> C` 类型的值可以传给或赋值给期待 `A.(B) -> C` 类型值的地方，反之亦然：

```
fun main() {
    val repeatFun: String.(Int) -> String = { times -> this.repeat(times) }
    val twoParameters: (String, Int) -> String = repeatFun // OK

    fun runTransformation(f: (String, Int) -> String): String {
        return f("hello", 3)
    }
    val result = runTransformation(repeatFun) // OK
    println("result = $result")
}
```

> **注意：**默认情况下推断出的是没有接收者的函数类型，即使变量是通过扩展函数引用来初始化的。 如需改变这点，请显式指定变量类型。

函数类型实例调用

函数类型的值可以通过其 `invoke(……)` 操作符调用： `f.invoke(x)` 或者直接 `f(x)`。

如果该值具有接收者类型，那么应该将接收者对象作为第一个参数传递。 调用带有接收者的函数类型值的另一个方式是在其前面加上接收者对象， 就好比该值是一个扩展函数：`1.foo(2)`。

例如：

```
fun main() {
    val stringPlus: (String, String) -> String = String::plus
    val intPlus: Int.(Int) -> Int = Int::plus

    println(stringPlus.invoke("<-", "->"))
    println(stringPlus("Hello, ", "world!"))

    println(intPlus.invoke(1, 1))
    println(intPlus(1, 2))
    println(2.intPlus(3)) // 类扩展调用
}
```

内联函数

有时使用内联函数可以为高阶函数提供灵活的控制流。

Lambda 表达式与匿名函数

lambda 表达式与匿名函数是*函数字面值*，函数字面值即没有声明而是立即做为表达式传递的函数。考虑下面的例子：

```
max(strings, { a, b -> a.length < b.length })
```

函数 `max` 是一个高阶函数，因为它接受一个函数作为第二个参数。 其第二个参数是一个表达式，它本身是一个函数，称为函数字面值，它等价于以下具名函数：

```
fun compare(a: String, b: String): Boolean = a.length < b.length
```

Lambda 表达式语法

Lambda 表达式的完整语法形式如下：

```
val sum: (Int, Int) -> Int = { x: Int, y: Int -> x + y }
```

- lambda 表达式总是括在花括号中。
- 完整语法形式的参数声明放在花括号内，并有可选的类型标注。
- 函数体跟在一个 `->` 之后。
- 如果推断出的该 lambda 的返回类型不是 `Unit`，那么该 lambda 主体中的最后一个（或可能是单个）表达式会视为返回值。

如果将所有可选标注都留下，看起来如下：

```
val sum = { x: Int, y: Int -> x + y }
```

传递末尾的 lambda 表达式

按照 Kotlin 惯例，如果函数的最后一个参数是函数，那么作为相应参数传入的 lambda 表达式可以放在圆括号之外：

```
val product = items.fold(1) { acc, e -> acc * e }
```

这种语法也称为*拖尾 lambda 表达式*。

如果该 lambda 表达式是调用时唯一的参数，那么圆括号可以完全省略：

```
run { println("...") }
```

it：单个参数的隐式名称

一个 lambda 表达式只有一个参数很常见。

If the compiler can parse the signature without any parameters, the parameter does not need to be declared and `->` can be omitted. 该参数会隐式声明为 `it`：

```
ints.filter { it > 0 } // 这个字面值是“(it: Int) -> Boolean”类型的
```

从 lambda 表达式中返回一个值

可以使用限定的返回语法从 lambda 显式返回一个值。 否则，将隐式返回最后一个表达式的值。

因此，以下两个片段是等价的：

```
ints.filter {
    val shouldFilter = it > 0
    shouldFilter
}

ints.filter {
    val shouldFilter = it > 0
    return@filter shouldFilter
}
```

这一约定连同在圆括号外传递 lambda 表达式一起支持 LINQ-风格 的代码：

```
strings.filter { it.length == 5 }.sortedBy { it }.map { it.uppercase() }
```

下划线用于未使用的变量

如果 lambda 表达式的参数未使用，那么可以用下划线取代其名称：

```
map.forEach { (_, value) -> println("$value!") }
```

在 lambda 表达式中解构

在 lambda 表达式中解构是作为解构声明的一部分描述的。

匿名函数

上文的 lambda 表达式语法缺少一个东西——指定函数的返回类型的能力。在大多数情况下， 这是不必要的。因为返回类型可以自动推断出来。然而，如果确实需要显式指定， 可以使用另一种语法： *匿名函数*。

```
fun(x: Int, y: Int): Int = x + y
```

匿名函数看起来非常像一个常规函数声明，除了其名称省略了。其函数体既可以是表达式（如上所示）也可以是代码块：

```
fun(x: Int, y: Int): Int {
    return x + y
}
```

参数和返回类型的指定方式与常规函数相同，除了能够从上下文推断出的参数类型可以省略：

```
ints.filter(fun(item) = item > 0)
```

匿名函数的返回类型推断机制与正常函数一样：对于具有表达式函数体的匿名函数将自动推断返回类型，但具有代码块函数体的返回类型必须显式指定（或者已假定为 `Unit`）。

> **注意:** 当匿名函数作为参数传递时，需将其放在括号内。 允许将函数留在圆括号外的简写语法仅适用于 lambda 表达式。

Lambda表达式与匿名函数之间的另一个区别是非局部返回的行为。 一个不带标签的 `return` 语句总是在用 `fun` 关键字声明的函数中返回。这意味着 lambda 表达式中的 `return` 将从包含它的函数返回，而匿名函数中的 `return` 将从匿名函数自身返回。

闭包

Lambda 表达式或者匿名函数（以及局部函数和对象表达式） 可以访问其*闭包*，其中包含在外部作用域中声明的变量。 在 lambda 表达式中可以修改闭包中捕获的变量：

```
var sum = 0
ints.filter { it > 0 }.forEach {
    sum += it
}
print(sum)
```

带有接收者的函数字面值

带有接收者的函数类型，例如 `A.(B) -> C`，可以用特殊形式的函数字面值实例化——带有接收者的函数字面值。

如上所述，Kotlin 提供了（当提供*接收者对象*时）调用带有接收者的函数类型实例的能力。

在这样的函数字面值内部，传给调用的接收者对象成为*隐式*的 `this`，以便访问接收者对象的成员而无需任何额外的限定符，亦可使用 `this` 表达式 访问接收者对象。

这种行为与扩展函数类似，扩展函数也允许在函数体内部访问接收者对象的成员。

这里有一个带有接收者的函数字面值及其类型的示例，其中在接收者对象上调用了 `plus`：

```
val sum: Int.(Int) -> Int = { other -> plus(other) }
```

匿名函数语法允许你直接指定函数字面值的接收者类型。 如果你需要使用带接收者的函数类型声明一个变量，并在之后使用它，这会非常有用。

```
val sum = fun Int.(other: Int): Int = this + other
```

当接收者类型可以从上下文推断时，lambda 表达式可以用作带接收者的函数字面值。 One of the most important examples of their usage is type-safe builders:

```
class HTML {
  fun body() { …… }
}

fun html(init: HTML.() -> Unit): HTML {
  val html = HTML()  // 创建接收者对象
  html.init()        // 将该接收者对象传给该 lambda
  return html
}

html {       // 带接收者的 lambda 由此开始
  body()   // 调用该接收者对象的一个方法
}
```

内联函数

使用高阶函数会带来一些运行时的效率损失：每一个函数都是一个对象，并且会捕获一个闭包。 闭包那些在函数体内会访问到的变量的作用域。 内存分配（对于函数对象和类）和虚拟调用会引入运行时间开销。

但是在许多情况下通过内联化 lambda 表达式可以消除这类的开销。 下述函数是这种情况的很好的例子。 `lock()` 函数可以很容易地在调用处内联。 考虑下面的情况：

```
lock(l) { foo() }
```

编译器没有为参数创建一个函数对象并生成一个调用。取而代之，编译器可以生成以下代码：

```
l.lock()
try {
    foo()
} finally {
    l.unlock()
}
```

为了让编译器这么做，需要使用 `inline` 修饰符标记 `lock()` 函数：

```
inline fun <T> lock(lock: Lock, body: () -> T): T { …… }
```

`inline` 修饰符影响函数本身和传给它的 lambda 表达式：所有这些都将内联到调用处。

内联可能导致生成的代码增加。不过如果使用得当（避免内联过大函数）， 性能上会有所提升，尤其是在循环中的“超多态（megamorphic）”调用处。

noinline

如果不希望内联所有传给内联函数的 lambda 表达式参数都内联，那么可以用 `noinline` 修饰符标记不希望内联的函数参数：

```
inline fun foo(inlined: () -> Unit, noinline notInlined: () -> Unit) { …… }
```

可以内联的 lambda 表达式只能在内联函数内部调用或者作为可内联的参数传递。 但是 `noinline` 的 lambda 表达式可以以任何喜欢的方式操作，包括存储在字段中、或者进行传递。

> **注意:** 如果一个内联函数没有可内联的函数参数并且没有具体化的类型参数，编译器会产生一个警告，因为内联这样的函数很可能并无益处（如果你确认需要内联， 那么可以用 `@Suppress("NOTHING_TO_INLINE")` 注解关掉该警告）。

Non-local jump expressions

Returns

在 Kotlin 中，只能对具名或匿名函数使用正常的、非限定的 return 来退出。 要退出一个 lambda 表达式，需要使用一个标签。 在 lambda 表达式内部禁止使用裸 `return`，因为 lambda 表达式不能使包含它的函数返回：

```
fun ordinaryFunction(block: () -> Unit) {
    println("hi!")
}
fun foo() {
    ordinaryFunction {
        return // 错误：不能使 `foo` 在此处返回
    }
}
fun main() {
    foo()
}
```

但是如果 lambda 表达式传给的函数是内联的，该 return 也可以内联。因此可以这样：

```
inline fun inlined(block: () -> Unit) {
    println("hi!")
}
fun foo() {
    inlined {
        return // OK：该 lambda 表达式是内联的
    }
}
fun main() {
    foo()
}
```

这种返回（位于 lambda 表达式中，但退出包含它的函数）称为*非局部*返回。 通常会在循环中用到这种结构，其内联函数通常包含：

```
fun hasZeros(ints: List<Int>): Boolean {
    ints.forEach {
        if (it == 0) return true // 从 hasZeros 返回
    }
    return false
}
```

请注意，一些内联函数可能调用传给它们的不是直接来自函数体、而是来自另一个执行上下文的 lambda 表达式参数，例如来自局部对象或嵌套函数。在这种情况下，该 lambda 表达式中也不允许非局部控制流。To indicate that the lambda parameter of the inline function cannot use non-local returns, mark the lambda parameter with the `crossinline` modifier:

```
inline fun f(crossinline body: () -> Unit) {
    val f = object: Runnable {
        override fun run() = body()
    }
    // ……
}
```

Break and continue

> **警告:** This feature is currently In preview. We're planning to stabilize it in future releases. To opt in, use the `-Xnon-local-break-continue` compiler option. We would appreciate your feedback on it in YouTrack.

Similar to non-local `return`, you can apply `break` and `continue` jump expressions in lambdas passed as arguments to an inline function that encloses a loop:

```
fun processList(elements: List<Int>): Boolean {
    for (element in elements) {
        val variable = element.nullableMethod() ?: run {
            log.warning("Element is null or invalid, continuing...")
            continue
        }
        if (variable == 0) return true
    }
    return false
}
```

具体化的类型参数

有时候需要访问一个作为参数传递的一个类型:

```
fun <T> TreeNode.findParentOfType(clazz: Class<T>): T? {
    var p = parent
    while (p != null && !clazz.isInstance(p)) {
        p = p.parent
    }
    @Suppress("UNCHECKED_CAST")
    return p as T?
}
```

在这里向上遍历一棵树并且检测每个节点是不是特定的类型。 这都没有问题，但是调用处不是很优雅:

```
treeNode.findParentOfType(MyTreeNode::class.java)
```

更好的解决方案是只要传一个类型给该函数，可以按以下方式调用它:

```
treeNode.findParentOfType<MyTreeNode>()
```

为能够这么做，内联函数支持*具体化的类型参数*，于是可以这样写：

```
inline fun <reified T> TreeNode.findParentOfType(): T? {
    var p = parent
    while (p != null && p !is T) {
        p = p.parent
    }
    return p as T?
}
```

上述代码使用 `reified` 修饰符来限定类型参数使其可以在函数内部访问它， 几乎就像是一个普通的类一样。由于函数是内联的，不需要反射，正常的操作符如 `!is` 和 `as` 现在均可用。此外，还可以按照如上所示的方式调用该函数：`myTree.findParentOfType<MyTreeNodeType>()`。

虽然在许多情况下可能不需要反射，但仍然可以对一个具体化的类型参数使用它：

```
inline fun <reified T> membersOf() = T::class.members

fun main(s: Array<String>) {
    println(membersOf<StringBuilder>().joinToString("\n"))
}
```

普通的函数（未标记为内联函数的）不能有具体化参数。 不具有运行时表示的类型（例如非具体化的类型参数或者类似于 `Nothing` 的虚构类型）不能用作具体化的类型参数的实参。

内联属性

`inline` 修饰符可用于没有幕后字段的属性的访问器。 你可以标注独立的属性访问器：

```
val foo: Foo
    inline get() = Foo()

var bar: Bar
    get() = ……
    inline set(v) { …… }
```

你也可以标注整个属性，将它的两个访问器都标记为内联（`inline`）：

```
inline var bar: Bar
    get() = ……
    set(v) { …… }
```

在调用处，内联访问器如同内联函数一样内联。

公有 API 内联函数的限制

当一个内联函数是 `public` 或 `protected` 而不是 `private` 或 `internal` 声明的一部分时， 就会认为它是一个模块级的公有 API。可以在其他模块中调用它，并且也可以在调用处内联这样的调用。

这带来了一些由模块做这样变更时导致的二进制兼容的风险—— 声明一个内联函数但调用它的模块在它修改后并没有重新编译。

为了消除这种由*非*公有 API 变更引入的不兼容的风险，公有 API 内联函数体内不允许使用非公有声明，即，不允许使用 `private` 与 `internal` 声明以及其部件。

一个 `internal` 声明可以由 `@PublishedApi` 标注，这会允许它在公有 API 内联函数中使用。 当一个 `internal` 内联函数标记有 `@PublishedApi` 时，也会像公有函数一样检测其函数体。

操作符重载

在 Kotlin 中可以为类型提供预定义的一组操作符的自定义实现。这些操作符具有预定义的符号表示（如 `+` 或 `*`）与优先级。为了实现这样的操作符，需要为相应的类型提供一个指定名称的成员函数或扩展函数。这个类型会成为二元操作符左侧的类型及一元操作符的参数类型。

To overload an operator, mark the corresponding function with the `operator` modifier:

```
interface IndexedContainer {
    operator fun get(index: Int)
}
```

When overriding your operator overloads, you can omit `operator`:

```
class OrdersList: IndexedContainer {
    override fun get(index: Int) { /*...*/ }
}
```

一元操作

一元前缀操作符

表达式	翻译为
`+a`	`a.unaryPlus()`
`-a`	`a.unaryMinus()`
`!a`	`a.not()`

这个表是说，当编译器处理例如表达式 `+a` 时，它执行以下步骤：

- 确定 `a` 的类型，令其为 `T`。
- 为接收者 `T` 查找一个带有 `operator` 修饰符的无参函数 `unaryPlus()`，即成员函数或扩展函数。
- 如果函数不存在或不明确，则导致编译错误。
- 如果函数存在且其返回类型为 `R`，那就表达式 `+a` 具有类型 `R`。

  注意: 这些操作以及所有其他操作都针对基本类型做了优化，不会为它们引入函数调用的开销。

以下是如何重载一元减运算符的示例：

```
data class Point(val x: Int, val y: Int)

operator fun Point.unaryMinus() = Point(-x, -y)

val point = Point(10, 20)
```

```
fun main() {
    println(-point)  // 输出“Point(x=-10, y=-20)”
}
```

递增与递减

表达式	翻译为
`a++`	`a.inc()` + 见下文
`a--`	`a.dec()` + 见下文

`inc()` 和 `dec()` 函数必须返回一个值，它用于赋值给使用 `++` 或 `--` 操作的变量。它们不应该改变在其上调用 `inc()` 或 `dec()` 的对象。

编译器执行以下步骤来解析*后缀*形式的操作符，例如 `a++`：

- 确定 `a` 的类型，令其为 `T`。
- 查找一个适用于类型为 `T` 的接收者的、带有 `operator` 修饰符的无参数函数 `inc()`。
- 检测函数的返回类型是 `T` 的子类型。

计算表达式的步骤是：

- 把 `a` 的初始值存储到临时存储 `a0` 中。
- 把 `a0.inc()` 结果赋值给 `a`。
- 把 `a0` 作为该表达式的结果返回。

对于 `a--`，步骤是完全类似的。

对于*前缀*形式 `++a` 和 `--a` 以相同方式解析，其步骤是：

- 把 `a.inc()` 结果赋值给 `a`。
- 把 `a` 的新值作为该表达式结果返回。

二元操作

算术运算符

表达式	翻译为
`a + b`	`a.plus(b)`
`a - b`	`a.minus(b)`
`a * b`	`a.times(b)`
`a / b`	`a.div(b)`
`a % b`	`a.rem(b)`

表达式	翻译为
`a..b`	`a.rangeTo(b)`
`a..<b`	`a.rangeUntil(b)`

对于此表中的操作，编译器只是解析成*翻译为*列中的表达式。

下面是一个从给定值起始的 `Counter` 类的示例，它可以使用重载的 `+` 运算符来增加计数：

```
data class Counter(val dayIndex: Int) {
    operator fun plus(increment: Int): Counter {
        return Counter(dayIndex + increment)
    }
}
```

in 操作符

表达式	翻译为
`a in b`	`b.contains(a)`
`a !in b`	`!b.contains(a)`

对于 `in` 和 `!in`，过程是相同的，但是参数的顺序是相反的。

索引访问操作符

表达式	翻译为
`a[i]`	`a.get(i)`
`a[i, j]`	`a.get(i, j)`
`a[i_1, ……, i_n]`	`a.get(i_1, ……, i_n)`
`a[i] = b`	`a.set(i, b)`
`a[i, j] = b`	`a.set(i, j, b)`
`a[i_1, ……, i_n] = b`	`a.set(i_1, ……, i_n, b)`

方括号转换为调用带有适当数量参数的 `get` 和 `set`。

invoke 操作符

表达式	翻译为
`a()`	`a.invoke()`
`a(i)`	`a.invoke(i)`
`a(i, j)`	`a.invoke(i, j)`

表达式	翻译为
`a(i_1, ……, i_n)`	`a.invoke(i_1, ……, i_n)`

圆括号转换为调用带有适当数量参数的 `invoke`。

广义赋值

表达式	翻译为
`a += b`	`a.plusAssign(b)`
`a -= b`	`a.minusAssign(b)`
`a *= b`	`a.timesAssign(b)`
`a /= b`	`a.divAssign(b)`
`a %= b`	`a.remAssign(b)`

对于赋值操作，例如 `a += b`，编译器执行以下步骤：

- 如果右列的函数可用：
  - 如果相应的二元函数（即 `plusAssign()` 对应于 `plus()`）也可用， `a` is a mutable variable, and the return type of `plus` is a subtype of the type of `a`, 那么报告错误（无法区分）。
  - 确保其返回类型是 `Unit`，否则报告错误。
  - 生成 `a.plusAssign(b)` 的代码。
- 否则试着生成 `a = a + b` 的代码（这里包含类型检测：`a + b` 的类型必须是 `a` 的子类型）。

  注意: 赋值在 Kotlin 中*不是*表达式。

相等与不等操作符

表达式	翻译为
`a == b`	`a?.equals(b) ?: (b === null)`
`a != b`	`!(a?.equals(b) ?: (b === null))`

这些操作符只使用函数 `equals(other: Any?): Boolean`， 可以覆盖它来提供自定义的相等性检测实现。不会调用任何其他同名函数（如 `equals(other: Foo)`）。

  注意: `===` 和 `!==`（同一性检测）不可重载，因此不存在对他们的约定。

这个 `==` 操作符有些特殊：它被翻译成一个复杂的表达式，用于筛选 `null` 值。 `null == null` 总是 true，对于非空的 `x`， `x == null` 总是 false 而不会调用 `x.equals()`。

比较操作符

表达式	翻译为
`a > b`	`a.compareTo(b) > 0`
`a < b`	`a.compareTo(b) < 0`
`a >= b`	`a.compareTo(b) >= 0`
`a <= b`	`a.compareTo(b) <= 0`

所有的比较都转换为对 `compareTo` 的调用，这个函数需要返回 `Int` 值

属性委托操作符

`provideDelegate`、 `getValue` 以及 `setValue` 操作符函数已在委托属性中描述。

具名函数的中缀调用

可以通过中缀函数的调用 来模拟自定义中缀操作符。

类型安全的构建器

通过使用命名得当的函数作为构建器，结合带有接收者的函数字面值， 可以在 Kotlin 中创建类型安全、静态类型的构建器。

类型安全的构建器可以创建基于 Kotlin 的适用于采用半声明方式构建复杂层次数据结构领域专用语言（DSL）。以下是构建器的范例应用场景：

- 使用 Kotlin 代码生成标记语言，例如 HTML 或 XML
- 为 Web 服务器配置路由：Ktor

考虑下面的代码：

```
import com.example.html.* // 参见下文声明

fun result() =
    html {
        head {
            title {+"XML encoding with Kotlin"}
        }
        body {
            h1 {+"XML encoding with Kotlin"}
            p  {+"this format can be used as an alternative markup to XML"}

            // 一个具有属性和文本内容的元素
            a(href = "https://kotlinlang.org") {+"Kotlin"}

            // 混合的内容
            p {
                +"This is some"
                b {+"mixed"}
                +"text. For more see the"
                a(href = "https://kotlinlang.org") {+"Kotlin"}
                +"project"
            }
            p {+"some text"}

            // 以下代码生成的内容
            p {
                for (arg in args)
                    +arg
            }
        }
    }
```

这是完全合法的 Kotlin 代码。 可以在这里在线运行上文代码（修改它并在浏览器中运行）。

实现原理

Assume that you need to implement a type-safe builder in Kotlin. 首先，定义想要构建的模型。在本例中我们需要建模 HTML 标签。 用一些类就可以轻易完成。 例如，`HTML` 是一个描述 `<html>` 标签的类，它定义了像 `<head>` 和 `<body>` 这样的子标签。 （参见下文它的声明。）

现在，让我们回想下为什么可以在代码中这样写：

```
html {
 // ……
}
```

`html` 实际上是一个函数调用，它接受一个 lambda 表达式 作为参数。 该函数定义如下：

```
fun html(init: HTML.() -> Unit): HTML {
    val html = HTML()
    html.init()
    return html
}
```

这个函数接受一个名为 `init` 的参数，该参数本身就是一个函数。 该函数的类型是 `HTML.() -> Unit`，它是一个*带接收者的函数类型*。 这意味着需要向函数传递一个 HTML 类型的实例（*接收者*）， 并且可以在函数内部调用该实例的成员。

该接收者可以通过 `this` 关键字访问：

```
html {
    this.head { …… }
    this.body { …… }
}
```

（`head` 和 `body` 是 `HTML` 的成员函数。）

现在，像往常一样， `this` 可以省略掉了，得到的东西看起来已经非常像一个构建器了：

```
html {
    head { …… }
    body { …… }
}
```

那么，这个调用做什么？ 让我们看看上面定义的 `html` 函数的主体。 它创建了一个 `HTML` 的新实例，然后通过调用作为参数传入的函数来初始化它 （在本例中，归结为在HTML实例上调用 `head` 和 `body`），然后返回此实例。 这正是构建器所应做的。

`HTML` 类中的 `head` 和 `body` 函数的定义与 `html` 类似。 唯一的区别是，它们将构建的实例添加到包含 `HTML` 实例的 `children` 集合中：

```
fun head(init: Head.() -> Unit): Head {
    val head = Head()
    head.init()
    children.add(head)
    return head
}

fun body(init: Body.() -> Unit): Body {
    val body = Body()
    body.init()
    children.add(body)
    return body
}
```

实际上这两个函数做同样的事情，所以可以有一个泛型版本，`initTag`：

```
protected fun <T : Element> initTag(tag: T, init: T.() -> Unit): T {
    tag.init()
    children.add(tag)
    return tag
}
```

所以，现在该函数很简单：

```
fun head(init: Head.() -> Unit) = initTag(Head(), init)

fun body(init: Body.() -> Unit) = initTag(Body(), init)
```

并且可以使用它们来构建 `<head>` 和 `<body>` 标签。

这里要讨论的另一件事是如何向标签体中添加文本。在上例中这样写到：

```
html {
    head {
        title {+"XML encoding with Kotlin"}
    }
    // ……
}
```

所以基本上，只是把一个字符串放进一个标签体内部，但在它前面有一个小的 `+`， 所以它是一个函数调用，调用一个前缀 `unaryPlus()` 操作。 该操作实际上是由一个扩展函数 `unaryPlus()` 定义的，该函数是 `TagWithText` 抽象类（`Title` 的父类）的成员：

```
operator fun String.unaryPlus() {
    children.add(TextElement(this))
}
```

所以，在这里前缀 `+` 所做的事情是把一个字符串包装到一个 `TextElement` 实例中，并将其添加到 `children` 集合中， 以使其成为标签树的一个适当的部分。

所有这些都在上面构建器示例顶部导入的包 `com.example.html` 中定义。 在最后一节中，你可以阅读这个包的完整定义。

作用域控制：@DslMarker

使用 DSL 时，可能会遇到上下文中可以调用太多函数的问题。 可以调用 lambda 表达式内部每个可用的隐式接收者的方法，因此得到一个不一致的结果， 就像在另一个 `head` 内部的 `head` 标记那样：

```
html {
    head {
        head {} // 应该禁止
    }
    // ……
}
```

在这个例子中，必须只有最近层的隐式接收者 `this@head` 的成员可用；`head()` 是外部接收者 `this@html` 的成员，所以调用它一定是非法的。

为了解决这个问题，有一种控制接收者作用域的特殊机制。

为了使编译器开始控制标记，我们只是必须用相同的标记注解来标注在 DSL 中使用的所有接收者的类型。 例如，对于 HTML 构建器，我们声明一个注解 `@HTMLTagMarker`：

```
@DslMarker
annotation class HtmlTagMarker
```

如果一个注解类使用 `@DslMarker` 注解标注，那么该注解类称为 DSL 标记。

在我们的 DSL 中，所有标签类都扩展了相同的超类 `Tag`。 只需使用 `@HtmlTagMarker` 来标注超类就足够了，之后，Kotlin 编译器会将所有继承的类视为已标注：

```
@HtmlTagMarker
abstract class Tag(val name: String) { …… }
```

不必用 `@HtmlTagMarker` 标注 `HTML` 或 `Head` 类，因为它们的超类已标注过：

```
class HTML() : Tag("html") { …… }

class Head() : Tag("head") { …… }
```

在添加了这个注解之后，Kotlin 编译器就知道哪些隐式接收者是同一个 DSL 的一部分，并且只允许调用最近层的接收者的成员：

```
html {
    head {
        head { } // 错误：外部接收者的成员
    }
```

```
    // ……
}
```

请注意，仍然可以调用外部接收者的成员，但是要做到这一点，你必须明确指定这个接收者：

```
html {
    head {
        this@html.head { } // 可能
    }
    // ……
}
```

You can also apply the `@DslMarker` annotation directly to function types. Simply annotate the `@DslMarker` annotation with `@Target(AnnotationTarget.TYPE)`:

```
@Target(AnnotationTarget.TYPE)
@DslMarker
annotation class HtmlTagMarker
```

As a result, the `@DslMarker` annotation can be applied to function types, most commonly to lambdas with receivers. For example:

```
fun html(init: @HtmlTagMarker HTML.() -> Unit): HTML { ... }

fun HTML.head(init: @HtmlTagMarker Head.() -> Unit): Head { ... }

fun Head.title(init: @HtmlTagMarker Title.() -> Unit): Title { ... }
```

When you call these functions, the `@DslMarker` annotation restricts access to outer receivers in the body of a lambda marked with it unless you specify them explicitly:

```
html {
    head {
        title {
            // Access to title, head or other functions of outer receivers is restricted her
e.
        }
    }
}
```

Only the nearest receiver's members and extensions are accessible within a lambda, preventing unintended interactions between nested scopes.

com.example.html 包的完整定义

这就是 `com.example.html` 包的定义（只有上面例子中使用的元素）。 它构建一个 HTML 树。代码中大量使用了扩展函数和带有接收者的 lambda 表达式。

```kotlin
package com.example.html

interface Element {
    fun render(builder: StringBuilder, indent: String)
}

class TextElement(val text: String) : Element {
    override fun render(builder: StringBuilder, indent: String) {
        builder.append("$indent$text\n")
    }
}

@DslMarker
annotation class HtmlTagMarker

@HtmlTagMarker
abstract class Tag(val name: String) : Element {
    val children = arrayListOf<Element>()
    val attributes = hashMapOf<String, String>()

    protected fun <T : Element> initTag(tag: T, init: T.() -> Unit): T {
        tag.init()
        children.add(tag)
        return tag
    }

    override fun render(builder: StringBuilder, indent: String) {
        builder.append("$indent<$name${renderAttributes()}>\n")
        for (c in children) {
            c.render(builder, indent + "  ")
        }
        builder.append("$indent</$name>\n")
    }

    private fun renderAttributes(): String {
        val builder = StringBuilder()
        for ((attr, value) in attributes) {
            builder.append(" $attr=\"$value\"")
        }
        return builder.toString()
    }

    override fun toString(): String {
        val builder = StringBuilder()
        render(builder, "")
        return builder.toString()
    }
}

abstract class TagWithText(name: String) : Tag(name) {
    operator fun String.unaryPlus() {
        children.add(TextElement(this))
    }
}

class HTML : TagWithText("html") {
```

```
    fun head(init: Head.() -> Unit) = initTag(Head(), init)

    fun body(init: Body.() -> Unit) = initTag(Body(), init)
}

class Head : TagWithText("head") {
    fun title(init: Title.() -> Unit) = initTag(Title(), init)
}

class Title : TagWithText("title")

abstract class BodyTag(name: String) : TagWithText(name) {
    fun b(init: B.() -> Unit) = initTag(B(), init)
    fun p(init: P.() -> Unit) = initTag(P(), init)
    fun h1(init: H1.() -> Unit) = initTag(H1(), init)
    fun a(href: String, init: A.() -> Unit) {
        val a = initTag(A(), init)
        a.href = href
    }
}

class Body : BodyTag("body")
class B : BodyTag("b")
class P : BodyTag("p")
class H1 : BodyTag("h1")

class A : BodyTag("a") {
    var href: String
        get() = attributes["href"]!!
        set(value) {
            attributes["href"] = value
        }
}

fun html(init: HTML.() -> Unit): HTML {
    val html = HTML()
    html.init()
    return html
}
```

空安全

Null safety is a Kotlin feature designed to significantly reduce the risk of null references, also known as The Billion-Dollar Mistake.

许多编程语言（包括 Java）中最常见的陷阱之一，就是访问空引用的成员会导致空引用异常。在 Java 中，这等同于 `NullPointerException` 或简称 *NPE*。

Kotlin explicitly supports nullability as part of its type system, meaning you can explicitly declare which variables or properties are allowed to be `null`. Also, when you declare non-null variables, the compiler enforces that these variables cannot hold a `null` value, preventing an NPE.

Kotlin's null safety ensures safer code by catching potential null-related issues at compile time rather than runtime. This feature improves code robustness, readability, and maintainability by explicitly expressing `null` values, making the code easier to understand and manage.

Kotlin 中 NPE 的可能的原因只可能是：

- 显式调用 `throw NullPointerException()`。
- 使用了非空断言操作符 `!!`。
- 数据在初始化期间不一致，例如当：
  - 一个在构造函数中出现的未初始化的 `this` 用于其他地方（“泄漏 `this`”）。
  - 超类的构造函数调用一个开放成员，该成员在派生类中的实现使用了未初始化的状态。
- Java 互操作：
  - 企图访问平台类型的 `null` 引用的成员。
  - 泛型类型的可空性问题。例如，一段 Java 代码可能会向 Kotlin 的 `MutableList<String>` 中添加 `null`，这需要 `MutableList<String?>` 才能正确处理。
  - 由外部 Java 代码引发的其他问题。

  提示： Besides NPE, another exception related to null safety is `UninitializedPropertyAccessException`. Kotlin throws this exception when you try to access a property that has not been initialized, ensuring that non-nullable properties are not used until they are ready. This typically happens with `lateinit` properties.

可空类型与非空类型

在 Kotlin 中，类型系统区分一个类型可以容纳 `null`（可空类型）还是 不能容纳（非空类型）。例如，String 类型的常规变量不能容纳 `null`：

```
fun main() {
    // Assigns a non-null string to a variable
    var a: String = "abc"
    // Attempts to re-assign null to the non-nullable variable
```

```
    a = null
    print(a)
    // Null can not be a value of a non-null type String
}
```

You can safely call a method or access a property on `a`. It's guaranteed not to cause an NPE because `a` is a non-nullable variable. The compiler ensures that `a` always holds a valid `String` value, so there's no risk of accessing its properties or methods when it's `null`:

```
fun main() {
    // Assigns a non-null string to a variable
    val a: String = "abc"
    // Returns the length of a non-nullable variable
    val l = a.length
    print(l)
    // 3
}
```

To allow `null` values, declare a variable with a `?` sign right after the variable type. For example, you can declare a nullable string by writing `String?`. This expression makes `String` a type that can accept `null`:

```
fun main() {
    // Assigns a nullable string to a variable
    var b: String? = "abc"
    // Successfully re-assigns null to the nullable variable
    b = null
    print(b)
    // null
}
```

If you try accessing `length` directly on `b`, the compiler reports an error. This is because `b` is declared as a nullable variable and can hold `null` values. Attempting to access properties on nullables directly leads to an NPE:

```
fun main() {
    // Assigns a nullable string to a variable
    var b: String? = "abc"
    // Re-assigns null to the nullable variable
    b = null
    // Tries to directly return the length of a nullable variable
    val l = b.length
    print(l)
    // Only safe (?.) or non-null asserted (!!.) calls are allowed on a nullable receiver of
type String?
}
```

In the example above, the compiler requires you to use safe calls to check for nullability before accessing properties or performing operations. There are several ways to handle nullables:

- Check for `null` with the `if` conditional
- Safe call operator `?.`
- Elvis operator `?:`
- Not-null assertion operator `!!`
- Nullable receiver
- `let` function
- Safe casts `as?`
- Collections of a nullable type

Read the next sections for details and examples of `null` handling tools and techniques.

Check for null with the if conditional

When working with nullable types, you need to handle nullability safely to avoid an NPE. One way to handle this is checking for nullability explicitly with the `if` conditional expression.

For example, check whether `b` is `null` and then access `b.length`:

```
fun main() {
    // Assigns null to a nullable variable
    val b: String? = null
    // Checks for nullability first and then accesses length
    val l = if (b != null) b.length else -1
    print(l)
    // -1
}
```

In the example above, the compiler performs a smart cast to change the type from nullable `String?` to non-nullable `String`. 它还会跟踪所执行检测的相关信息，并允许在 `if` 条件内部调用 `length`。

同时，也支持更复杂的条件：

```
fun main() {
    // Assigns a nullable string to a variable
    val b: String? = "Kotlin"

    // Checks for nullability first and then accesses length
    if (b != null && b.length > 0) {
        print("String of length ${b.length}")
        // String of length 6
    } else {
        // Provides alternative if the condition is not met
        print("Empty string")
    }
}
```

Note that the example above only works when the compiler can guarantee that `b` doesn't change between the check and its usage, same as the smart cast prerequisites.

Safe call operator

The safe call operator `?.` allows you to handle nullability safely in a shorter form. Instead of throwing an NPE, if the object is `null`, the `?.` operator simply returns `null`:

```
fun main() {
    // Assigns a nullable string to a variable
    val a: String? = "Kotlin"
    // Assigns null to a nullable variable
    val b: String? = null

    // Checks for nullability and returns length or null
    println(a?.length)
    // 6
    println(b?.length)
    // null
}
```

The `b?.length` expression checks for nullability and 如果 `b` 非空，就返回 `b.length`，否则返回 `null`，这个表达式的类型是 `Int?`。

You can use the `?.` operator with both `var` and `val` variables in Kotlin:

- A nullable `var` can hold a `null` (for example, `var nullableValue: String? = null`) or a non-null value (for example, `var nullableValue: String? = "Kotlin"`). If it's a non-null value, you can change it to `null` at any point.
- A nullable `val` can hold a `null` (for example, `val nullableValue: String? = null`) or a non-null value (for example, `val nullableValue: String? = "Kotlin"`). If it's a non-null value, you cannot change it to `null` subsequently.

安全调用在链式调用中很有用。例如，一个员工 Bob 可能会（或者不会）分配给一个部门。 可能有另外一个员工是该部门的负责人。获取 Bob 所在部门负责人（如果有的话）的名字， 写作：

```
bob?.department?.head?.name
```

如果链中的任何属性（环节）为 `null`，这个链式调用就会返回 `null`。

也可以将安全调用放在赋值的左侧：

```
person?.department?.head = managersPool.getManager()
```

In the example above, if one of the receivers in the safe call chain is `null`, the assignment is skipped, and the expression on the right is not evaluated at all. For example, if either `person` or `person.department` is `null`, the function is not called. Here's the equivalent of the same safe call but with the `if` conditional:

```
if (person != null && person.department != null) {
    person.department.head = managersPool.getManager()
}
```

Elvis 操作符

When working with nullable types, you can check for `null` and provide an alternative value. For example, if `b` is not `null`, access `b.length`. Otherwise, return an alternative value:

```
fun main() {
    // Assigns null to a nullable variable
    val b: String? = null
    // Checks for nullability. If not null, returns length. If null, returns 0
    val l: Int = if (b != null) b.length else 0
    println(l)
    // 0
}
```

除了写完整的 `if` 表达式，还可以使用 Elvis 操作符 `?:` 以更简洁的方式处理：

```
fun main() {
    // Assigns null to a nullable variable
    val b: String? = null
    // Checks for nullability. If not null, returns length. If null, returns a non-null valu
e
    val l = b?.length ?: 0
    println(l)
    // 0
}
```

如果 `?:` 左侧表达式不是 `null`，Elvis 操作符就返回其左侧表达式。否则，Elvis 操作符返回右侧表达式。当且仅当左侧为 `null` 时，才会对右侧表达式求值。

因为 `throw` 和 `return` 在 Kotlin 中都是表达式，也可以将它们用在 elvis 操作符右侧。这可能会很方便，例如，检测函数参数：

```
fun foo(node: Node): String? {
    // Checks for getParent(). If not null, it's assigned to parent. If null, returns null
    val parent = node.getParent() ?: return null
    // Checks for getName(). If not null, it's assigned to name. If null, throws exception
    val name = node.getName() ?: throw IllegalArgumentException("name expected")
    // ……
}
```

非空断言操作符

非空断言操作符（`!!`）将任何值转换为非空类型。

When you apply the `!!` operator to a variable whose value is not `null`, it's safely handled as a non-nullable type, and the code executes normally. However, if the value is `null`, the `!!` operator forces it to be treated as non-nullable, which results in an NPE.

When `b` is not `null` and the `!!` operator makes it return its non-null value (which is a `String` in this example), it accesses `length` correctly:

```
fun main() {
    // Assigns a nullable string to a variable
    val b: String? = "Kotlin"
    // Treats b as non-null and accesses its length
    val l = b!!.length
    println(l)
    // 6
}
```

When `b` is `null` and the `!!` operator makes it return its non-null value, and an NPE occurs:

```
fun main() {
    // Assigns null to a nullable variable
    val b: String? = null
    // Treats b as non-null and tries to access its length
    val l = b!!.length
    println(l)
    // Exception in thread "main" java.lang.NullPointerException
}
```

The `!!` operator is particularly useful when you are confident that a value is not `null` and there's no chance of getting an NPE, but the compiler cannot guarantee this due to certain rules. In such cases, you can use the `!!` operator to explicitly tell the compiler that the value is not `null`.

Nullable receiver

You can use extension functions with a nullable receiver type, allowing these functions to be called on variables that might be `null`.

By defining an extension function on a nullable receiver type, you can handle `null` values within the function itself instead of checking for `null` at every place where you call the function.

For example, the `.toString()` extension function can be called on a nullable receiver. When invoked on a `null` value, it safely returns the string `"null"` without throwing an exception:

```
fun main() {
    // Assigns null to a nullable Person object stored in the person variable
    val person: Person? = null

    // Applies .toString to the nullable person variable and prints a string
    println(person.toString())
```

```
    // null
}

// Defines a simple Person class
data class Person(val name: String)
```

In the example above, even though `person` is `null`, the `.toString()` function safely returns the string `"null"`. This can be helpful for debugging and logging.

If you expect the `.toString()` function to return a nullable string (either a string representation or `null`), use the safe-call operator `?.`. The `?.` operator calls `.toString()` only if the object is not `null`, otherwise it returns `null`:

```
fun main() {
    // Assigns a nullable Person object to a variable
    val person1: Person? = null
    val person2: Person? = Person("Alice")

    // Prints "null" if person is null; otherwise prints the result of person.toString()
    println(person1?.toString())
    // null
    println(person2?.toString())
    // Person(name=Alice)
}

// Defines a Person class
data class Person(val name: String)
```

The `?.` operator allows you to safely handle potential `null` values while still accessing properties or functions of objects that might be `null`.

Let function

To handle `null` values and perform operations only on non-null types, you can use the safe call operator `?.` together with the `let` function.

This combination is useful for evaluating an expression, check the result for `null`, and execute code only if it's not `null`, avoiding manual null checks:

```
fun main() {
    // Declares a list of nullable strings
    val listWithNulls: List<String?> = listOf("Kotlin", null)

    // Iterates over each item in the list
    for (item in listWithNulls) {
        // Checks if the item is null and only prints non-null values
        item?.let { println(it) }
        //Kotlin
```

```
    }
}
```

安全的类型转换

The regular Kotlin operator for type casts is the `as` operator. However, regular casts can result in an exception if the object is not of the target type.

You can use the `as?` operator for safe casts. It tries to cast a value to the specified type and returns `null` if the value is not of that type:

```
fun main() {
    // Declares a variable of type Any, which can hold any type of value
    val a: Any = "Hello, Kotlin!"

    // Safe casts to Int using the 'as?' operator
    val aInt: Int? = a as? Int
    // Safe casts to String using the 'as?' operator
    val aString: String? = a as? String

    println(aInt)
    // null
    println(aString)
    // "Hello, Kotlin!"
}
```

The code above prints `null` because `a` is not an `Int`, so the cast fails safely. It also prints `"Hello, Kotlin!"` because it matches the `String?` type, so the safe cast succeeds.

可空类型的集合

如果你有一个包含可空元素的集合，并且只想保留其中非空的元素，可使用 filterNotNull() 函数：

```
fun main() {
    // Declares a list containing some null and non-null integer values
    val nullableList: List<Int?> = listOf(1, 2, null, 4)

    // Filters out null values, resulting in a list of non-null integers
    val intList: List<Int> = nullableList.filterNotNull()

    println(intList)
    // [1, 2, 4]
}
```

下一步做什么？

- 了解如何在 Java 与 Kotlin 中处理可空性。

- Learn about generic types that are definitely non-nullable.

相等性

Kotlin 中有两类相等性：

- *结构*相等（`==`——用 `equals()` 检测）
- *引用*相等（`===`——检测两个引用指向同一对象）

结构相等

Structural equality verifies if two objects have the same content or structure. 结构相等由 `==` 操作 及其否定形式 `!=` 操作判断。 按照约定，像 `a == b` 这样的表达式会翻译成：

```
a?.equals(b) ?: (b === null)
```

如果 `a` 不是 `null` 则调用 `equals(Any?)` 函数。否则（`a` 是 `null`），检测 `b` 是否与`null` 引用相等：

```
fun main() {
    var a = "hello"
    var b = "hello"
    var c = null
    var d = null
    var e = d

    println(a == b)
    // true
    println(a == c)
    // false
    println(c == e)
    // true
}
```

请注意，当与 `null` 显式比较时完全没必要优化你的代码： `a == null` 会被自动转换为 `a === null`。

In Kotlin, the `equals()` function is inherited by all classes from the `Any` class. By default, the `equals()` function implements referential equality. However, classes in Kotlin can override the `equals()` function to provide a custom equality logic and, in this way, implement structural equality.

Value classes and data classes are two specific Kotlin types that automatically override the `equals()` function. That's why they implement structural equality by default.

However, in the case of data classes, if the `equals()` function is marked as `final` in the parent class, its behavior remains unchanged.

Distinctly, non-data classes (those not declared with the `data` modifier) do not override the `equals()` function by default. Instead, non-data classes implement referential equality behavior inherited from the `Any` class. To implement structural equality, non-data classes require a custom equality logic to override the `equals()` function.

如需提供自定义的相等检测实现，请覆盖 `equals(other: Any?): Boolean` 函数：

```
class Point(val x: Int, val y: Int) {
    override fun equals(other: Any?): Boolean {
        if (this === other) return true
        if (other !is Point) return false

        // Compares properties for structural equality
        return this.x == other.x && this.y == other.y
    }
}
```

> **注意：** When overriding the equals() function, you should also override the hashCode() function to keep consistency between equality and hashing and ensure a proper behavior of these functions.

名称相同但签名不同的函数（如 `equals(other: Foo)`）并不会影响操作符 `==` 与 `!=` 的相等性检测。

结构相等与 `Comparable<……>` 接口定义的比较无关，因此只有自定义的 `equals(Any?)` 实现可能会影响该操作符的行为。

引用相等

Referential equality verifies the memory addresses of two objects to determine if they are the same instance.

引用相等由 `===`（以及其否定形式 `!==`）操作判断。`a === b` 当且仅当 `a` 与 `b` 指向同一个对象时求值为 true：

```
fun main() {
    var a = "Hello"
    var b = a
    var c = "world"
    var d = "world"

    println(a === b)
    // true
    println(a === c)
    // false
    println(c === d)
    // true

}
```

对于运行时以原生类型表示的值（例如 `Int`），`===` 相等检测等价于 `==` 检测。

> **提示:** The referential equality is implemented differently in Kotlin/JS. For more information about equality, see the Kotlin/JS documentation.

浮点数相等性

当相等性检测的两个操作数都是静态已知的（可空或非空的）`Float` 或 `Double` 类型时，该检测遵循 IEEE 754 浮点数运算标准。

The behavior is different for operands that are not statically typed as floating-point numbers. In these cases, structural equality is implemented. As a result, checks with operands not statically typed as floating-point numbers differ from the IEEE standard. In this scenario:

- `NaN` 等于其自身
- `NaN` 比（包括 `POSITIVE_INFINITY` 在内的）任何其他元素都大
- `-0.0` 不等于 `0.0`

更多信息请参见浮点数比较。

数组相等性

如需比较两个数组是否具有相同顺序的相同元素，请使用 `contentEquals()`。

更多信息请参见比较数组。

this 表达式

表示当前的 *接收者* 可使用 `this` 表达式：

- 在类的成员中，`this` 指的是该类的当前对象。
- 在扩展函数或者带有接收者的函数字面值中，`this` 表示在点左侧传递的 *接收者* 参数。

如果 `this` 没有限定符，它指的是最内层的包含它的作用域。要引用其他作用域中的 `this`，请使用 *标签限定符*：

限定的 this

要访问来自外部作用域的 `this`（一个类 或者扩展函数， 或者带标签的带有接收者的函数字面值）使用 `this@label`， 其中 `@label` 是一个代指 `this` 来源的标签：

```
class A { // 隐式标签 @A
    inner class B { // 隐式标签 @B
        fun Int.foo() { // 隐式标签 @foo
            val a = this@A // A 的 this
            val b = this@B // B 的 this

            val c = this // foo() 的接收者，一个 Int
            val c1 = this@foo // foo() 的接收者，一个 Int

            val funLit = lambda@ fun String.() {
                val d = this // funLit 的接收者，一个 String
            }

            val funLit2 = { s: String ->
                // foo() 的接收者，因为它包含的 lambda 表达式
                // 没有任何接收者
                val d1 = this
            }
        }
    }
}
```

Implicit this

当对 `this` 调用成员函数时，可以省略 `this.` 部分。 但是如果有一个同名的非成员函数时，请谨慎使用，因为在某些情况下会调用同名的非成员：

```
fun main() {
    fun printLine() { println("Local function") }

  class A {
    fun printLine() { println("Member function") }
```

```
        fun invokePrintLine(omitThis: Boolean = false) {
      if (omitThis) printLine()
      else this.printLine()
    }
  }

  A().invokePrintLine() // Member function
    A().invokePrintLine(omitThis = true) // Local function
}
```

异步程序设计

几十年以来，作为开发人员，我们面临着需要解决的问题——如何防止我们的应用程序被阻塞。 当我们正在开发桌面应用，移动应用，甚至服务器端应用程序时，我们希望避免让用户等待或导致更糟糕的原因成为阻碍应用程序扩展的瓶颈。

有很多途径来解决这种问题，包括：

- 线程
- 回调
- Future、 Promise 及其他
- 反应式扩展
- 协程

在解释协程的含义之前，让我们简要回顾一些其他解决方案。

线程

到目前为止，线程可能是最常见的避免应用程序阻塞的方法。

```
fun postItem(item: Item) {
    val token = preparePost()
    val post = submitPost(token, item)
    processPost(post)
}

fun preparePost(): Token {
    // 发起请求并因此阻塞了主线程
    return token
}
```

让我们假设在上面的代码中，`preparePost` 是一个长时间运行的进程，因此会阻塞用户界面。我们可以做的是在一个单独的线程中启动它。这样就可以允许我们避免阻塞 UI。这是一种非常常见的技术，但有一系列缺点：

- 线程并非廉价的。线程需要昂贵的上下文切换。
- 线程不是无限的。可被启动的线程数受底层操作系统的限制。在服务器端应用程序中，这可能会导致严重的瓶颈。
- 线程并不总是可用。在一些平台中，比如 JavaScript 甚至不支持线程。
- 线程不容易使用。调试线程与避免竞争条件是我们在多线程编程中遇到的常见问题。

回调

使用回调，其想法是将一个函数作为参数传递给另一个函数，并在处理完成后调用此函数。

```
fun postItem(item: Item) {
    preparePostAsync { token ->
        submitPostAsync(token, item) { post ->
            processPost(post)
        }
    }
}

fun preparePostAsync(callback: (Token) -> Unit) {
    // 发起请求并立即返回
    // 设置稍后调用的回调
}
```

原则上这感觉就像一个更优雅的解决方案，但又有几个问题：

- 回调嵌套的难度。通常被用作回调的函数，经常最终需要自己的回调。这导致了一系列回调嵌套并导致出现难以理解的代码。该模式通常被称为回调地狱，or the pyramid of doom due to the triangular shape that indentations from these deeply nested callbacks create.
- 错误处理很复杂。嵌套模型使错误处理和传播变得更加复杂。

回调在诸如 JavaScript 之类的事件循环体系结构中非常常见，但即使在那里，通常人们已经转而使用其他方法，例如 promises 或反应式扩展。

Future、 Promise 及其他

futures 或 promises 背后的想法（根据语言或平台的不同可能会使用其他术语），是当我们发起调用的时候，对我们*承诺*在某些时候该调用会返回一个 `Promise` 对象，然后我们可以对其进行操作。

```
fun postItem(item: Item) {
    preparePostAsync()
        .thenCompose { token ->
            submitPostAsync(token, item)
        }
        .thenAccept { post ->
            processPost(post)
        }

}

fun preparePostAsync(): Promise<Token> {
    // 发起请求并当稍后的请求完成时返回一个 promise
    return promise
}
```

这种方法需要对我们的编程方式进行一系列更改，尤其是：

- 不同的编程模型。与回调类似，编程模型从自上而下的命令式方法转变为具有链式调用的组合模型。传统的编程结构例如循环，异常处理，等等。通常在此模型中不再有效。
- 不同的 API。通常这需要学习完整的新 API 诸如 `thenCompose` 或 `thenAccept`，这也可能因平台而异。

- 具体的返回值类型。返回类型远离我们需要的实际数据，而是返回一个必须被内省的新类型“Promise”。
- 异常处理会很复杂。错误的传播和链接并不总是直截了当的。

反应式扩展

Erik Meijer 将反应式扩展（Rx）引入了 C#. 虽然它在 .NET 平台上是毫无疑义的， 但是在 Netflix 将它移植到 Java 并取名为 RxJava 之前绝对不是主流。从那时起，反应式被移植到各种平台，包括 JavaScript（RxJS）。

Rx 背后的想法是走向所谓的“可观察流”，我们现在将数据视为流（无限量的数据），并且可以观察到这些流。 实际上，Rx 很简单， Observer Pattern 带有一系列扩展，允许我们对数据进行操作。

在方法上它与 Futures 非常相似，但是人们可以将 Future 视为一个离散元素，而 Rx 返回一个流。然而，与前面类似，它还介绍了一种全新的思考我们的编程模型的方式，著名的表述是：

> “一切都是流，并且它是可被观察的”

这意味着处理问题的方式不同，并且在编写同步代码时从我们使用的方式发生了相当大的转变。与 Futures 相反的一个好处是，它被移植到这么多平台，通常我们可以找到一致的 API 体验，无论我们使用 C#、Java、JavaScript，还是 Rx 可用的任何其他语言。

此外，Rx 确实引入了一种更好的错误处理方法。

协程

Kotlin 编写异步代码的方式是使用协程，这是一种计算可被挂起的想法。即一种函数可以在某个时刻暂停执行并稍后恢复的想法。

协程的一个好处是，当涉及到开发人员时，编写非阻塞代码与编写阻塞代码基本相同。编程模型本身并没有真正改变。

以下面的代码为例：

```
fun postItem(item: Item) {
    launch {
        val token = preparePost()
        val post = submitPost(token, item)
        processPost(post)
    }
}

suspend fun preparePost(): Token {
    // 发起请求并挂起该协程
    return suspendCoroutine { /* ... */ }
}
```

此代码将启动长时间运行的操作，而不会阻塞主线程。`preparePost` 就是所谓的 `可挂起的函数`，因此它含有 `suspend` 前缀。这意味着如上所述，该函数将被执行、暂停执行以及在某个时间点恢复。

- 该函数的签名保持完全相同。唯一的不同是它被添加了 `suspend` 修饰符。但是返回类型依然是我们想要的类型。
- 编写这段代码代码就好像我们正在编写同步代码，自上而下，不需要任何特殊语法，除了使用一个名为 `launch` 的函数，它实质上启动了该协程（在其他教程中介绍）。
- 编程模型和 API 保持不变。我们可以继续使用循环，异常处理等，而且不需要学习一整套新的 API。
- 它与平台无关。无论我们是面向 JVM，JavaScript 还是其他任何平台，我们编写的代码都是相同的。编译器负责将其适应每个平台。

协程并不是一个新的概念，它并不是 Kotlin 发明的。它们已经存在了几十年，并且在 Go 等其他一些编程语言中很受欢迎。但重要的是要注意就是他们在 Kotlin 中实现的方式，大部分功能都委托给了库。事实上，除了 `suspend` 关键字，没有任何其他关键字被添加到语言中。这也是与其他语言的不同之处，例如 C# 将 `async` 以及 `await` 作为语法的一部分。而在 Kotlin 中，他们都只是库函数。

For more information, see the Coroutines reference.

注解

注解是将元数据附加到代码的方法。要声明注解，请将 `annotation` 修饰符放在类的前面：

```
annotation class Fancy
```

注解的附加属性可以通过用元注解标注注解类来指定：

- `@Target` 指定可以用该注解标注的元素的可能的类型（类、函数、属性与表达式）；
- `@Retention` 指定该注解是否存储在编译后的 class 文件中，以及它在运行时能否通过反射可见 （默认都是 true）；
- `@Repeatable` 允许在单个元素上多次使用相同的该注解；
- `@MustBeDocumented` 指定该注解是公有 API 的一部分，并且应该包含在生成的 API 文档中显示的类或方法的签名中。

```
@Target(AnnotationTarget.CLASS, AnnotationTarget.FUNCTION,
        AnnotationTarget.TYPE_PARAMETER, AnnotationTarget.VALUE_PARAMETER,
        AnnotationTarget.EXPRESSION)
@Retention(AnnotationRetention.SOURCE)
@MustBeDocumented
annotation class Fancy
```

用法

```
@Fancy class Foo {
    @Fancy fun baz(@Fancy foo: Int): Int {
        return (@Fancy 1)
    }
}
```

如果需要对类的主构造函数进行标注，则需要在构造函数声明中添加 `constructor` 关键字，并将注解添加到其前面：

```
class Foo @Inject constructor(dependency: MyDependency) { …… }
```

你也可以标注属性访问器：

```
class Foo {
    var x: MyDependency? = null
        @Inject set
}
```

构造函数

注解可以有接受参数的构造函数。

```
annotation class Special(val why: String)

@Special("example") class Foo {}
```

允许的参数类型有：

- 对应于 Java 原生类型的类型（Int、 Long等）
- 字符串
- 类（`Foo::class`）
- 枚举
- 其他注解
- 上面已列类型的数组

注解参数不能有可空类型，因为 JVM 不支持将 `null` 作为注解属性的值存储。

如果注解用作另一个注解的参数，则其名称不以 `@` 字符为前缀：

```
annotation class ReplaceWith(val expression: String)

annotation class Deprecated(
        val message: String,
        val replaceWith: ReplaceWith = ReplaceWith(""))

@Deprecated("This function is deprecated, use === instead", ReplaceWith("this === other"))
```

如果需要将一个类指定为注解的参数，请使用 Kotlin 类 （KClass）。Kotlin 编译器会自动将其转换为 Java 类，以便 Java 代码能够正常访问该注解与参数 。

```

import kotlin.reflect.KClass

annotation class Ann(val arg1: KClass<*>, val arg2: KClass<out Any>)

@Ann(String::class, Int::class) class MyClass
```

Instantiation

In Java, an annotation type is a form of an interface, so you can implement it and use an instance. As an alternative to this mechanism, Kotlin lets you call a constructor of an annotation class in arbitrary code and similarly use the resulting instance.

```
annotation class InfoMarker(val info: String)

```

```
fun processInfo(marker: InfoMarker): Unit = TODO()

fun main(args: Array<String>) {
    if (args.isNotEmpty())
        processInfo(getAnnotationReflective(args))
    else
        processInfo(InfoMarker("default"))
}
```

Learn more about instantiation of annotation classes in this KEEP.

Lambda 表达式

注解也可以用于 lambda 表达式。它们会被应用于生成 lambda 表达式体的 `invoke()` 方法上。这对于像 Quasar 这样的框架很有用， 该框架使用注解进行并发控制。

```
annotation class Suspendable

val f = @Suspendable { Fiber.sleep(10) }
```

注解使用处目标

当对属性或主构造函数参数进行标注时，从相应的 Kotlin 元素生成的 Java 元素会有多个，因此在生成的 Java 字节码中该注解有多个可能位置 。如果要指定精确地指定应该如何生成该注解，请使用以下语法：

```
class Example(@field:Ann val foo,    // 标注 Java 字段
              @get:Ann val bar,      // 标注 Java getter
              @param:Ann val quux)   // 标注 Java 构造函数参数
```

可以使用相同的语法来标注整个文件。 要做到这一点，把带有目标 `file` 的注解放在文件的顶层、package 指令之前或者在所有导入之前（如果文件在默认包中的话）：

```
@file:JvmName("Foo")

package org.jetbrains.demo
```

如果你对同一目标有多个注解，那么可以这样来避免目标重复——在目标后面添加方括号并将所有注解放在方括号内：

```
class Example {
    @set:[Inject VisibleForTesting]
    var collaborator: Collaborator
}
```

支持的使用处目标的完整列表为：

- `file`

- `property`（具有此目标的注解对 Java 不可见）
- `field`
- `get`（属性 getter）
- `set`（属性 setter）
- `receiver`（扩展函数或属性的接收者参数）
- `param`（构造函数参数）
- `setparam`（属性 setter 参数）
- `delegate`（为委托属性存储其委托实例的字段）

要标注扩展函数的接收者参数，请使用以下语法：

```
fun @receiver:Fancy String.myExtension() { ... }
```

如果不指定使用处目标，则根据正在使用的注解的 `@Target` 注解来选择目标 。如果有多个适用的目标，则使用以下列表中的第一个适用目标：

- `param`
- `property`
- `field`

Java 注解

Java 注解与 Kotlin 100% 兼容：

```
import org.junit.Test
import org.junit.Assert.*
import org.junit.Rule
import org.junit.rules.*

class Tests {
    // 将 @Rule 注解应用于属性 getter
    @get:Rule val tempFolder = TemporaryFolder()

    @Test fun simple() {
        val f = tempFolder.newFile()
        assertEquals(42, getTheAnswer())
    }
}
```

因为 Java 编写的注解没有定义参数顺序，所以不能使用常规函数调用语法来传递参数。相反，你需要使用具名实参语法：

```
// Java
public @interface Ann {
    int intValue();
    String stringValue();
}
```

```kotlin
// Kotlin
@Ann(intValue = 1, stringValue = "abc") class C
```

就像在 Java 中一样，一个特殊的情况是 `value` 参数；它的值无需显式名称指定：

```java
// Java
public @interface AnnWithValue {
    String value();
}
```

```kotlin
// Kotlin
@AnnWithValue("abc") class C
```

数组作为注解参数

如果 Java 中的 `value` 参数具有数组类型，它会成为 Kotlin 中的一个 `vararg` 参数：

```java
// Java
public @interface AnnWithArrayValue {
    String[] value();
}
```

```kotlin
// Kotlin
@AnnWithArrayValue("abc", "foo", "bar") class C
```

对于具有数组类型的其他参数，你需要显式使用数组字面值语法或者 `arrayOf(……)`：

```java
// Java
public @interface AnnWithArrayMethod {
    String[] names();
}
```

```kotlin
@AnnWithArrayMethod(names = ["abc", "foo", "bar"])
class C
```

访问注解实例的属性

注解实例的值会作为属性暴露给 Kotlin 代码：

```java
// Java
public @interface Ann {
    int value();
}
```

```kotlin
// Kotlin
fun foo(ann: Ann) {
    val i = ann.value
}
```

Ability to not generate JVM 1.8+ annotation targets

If a Kotlin annotation has `TYPE` among its Kotlin targets, the annotation maps to `java.lang.annotation.ElementType.TYPE_USE` in its list of Java annotation targets. This is just like how the `TYPE_PARAMETER` Kotlin target maps to the `java.lang.annotation.ElementType.TYPE_PARAMETER` Java target. This is an issue for Android clients with API levels less than 26, which don't have these targets in the API.

To avoid generating the `TYPE_USE` and `TYPE_PARAMETER` annotation targets, use the new compiler argument `-Xno-new-java-annotation-targets`.

Repeatable annotations

Just like in Java, Kotlin has repeatable annotations, which can be applied to a single code element multiple times. To make your annotation repeatable, mark its declaration with the `@kotlin.annotation.Repeatable` meta-annotation. This will make it repeatable both in Kotlin and Java. Java repeatable annotations are also supported from the Kotlin side.

The main difference with the scheme used in Java is the absence of a *containing annotation*, which the Kotlin compiler generates automatically with a predefined name. For an annotation in the example below, it will generate the containing annotation `@Tag.Container`:

```
@Repeatable
annotation class Tag(val name: String)

// The compiler generates the @Tag.Container containing annotation
```

You can set a custom name for a containing annotation by applying the `@kotlin.jvm.JvmRepeatable` meta-annotation and passing an explicitly declared containing annotation class as an argument:

```
@JvmRepeatable(Tags::class)
annotation class Tag(val name: String)

annotation class Tags(val value: Array<Tag>)
```

To extract Kotlin or Java repeatable annotations via reflection, use the `KAnnotatedElement.findAnnotations()` function.

Learn more about Kotlin repeatable annotations in this KEEP.

解构声明

有时把一个对象*解构*成很多变量会很方便，例如：

```
val (name, age) = person
```

这种语法称为*解构声明*。一个解构声明同时创建多个变量。 声明了两个新变量： `name` 和 `age`，并且可以独立使用它们：

```
println(name)
println(age)
```

一个解构声明会被编译成以下代码：

```
val name = person.component1()
val age = person.component2()
```

其中的 `component1()` 和 `component2()` 函数是在 Kotlin 中广泛使用的*约定原则*的另一个示例。 （参见像 `+` 和 `*`、`for`-循环作为示例）。 任何表达式都可以出现在解构声明的右侧，只要可以对它调用所需数量的 component 函数即可。 当然，可以有 `component3()` 和 `component4()` 等等。

> **注意:** `componentN()` 函数需要用 `operator` 关键字标记，以允许在解构声明中使用它们 。

解构声明也可以用在 `for`-循环中：

```
for ((a, b) in collection) { …… }
```

变量 `a` 和 `b` 的值取自对集合中的元素上调用 `component1()` 和 `component2()` 的返回值。

例：从函数中返回两个值

让我们假设我们需要从一个函数返回两个东西。例如，一个结果对象和一个某种状态。 在 Kotlin 中一个简洁的实现方式是声明一个数据类 并返回其实例：

```
data class Result(val result: Int, val status: Status)
fun function(……): Result {
    // 各种计算

    return Result(result, status)
}

// 现在，使用该函数：
val (result, status) = function(……)
```

因为数据类自动声明 `componentN()` 函数，所以这里可以用解构声明。

> **注意:** 我们也可以使用标准类 `Pair` 并且让 `function()` 返回 `Pair<Int, Status>`， 但是让数据合理命名通常更好。

例：解构声明与映射

可能遍历一个映射（map）最好的方式就是这样：

```
for ((key, value) in map) {
    // 使用该 key、value 做些事情
}
```

为使其能用，我们应该

- 通过提供一个 `iterator()` 函数将映射表示为一个值的序列。
- 通过提供函数 `component1()` 和 `component2()` 来将每个元素呈现为一对。

当然事实上，标准库提供了这样的扩展：

```
operator fun <K, V> Map<K, V>.iterator(): Iterator<Map.Entry<K, V>> = entrySet().iterator()
operator fun <K, V> Map.Entry<K, V>.component1() = getKey()
operator fun <K, V> Map.Entry<K, V>.component2() = getValue()
```

因此你可以在 `for`-循环中对映射（以及数据类实例的集合等）自由使用解构声明。

下划线用于未使用的变量

如果在解构声明中你不需要某个变量，那么可以用下划线取代其名称：

```
val (_, status) = getResult()
```

对于以这种方式跳过的组件，不会调用相应的 `componentN()` 操作符函数。

在 lambda 表达式中解构

你可以对 lambda 表达式参数使用解构声明语法。 如果 lambda 表达式具有 `Pair` 类型（或者 `Map.Entry` 或任何其他具有相应 `componentN` 函数的类型）的参数，那么可以通过将它们放在括号中来引入多个新参数来取代单个新参数：

```
map.mapValues { entry -> "${entry.value}!" }
map.mapValues { (key, value) -> "$value!" }
```

注意声明两个参数和声明一个解构对来取代单个参数之间的区别：

```
{ a //-> …… } // 一个参数
{ a, b //-> …… } // 两个参数
{ (a, b) //-> …… } // 一个解构对
{ (a, b), c //-> …… } // 一个解构对以及其他参数
```

如果解构的参数中的一个组件未使用，那么可以将其替换为下划线，以避免编造其名称：

```
map.mapValues { (_, value) -> "$value!" }
```

你可以指定整个解构的参数的类型或者分别指定特定组件的类型：

```
map.mapValues { (_, value): Map.Entry<Int, String> -> "$value!" }

map.mapValues { (_, value: String) -> "$value!" }
```

反射

*反射*是这样的一组语言和库功能，让你可以在运行时自省你的程序的结构。 Kotlin 中的函数和属性是一等公民，而对其自省（即在运行时获悉一个属性或函数的名称或类型）能力是使用函数式或反应式风格时所必需的。

> **注意:** Kotlin/JS provides limited support for reflection features. Learn more about reflection in Kotlin/JS.

JVM dependency

On the JVM platform, the Kotlin compiler distribution includes the runtime component required for using the reflection features as a separate artifact, `kotlin-reflect.jar`. This is done to reduce the required size of the runtime library for applications that do not use reflection features.

To use reflection in a Gradle or Maven project, add the dependency on `kotlin-reflect`:

- In Gradle:

Kotlin

```
```kotlin
dependencies {
    implementation(kotlin("reflect"))
}
```
```

Groovy

```
```groovy
dependencies {
    implementation "org.jetbrains.kotlin:kotlin-reflect:2.1.21"
}
```
```

- In Maven:

```
<dependencies>
    <dependency>
        <groupId>org.jetbrains.kotlin</groupId>
        <artifactId>kotlin-reflect</artifactId>
    </dependency>
</dependencies>
```

If you don't use Gradle or Maven, make sure you have `kotlin-reflect.jar` in the classpath of your project. In other supported cases (IntelliJ IDEA projects that use the command-line compiler or Ant), it is added by default. In the command-line compiler and Ant, you can use the `-no-reflect` compiler option to exclude `kotlin-reflect.jar` from the classpath.

类引用

最基本的反射功能是获取 Kotlin 类的运行时引用。要获取对静态已知的 Kotlin 类的引用，可以使用 *类字面值* 语法：

```
val c = MyClass::class
```

该引用是 KClass 类型的值。

> **注意：** 对于 JVM 平台：Kotlin 类引用与 Java 类引用不同。要获得 Java 类引用， 请在 `KClass` 实例上使用 `.java` 属性。

绑定的类引用

通过使用对象作为接收者，可以用相同的 `::class` 语法获取指定对象的类的引用：

```
val widget: Widget = ……
assert(widget is GoodWidget) { "Bad widget: ${widget::class.qualifiedName}" }
```

你会获得对象的精确类的引用，例如 `GoodWidget` 或 `BadWidget`， 尽管接收者表达式的类型是 `Widget`。

可调用引用

函数、属性以及构造函数的引用可以用于调用或者用作函数类型的实例。

所有可调用引用的公共超类型是 `KCallable<out R>`， 其中 `R` 是返回值类型。对于属性是属性类型，对于构造函数是所构造类型。

函数引用

当有一个具名函数声明如下， you can call it directly (`isOdd(5)`):

```
fun isOdd(x: Int) = x % 2 != 0
```

Alternatively, you can use the function as a function type value, that is, pass it to another function. To do so, use the `::` operator:

```
fun isOdd(x: Int) = x % 2 != 0

fun main() {
```

```
    val numbers = listOf(1, 2, 3)
    println(numbers.filter(::isOdd))
}
```

这里 `::isOdd` 是函数类型 `(Int) -> Boolean` 的一个值。

函数引用属于 `KFunction<out R>` 的子类型之一，取决于参数个数。例如 `KFunction3<T1, T2, T3, R>`。

当上下文中已知函数期望的类型时，`::` 可以用于重载函数。 例如：

```
fun main() {
    fun isOdd(x: Int) = x % 2 != 0
    fun isOdd(s: String) = s == "brillig" || s == "slithy" || s == "tove"

    val numbers = listOf(1, 2, 3)
    println(numbers.filter(::isOdd)) // 引用到 isOdd(x: Int)
}
```

或者，你可以通过将方法引用存储在具有显式指定类型的变量中来提供必要的上下文：

```
val predicate: (String) -> Boolean = ::isOdd   // 引用到 isOdd(x: String)
```

如果需要使用类的成员函数或扩展函数，它需要是限定的，例如 `String::toCharArray`。

即使以扩展函数的引用初始化一个变量，其推断出的函数类型也会没有接收者，但是它会有一个接受接收者对象的额外参数。如需改为带有接收者的函数类型，请明确指定其类型：

```
val isEmptyStringList: List<String>.() -> Boolean = List<String>::isEmpty
```

示例：函数组合

考虑以下函数：

```
fun <A, B, C> compose(f: (B) -> C, g: (A) -> B): (A) -> C {
    return { x -> f(g(x)) }
}
```

它返回一个传给它的两个函数的组合：`compose(f, g) = f(g(*))`。 你可以将该函数应用于可调用引用：

```
fun <A, B, C> compose(f: (B) -> C, g: (A) -> B): (A) -> C {
    return { x -> f(g(x)) }
}

fun isOdd(x: Int) = x % 2 != 0

fun main() {
    fun length(s: String) = s.length

    val oddLength = compose(::isOdd, ::length)
    val strings = listOf("a", "ab", "abc")
```

```
    println(strings.filter(oddLength))
}
```

属性引用

要把属性作为 Kotlin中 的一等对象来访问，可以使用 `::` 操作符：

```
val x = 1

fun main() {
    println(::x.get())
    println(::x.name)
}
```

表达式 `::x` 求值为 `KProperty0<Int>` 类型的属性对象，可以使用 `get()` 读取它的值，或者使用 `name` 属性来获取属性名。更多信息请参见关于 `KProperty` 类的文档。

对于可变属性，例如 `var y = 1`， `::y` 返回 `KMutableProperty0<Int>` 类型的一个值， 该类型有一个 `set()` 方法。

```
var y = 1

fun main() {
    ::y.set(2)
    println(y)
}
```

属性引用可以用在预期具有单个泛型参数的函数的地方：

```
fun main() {
    val strs = listOf("a", "bc", "def")
    println(strs.map(String::length))
}
```

要访问属于类的成员的属性，这样限定它：

```
fun main() {
    class A(val p: Int)
    val prop = A::p
    println(prop.get(A(1)))
}
```

对于扩展属性：

```
val String.lastChar: Char
    get() = this[length - 1]

fun main() {
```

```
    println(String::lastChar.get("abc"))
}
```

与 Java 反射的互操作性

在 JVM 平台上，标准库包含反射类的扩展，它提供了与 Java 反射对象之间映射（参见 `kotlin.reflect.jvm` 包）。 例如，要查找一个用作 Kotlin 属性 getter 的 幕后字段或 Java方法，可以这样写：

```
import kotlin.reflect.jvm.*

class A(val p: Int)

fun main() {
    println(A::p.javaGetter) // 输出 "public final int A.getP()"
    println(A::p.javaField)  // 输出 "private final int A.p"
}
```

要获得对应于 Java 类的 Kotlin 类，请使用 `.kotlin` 扩展属性：

```
fun getKClass(o: Any): KClass<Any> = o.javaClass.kotlin
```

构造函数引用

构造函数可以像方法和属性那样引用。可以将其用于程序期待这样函数类型对象的任何地方：它与该构造函数接受相同参数并且返回相应类型的对象。 通过使用 `::` 操作符并添加类名来引用构造函数。考虑下面的函数，它期待一个无参并返回 `Foo` 类型的函数参数：

```
class Foo

fun function(factory: () -> Foo) {
    val x: Foo = factory()
}
```

使用 `::Foo`，类 `Foo` 的零参数构造函数，可以这样调用它：

```
function(::Foo)
```

构造函数的可调用引用的类型也是 `KFunction<out R>` 的子类型之一， 取决于其参数个数。

绑定的函数与属性引用

你可以引用特定对象的实例方法：

```
fun main() {
    val numberRegex = "\\d+".toRegex()
    println(numberRegex.matches("29"))

    val isNumber = numberRegex::matches
```

```
    println(isNumber("29"))
}
```

取代直接调用方法 `matches` 的是使用其引用。 这样的引用会绑定到其接收者上。 它可以直接调用（如上例所示）或者用于任何期待一个函数类型表达式的时候：

```
fun main() {
    val numberRegex = "\\d+".toRegex()
    val strings = listOf("abc", "124", "a70")
    println(strings.filter(numberRegex::matches))
}
```

比较绑定的引用与未绑定的引用的类型。 绑定的可调用引用有其接收者“附加”到其上，因此接收者的类型不再是参数：

```
val isNumber: (CharSequence) -> Boolean = numberRegex::matches

val matches: (Regex, CharSequence) -> Boolean = Regex::matches
```

属性引用也可以绑定：

```
fun main() {
    val prop = "abc"::length
    println(prop.get())
}
```

无需显式指定 `this` 作为接收者：`this::foo` 与 `::foo` 是等价的。

绑定的构造函数引用

内部类的构造函数的绑定的可调用引用可通过提供外部类的实例来获得：

```
class Outer {
    inner class Inner
}

val o = Outer()
val boundInnerCtor = o::Inner
```

Android

Kotlin 用于 Android 开发

自 2019 年 Google I/O 以来，Kotlin 就成为了 Android 移动开发的首选。

超过 50% 的专业 Android 开发者使用 Kotlin 作为主要语言，而只有 30% 使用 Java 作为主要语言。 70% 以 Kotlin 为主要语言的开发人员表示，Kotlin 提高了他们的工作效率。

使用 Kotlin 进行 Android 开发，可以得益于：

- **代码更少、可读性更强**。 花更少的时间来编写代码与理解他人的代码 。
- **常见错误更少**。 根据 Google的内部数据，用 Kotlin 构建的应用崩溃的可能性低 20%。
- **Jetpack 库中的 Kotlin 支持**。 Jetpack Compose 是 Android 推荐的现代工具包，用于在 Kotlin 中构建原生 UI。 KTX 扩展为现有的 Android 库添加了 Kotlin 语言特性，如协程、 扩展函数、 lambda 表达式与命名参数。
- **支持多平台开发**。 Kotlin 多平台不仅支持 Android 开发，还支持 iOS、 后端以及 web 应用程序开发。 一些 Jetpack 库已经是多平台的了。 Compose 多平台是 JetBrains 基于 Kotlin 与 Jetpack Compose 的声明式 UI 框架， 能够跨平台（iOS、Android、桌面与 web）共享 UI。
- **成熟的语言与环境**。 自 2011 年创建以来，Kotlin 不仅通过语言而且通过强大的工具在整个生态系统中不断发展。 现在，它已无缝集成到 Android Studio 中， 并被许多公司积极用于开发 Android 应用程序。
- **与 Java 的互操作性**。 可以在应用程序中将 Kotlin 与 Java 编程语言一起使用， 而无需将所有代码迁移到 Kotlin。
- **易学易用**。Kotlin 非常易于学习，尤其是对于 Java 开发人员而言。
- **大社区**。 Kotlin 得到了社区的大力支持与许多贡献，该社区在全世界范围内都在增长 。 排名前一千的 Android 应用中超过 95% 使用 Kotlin。

许多初创公司与财富 500 强公司已经使用 Kotlin 开发了 Android 应用程序，参见面向 Android 开发者的谷歌网站上的列表。

开始将 Kotlin 用于：

- Android 开发，请阅读 Google 关于用 Kotlin 开发 Android 应用的文档。
- 开发跨平台移动应用程序，请参见创建具有共享逻辑和原生 UI 的应用程序。